

目录

目录

Python 学习心得

1 Python的基本介绍

1.1 Python的优势特点

1.2 本系列学习要点

1.3 Python的安装

1.4 从IDLE启动Python

1.5 课堂总结

1.5.1 Python 不兼容其他C语言等语法格式

1.5.2 Python的几个好处点

1.6 第一讲课后练习精选

2 用Python 设计第一个游戏

2.1制作步骤

2.2 名词解释

2.2.1 BIF == Built-in function 内置函数

2.2.2如何用Python查看帮助

2.3 第2讲课后练习精选

3 插曲之变量/字符串/改进我们的游戏

3.1 Python与其他语言关于变量的不同点

3.2 变量名使用规范

3.3 字符串

3.4 多\转义，跨越多行字符串

3.5 游戏改进

3.5.1 条件分支

3.5.2 while 循环

3.5.3 and 逻辑操作符

3.5.4 可以通过randint, 将8变成一个随机的数:

3.5.5 改进的结果

4 Python的数据类型/操作符

4.1 e记法

4.2 如何确定类型一致

4.2.1 type用法

4.2.2 isinstance 用法

4.3 数据类型课后习题精选

4.4 操作符

4.4.1 Python 地板除法 //

4.4.2 Python 幂运算 **

4.4.3 逻辑操作符 and 、or、 not

4.5 操作符的优先级问题

6 打了激素的数组

6.1 列表的创建

6.2 向列表添加元素

6.2.1 append添加方法

6.2.2 extend 添加列表

6.2.3 insert 插入一个元素

6.2 列表中的元素操作

6.2.1 元素索引

6.2.2 列表中元素删除 remove ()

6.2.3 del() 删除

6.2.4 pop 删除

6.2.5 列表的分片 (强大)

6.3列表的关系操作符

6.3.1 列表比大小

6.3.2 列表的拼接

6.3.3 列表的成员关系操作符

6.3.4 列表中的列表

6.3.5 计算列表内元素的数目

6.3.6 查找某元素出现的范围

6.3.8 列表的整排 (默认从小到大)

6.3.9 列表拷贝的补充

6.4 关进笼子的列表----元组

6.4.1 元组的创建和一些基本用法

6.4.2 元组的实质

6.4.3 元组和列表的不同之处

6.5 课后习题精选

6.5.1 上一题打印的样式不是很好, 能不能修改一下代码打印成下图的样式呢?

7 字符串

7.1 字符串的一些内置函数用法

7.2 字符串的格式化方法

7.2.1 位置参数和关键字参数 (format)

7.2.2 参数保留一位小数点的方法

7.2.2 格式化参数的介绍

7.2.3 格式化操作符辅助指令

7.3 序列

7.3.1 list 基本使用

7.3.2 与list相关的一些内置函数

8 函数

8.1 Python 函数的基本介绍

8.1.1 函数的创建和调用

8.1.2 函数的参数

8.2 函数的参数类型

8.2.1 形参和实参

8.2.2 查阅函数文档

8.2.3 关键字参数

8.2.4 默认参数

8.2.5 收集参数（牛逼）

8.3 函数变量的作用域/函数的返回值

8.3.1 函数和过程

8.3.2 Python是没有变量的，只有名字

8.3.3 Python 函数变量的作用域问题

8.4 全局和闭包的概念

8.4.1 global 关键字

8.4.2 内嵌函数

8.4.3 闭包

8.4.4 nonlocal 声明

8.5 lambda 函数

8.5.1 lambda 函数单参数使用

8.5.2 双参数的lambda函数的使用

8.5.3 filter 过滤器

8.5.4 map 函数 映射

9 递归算法/字典

9.1 递归的基本介绍

9.1.1 递归举例

9.1.2 递归求阶乘

9.1.3 斐波那契数列 Fibonacci

9.1.4 汉诺塔游戏

9.2 字典

9.2.1 创建和访问字典

9.2.2 fromkey 创建字典

9.2.3 访问字典的几个方法 (keys(), values(), items())

9.2.4 通过get的方法查询字典中的值

9.2.5 通过in 和 not in 检查字典中的成员存在与否

9.2.6 通过clear() 清空字典

9.2.7 通过copy()浅拷贝

9.2.8 字典里的pop选项, 和popitem选项

9.2.9 字典通过setdefault 增加项目

9.2.10 通过另一个字典 更新当前字典里面的选项 (update)

9.3 字典的表亲 集合

9.3.1 集合的特点 (集合里面的元素的唯一性)

9.3.2 如何创建一个集合

9.3.3 可以使用in / not in 判断集合内元素是否存在

9.3.4 用add 和remove 增加删减集合中的元素

9.3.5 frozen 不可变集合

10 文件

10.1 文件的基本介绍

10.1.1 文件的打开

10.1.2文件对象方法

10.2 模块概念

10.2.1 OS模块

10.2.2 os.path 模块

10.3 pickle 模块（重要）

10.3.1 举例说明

10.4 异常说明

10.4.1 常用异常

10.4.2 try-except 语句（检测异常）

10.4.3 try-finally 语句

10.4.4 raise 语句 （自己引发异常）

10.5 else语句搭配

10.5.1 while - else举例说明

10.5.2 Try-else 语句搭配

10.6 with 语句

10.6.1 with 语句作用

11 easygui 和类

11.1 easygui 学习

11.1.1 easygui 学习资料

11.1.2 easygui的使用

11.1.3 修改默认的宽高

11.2 面向对象编程

11.2.1 继承

11.2.2 多态

11.2.3 self 是什么?

11.2.3 __init__(self)构造方法 (自动调用)

11.2.4 公有和私有

11.2.5 super()直接调用

11.2.6 组合

11.3 一些与类相关的BIF

11.3.1 property 函数

12 魔法方法

12.1 魔法方法的概述

12.1.1 __init__ (self,...)构造和析构

12.2 类的算术运算

12.2.1 如何看出类可以算术运算?

12.2.2 把加法改写成减法

12.2.3 内置方法列表

12.2.4 反运算

12.2.5 属性访问

利用收集参数变动的去定义一个传入参数 (自己想的)

描述符知识点

容器类型的协议

迭代

生成器

模块

```
if __name__ == "__main__":
```

搜索路径

包

python网络爬虫篇

python 如何访问互联网

urllib 实际是一个包

从网站上爬取一张小猫的照片

利用有道词典翻译文本

有道词典F12的简单介绍

修改header操作

如何避开检查

如何爬取网页上的图片：

使用正则表达式：

IP地址用正则表达式表示：

\+ 普通字母构成的正则表达式：

正则表达式的编译：

正则表达式开启详细注释模式，verbose

正则表达式使用方法：

一般处理异常的写法：

scrapy抓取步骤：

scrapy框架

scrapy 实例应用

GUI 的终极选择 Tkinter

简单示例

如何导入一个图片，Label的用法

将文字插入到图片中间

button 组件

checkboxbutton 介绍使用

radiobutton的使用方法：

多重单选可以使用列表包含元素的形式实现

Python中的for语句可以设置2个变量，尤其在列表中是元组表示时表示一前一后。

entry组件

用列表的方式管理组件，非frame用法：

如何输入密码：

如何设置一个计算器

验证的函数应用：

冷却触发详解：

listbox 提供大量的选项框

滚动条组件

滑块组件设置

text 组件（IDLE就是一个text组件）

text组件中的索引（Mark, Tags）：

实战演练 tag

如何检查内容是否有过改动

文本搜索

text组件还有恢复，撤销的操作undo：

用Tkinter创建一个画布：

画椭圆

画五角星（多边形）

利用canvas设置画板

画下拉菜单

右键弹出菜单

在下拉菜单中使用checkboxbutton（多选）和radiobutton（单选）

设置下拉按钮

选择菜单（涉及到*的用法） 形参+*表示打包，实参+*表示解包

事件绑定

通过事件绑定捕捉移动

Message

Spinbox 从一些固定的值中选一个

展现窗格系统()

Toplevel

Tkinter 布局管理器

pack是顺序填充的

grid管理器首选（特别灵活）

place通常不建议使用

Tkinter 模块-messagebox

filedialog（文件对话框）

colorchooser（颜色选择对话框）

pygame 篇章

pygame 安装

作用：

具体资料：

pygame答疑

py里面事件的本质：

如何做一个刷新的事件界面

如何提高pygame游戏的颜值：

如何设置全屏模式：

如何设置可调节的窗口：

如何设置（surface）的放大和缩小

通过rotate实现贴边飞行

实现裁剪功能的实现

如何把图片转化为透明

pygame里面的绘制内容

绘制矩形

绘制多边形

绘制圆形

绘制椭圆形

绘制弧形

线段的绘制

动画精灵

碰撞检测

如何给游戏插入声音

关于自定义鼠标的光标：

设置小球响应光标的移动频率

Python 学习心得

1 Python的基本介绍

1.1 Python的优势特点

- 1) 跨平台
- 2) 代码量少，相同功能的条件下
- 3) 维护方便
- 4) 应用范围广（maya都能嵌入Python，open ERP，云计算）。

1.2 本系列学习要点

- 0) Python3的所有常用语法
- 1) 面向对象的编程思维
- 2) 运用模块进行编程
- 3) 游戏编程
- 4) 计算机仿真

1.3 Python的安装

- Python版本

视频：3.3.2

我下载了：3.7.3

Version	Operating System	Description	M
Gzipped source tarball	Source release		2
XZ compressed source tarball	Source release		9
macOS 64-bit/32-bit installer	Mac OS X	for Mac OS X 10.6 and later	5
macOS 64-bit installer	Mac OS X	for OS X 10.9 and later	4
Windows help file	Windows		7
Windows x86-64 embeddable zip file	Windows	for AMD64/EM64T/x64	8
Windows x86-64 executable installer	Windows	for AMD64/EM64T/x64	a
Windows x86-64 web-based installer	Windows	for AMD64/EM64T/x64	0
Windows x86 embeddable zip file	Windows		7
Windows x86 executable installer	Windows		e
Windows x86 web-based installer	Windows		c

1.4 从IDLE启动Python

IDLE是一个Python的shell，就是一个通过键入文本与程序交互的一个途径（类似Linux的Xshell）。

>>>这个提示符告诉你Python已经准备好了，等待你追加的指令呢！

- Python基本操作控制

Alt+n 前进一条成功的语句, Alt+p 后退一条成功的语句

如果想选择最近的语句修改, 按Alt+p

1.5 课堂总结

1.5.1 Python 不兼容其他C语言等语法格式

```
Type "help", "copyright", "credits" or "license()" for more information.
>>> print("I love fishC.com")
I love fishC.com
>>> print "I love you!"
SyntaxError: Missing parentheses in call to 'print'. Did you mean print("I love you!")?
>>> print("I love fishC.com")
I love fishC.com
>>> printf(" I love you")
Traceback (most recent call last):
  File "<pyshell#3>", line 1, in <module>
    printf(" I love you")
NameError: name 'printf' is not defined
```

Python不兼容其他语法格式

1.5.2 Python的几个好处点

```
NameError: name print is not defined
>>> print(5+3)
8
>>> 5+3
8
>>> 123456789*987654321
121932631112635269
>>> print("well water"+"river")
well waterriver
>>> print("well water"+" river")
well water river
>>> print("I love fishC.com"*8)
I love fishC.comI love fishC.comI love fishC.comI love fishC.comI love fishC.com
I love fishC.comI love fishC.comI love fishC.comI love fishC.com
>>> print("I love fishC.com\n"*8)
I love fishC.com
I love fishC.com
I love fishC.com
I love fishC.com
I love fishC.com
I love fishC.com
I love fishC.com
I love fishC.com
>>> print("I love fishC.com\n"*8)
```

直接两个数相加

不用担心溢出限制

直接两个字符串相加

通过数字相乘直接复制字符串

1.6 第一讲课后练习精选

4. 为什么 >>>print('I love fishc.com ' * 5) 可以正常执行, 但 >>>print('I love fishc.com ' + 5) 却报错?

在 Python 中不能把两个完全不同的东西加在一起, 比如说数字和文本, 正是这个原因, >>>print('I love fishc.com ' + 5) 才会报错。这就像是在说“五只小甲鱼加上苍井空会是多少?”一样没有多大意义, 结果可能是五, 可能是六, 也可能是八! 不过乘以一个整数来翻倍就具有一定的意义了, 前边的例子就是将“I love fishc.com”这个字符串打印五次。


```
>>> help(input)
Help on built-in function input in module builtins:

input(prompt=None, /)
    Read a string from standard input.  The trailing newline is
    stripped.  The prompt string, if given, is printed to standard output w
    trailing newline before reading input.
```

2.3 第2讲课后练习精选

3. 在小甲鱼看来，Python 中什么是最重要的？你赞同吗？

在小甲鱼看来，缩进是 Python 的灵魂，缩进的严格要求使得 Python 的代码显得非常精简并且有层次（小甲鱼阅读过很多大牛的代码，那个乱.....C语言不是有国际乱码大赛嘛.....）。

所以在 Python 里对待缩进代码要十分小心，如果没有正确地缩进，代码所做的事情可能和你的期望相去甚远（就像C语言里边括号打错了位置）。

如果在正确的位置输入冒号“:”，IDLE 会自动将下一行缩进！

3 插曲之变量/字符串/改进我们的游戏

3.1 Python与其他语言关于变量的不同点

- 其他语言中，都是把“值存储在变量中”，即给变量赋值。
- Python并不是把值存储在变量中，**将名字贴在值上面**。

3.2 变量名使用规范

- 1) 必须先赋值
- 2) 变量名可以包括字母，数字，下划线，但**不能以数字开头**。
- 3) 大小写不同
- 4) =两边不能写反，左边名字，右边值。
- 5) 请对规范变量命名

3.3 字符串

- 1) 字符串其实就是文本。
- 2) 字符串在Python中可以用单引号（'）和双引号（"）表示，但必须配对。
- 3) 用“\”可以对字符串中的（'，"）表示转义。

3.4 多\转义，跨越多行字符串

- 如果一个字符串有很多个反斜杠：

```
Type "help", "copyright", "credits" or "license()" for more information.
>>> str=r'C:\now\fichC\la'
>>> str
'C:\now\fichC\la'
>>> str=r'C:\now\fichC\la'
SyntaxError: EOL while scanning string literal
>>> print(str)
C:\now\fichC\la
>>>
```

通过r+字符串，自动将\转义

字符串的最后不能加\，不然报错

- 如何得到一个跨越多行的字符串：
 - 三重引号字符串（print打印）

```
C:\now\fichC\la
>>> str="""6月4日，国家发改委相关司局召开稀土行业专家座谈会，分析行业发展现状，就推动中国稀土产业高质量发展听取意见建议。与会专家建议，要加强稀土出口管控，建立稀土出口全流程追溯和审查机制；加快实现由依靠资源消耗为主的粗放型规模化扩张，转向依靠基础研发能力提升和技术进步的创新驱动发展。"""
>>> str
'6月4日，国家发改委相关司局召开稀土行业专家座谈会，分析行业发展现状，就推动中国稀土产业高质量发展听取意见建议。\\n与会专家建议，要加强稀土出口管控，建立稀土出口全流程追溯和审查机制。\\n加快实现由依靠资源消耗为主的粗放型规模化扩张，转向依靠基础研发能力提升和技术进步的创新驱动发展。\\n'
>>>
```

3.5 游戏改进

3.5.1 条件分支

if 条件：

条件为真执行的操作

else：

条件为假执行的操作

3.5.2 while 循环

while 条件：

条件为真时，执行操作

3.5.3 and 逻辑操作符

and 将任意表达式连接在一起，返回一个bool量。

3.5.4 可以通过randint，将8变成一个随机的数：

```
import random
times = 3
secret = random.randint(1,10)
```

3.5.5 改进的结果

```
import random

times = 3

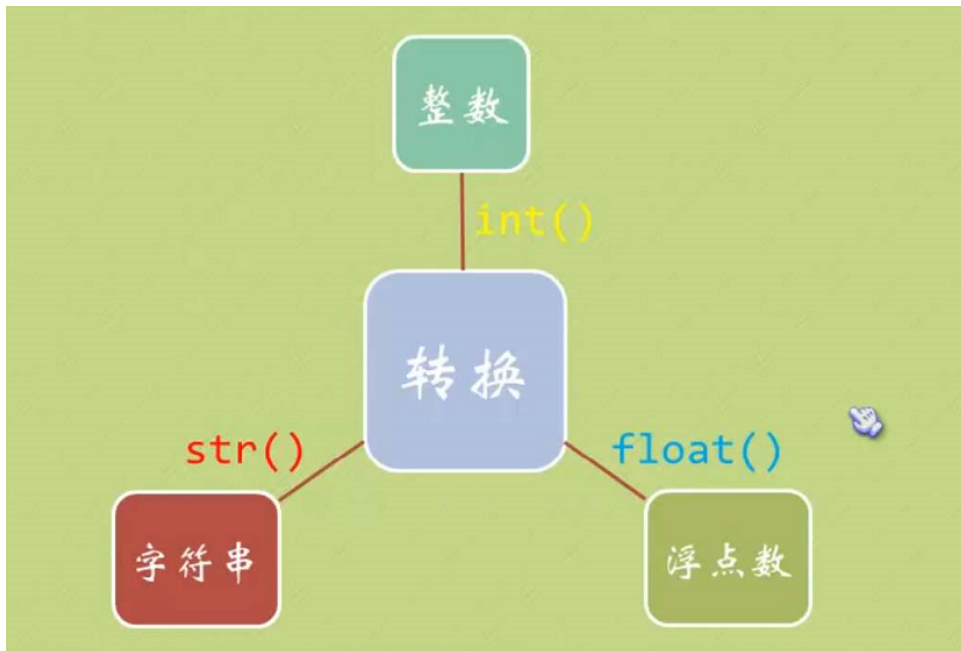
secret = random.randint(1,10)
```

```

print('-----我爱鱼C工作室-----')
# 这里先给guess赋值（赋一个绝对不等于secret的值）
guess = 0
# print()默认是打印完字符串会自动添加一个换行符，end="" 参数告诉
print()用空格代替换行
# 嗯，小甲鱼觉得富有创意的你应该会尝试用 end="JJ"?
print("不妨猜一下小甲鱼现在心里想的是哪个数字：", end=" ")
while (guess != secret) and (times > 0):
    temp = input()
    guess = int(temp)
    times = times - 1 # 用户每输入一次，可用机会就-1
    if guess == secret:
        print("我草，你是小甲鱼心里的蛔虫吗?! ")
        print("哼，猜中了也没有奖励!")
    else:
        if guess > secret:
            print("哥，大了大了~~~")
        else:
            print("嘿，小了，小了~~~")
            if times > 0:
                print("再试一次吧：", end=" ")
            else:
                print("机会用光咯T_T")
print("游戏结束，不玩啦^_^")

```

4 Python的数据类型/操作符



4.1 e记法

e记法：e代表10，会自动将该数转化为浮点数。

```
type help, copy
>>> 15e11
1500000000000.0
...
```

4.2 如何确定类型一致

4.2.1 type用法

```
type help, copy
>>> a='530'
>>> type(a)
<class 'str'>
>>> type(5.2)
<class 'float'>
...
```

4.2.2 isinstance 用法

判断前后类型是否一致，并返回一个布尔变量

```
type help, copyright, credits
>>> a='530'
>>> type(a)
<class 'str'>
>>> type(5.2)
<class 'float'>
>>> isinstance(a, str)
True
>>> isinstance(a, int)
False
>>>
```

4.3 数据类型课后习题精选

1. 你知道为什么布尔类型(bool)的 True 和 False 分别用 1 和 0 来代替吗？

你可能听说过计算机是很“笨”的，究其根本是因为它只认识二进制数，所以所有的编程语言最终都会转换成简单的二进制序列给 CPU 按照一定的规则解析。

由于二进制只有两个数：0 和 1，因此用 0 和 1 来表示 False 和 True 再适合不过了，因为不用浪费资源在转换的过程上！

5. Python3 可以给变量命名中文名，知道为什么吗？

Python3 源码文件默认使用utf-8编码（支持中文），这就使得以下代码是合法的：

```
5
>>> 汪宇='你好'
>>> print(汪宇)

SyntaxError: invalid character
>>> print(汪宇)
你好
>>>
```

4.4 操作符

4.4.1 Python 地板除法 //

```
10
>>> 10 // 8
1
>>> 3.0 // 2
1.0
>>> 3/2
1.5
>>>
```

4.4.2 Python 幂运算 **

```
11
>>> 3**2
9
>>> 3 ** 5
243
>>>
```

4.4.3 逻辑操作符 and 、or、 not

```
0.
>>> 3<4<5
True
>>> 3<4 and 4 < 5
True
>>> not 5
False
>>> not 5 or not 0
True
>>>
```

4.5 操作符的优先级问题

```
243
>>> -3*2+5/-2-4
-12.5
>>> (-3)*2+(5/(-2))-4
-12.5
>>>
```

幂操作符，比左侧的优先级要高，先计算，比右侧的优先级低，后计算。

```
11
>>> -3 ** 2
-9
>>> -(3 ** 2)
-9
>>> 3 ** -2
0.1111111111111111
>>>
```

优先级图：



6 打了激素的数组

6.1 列表的创建

```
Type "help", "copyright", "credits" or "license()" f
>>> member=['小甲鱼','apple','junire','switch']
>>> member
['小甲鱼', 'apple', 'junire', 'switch']
>>>
```

- Python中列表的优势

可以多种元素混合存在一个列表中，可以创建一个空列表

```
>>> mix = [1,2,'nihao',3.14,[1,2,3]]
>>> mix
[1, 2, 'nihao', 3.14, [1, 2, 3]]
>>> empty=[]
>>> empty
[]
>>>
```

6.2 向列表添加元素

6.2.1 append添加方法

```
>>> member.append('植树poi')
>>> member
['小甲鱼', 'apple', 'junire', 'switch', '植树poi']
>>> len(member)
5
```

- append 方法在列表元素最后添加。
- append 方法不能添加2个及以上元素。

6.2.2 extend 添加列表


```

>>> member.append('hello','xuejunru')
Traceback (most recent call last):
  File "<pyshell#9>", line 1, in <module>
    member.append('hello','xuejunru')
TypeError: append() takes exactly one argument (2 given)
>>> member.extend('hello','xuejunru')
Traceback (most recent call last):
  File "<pyshell#10>", line 1, in <module>
    member.extend('hello','xuejunru')
TypeError: extend() takes exactly one argument (2 given)
>>> member.extend(['hello','xuejunru'])
>>> member
['小甲鱼', 'apple', 'junire', 'switch', '植树poi', 'hello', 'xuejunru']
>>>

```

extend 不能直接添加2个元素

必须变成列表添加才行

6.2.3 insert 插入一个元素

```

>>> member.insert(0,'wwangjianjun')
>>> member
['wwangjianjun', '小甲鱼', 'apple', 'junire', 'switch', '植树poi', 'hello', 'xuejunru']
>>>

```

- insert 从0开始。

6.2 列表中的元素操作

6.2.1 元素索引

```

>>> member[0]
'wwangjianjun'
>>> member[6]
'hello'
>>> member[8]
Traceback (most recent call last):
  File "<pyshell#17>", line 1, in <module>
    member[8]
IndexError: list index out of range
>>> temp=member[0]
>>> member[0]=member[1]
>>> member[1]=temp
>>> member
['小甲鱼', 'wwangjianjun', 'apple', 'junire', 'switch', '植树poi', 'hello', 'xuejunru']
>>>

```

6.2.2 列表中元素删除 remove ()

```

>>> member
['小甲鱼', 'wwangjianjun', 'apple', 'junire', 'switch', '植树poi', 'hello', 'xuejunru']
>>> member.remove('apple')
>>> member
['小甲鱼', 'wwangjianjun', 'junire', 'switch', '植树poi', 'hello', 'xuejunru']
>>>

```

6.2.3 del() 删除

```

>>> del member[1]
NameError: name 'de' is not def:
>>> del member[1]
>>> member
['小甲鱼', 'junire', 'switch', '植树poi', 'hello', 'xuejunru']
>>> del member
>>>

```

- del 还可以删除整个列表

6.2.4 pop 删除

```

Type "help", "copyright", "credits" or "license()" for more information
>>> member=['xiaojiaayu','junire','zhishupoi','switch','wangjianjun']
>>> member
['xiaojiaayu', 'junire', 'zhishupoi', 'switch', 'wangjianjun']
>>> name=member.pop(1)
>>> name
'junire'
>>>

```

pop可以带参数, 指定pop的元素

6.2.5 列表的分片 (强大)

```

jupyter
>>> member[1:3]
['zhishupoi', 'switch']
>>> member
['xiaojiayu', 'zhishupoi', 'switch', 'wangjianjun']
>>> member[:3]
['xiaojiayu', 'zhishupoi', 'switch']
>>> member[1:]
['zhishupoi', 'switch', 'wangjianjun']
>>> member[:]
['xiaojiayu', 'zhishupoi', 'switch', 'wangjianjun']
>>> member2=member[1:3]
>>> member2
['zhishupoi', 'switch']

```

→ 常用使用方法，得到拷贝

6.3 列表的关系操作符

6.3.1 列表比大小

```

>>> list1=[123,456]
>>> list2=[234,123]
>>> list1>list2
False
>>>

```

- list 比大小时，只比较第一个数组。

6.3.2 列表的拼接

```

>>> (list1<list2) and (list1==list3)
True
>>> list4=list1+list2
>>> list4
[123, 456, 234, 123]
>>> list1+'小甲鱼'
Traceback (most recent call last):
  File "<pyshell#7>", line 1, in <module>
    list1+'小甲鱼'
TypeError: can only concatenate list (not "str")
>>> list3*3
[123, 456, 123, 456, 123, 456]
>>> list3*=3
>>> list3
[123, 456, 123, 456, 123, 456]
>>>

```

- list在添加元素时，最好使用append，和extend。

6.3.3 列表的成员关系操作符

```

123, 456, 123, 456, 123, 456
>>> 123 in list3
True
>>> 123 not in list3
False
>>>

```

6.3.4 列表中的列表

- 列表只影响其所属的一层的成员关系，见图

```

>>> list5=[123,['xiaojiayu','mudan'],456]
>>> 'xiaojiayu' in list5
False
>>> 'xiaojiayu' in list5[1]
True
>>> list5[1][1]
'mudan'

```

→ 直接访问

6.3.5 计算列表内元素的数目

- list3.count(123)

```

>>> list3.count(123)
3
>>>

```

6.3.6 查找某元素出现的范围

```

>>> list3.index(123)
0
>>> list3.index(123, 2, 7)
2
>>>

```

6.3.7 列表的翻转

```

>>> list3.reverse()
>>> list3
[456, 123, 456, 123, 456, 123, 456, 123, 456]
>>>

```

6.3.8 列表的整排（默认从小到大）

```

>>> list6=[1,8,6,4,23,9,46]
>>> list6.sort()
>>> list6
[1, 4, 6, 8, 9, 23, 46]

```

直接从大到小整理：

```

>>> list6.reverse()
>>> list6.sort(reverse=True)
>>> list6
[46, 23, 9, 8, 6, 4, 1]

```

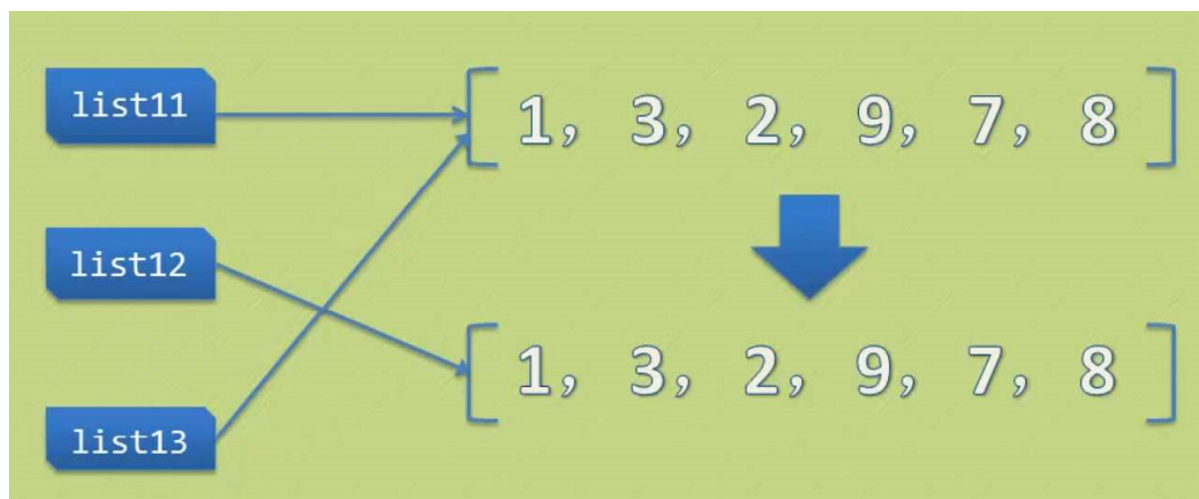
6.3.9 列表拷贝的补充

```

>>> list7=list6[:]
>>> list8=list6
>>> list6.sort()
>>> list7
[46, 23, 9, 8, 6, 4, 1]
>>> list8
[1, 4, 6, 8, 9, 23, 46]
>>> list6
[1, 4, 6, 8, 9, 23, 46]
>>>

```

- Python中变量就像个标签，这里就是list8的标签贴到了list6上，所以list6变化，list8就会跟着变化；然而分片复制的list7则不然。



6.4 关进笼子的列表——元组

6.4.1 元组的创建和一些基本用法

```

type temp, copyright, credits or license
>>> tuple1=(1,2,3,4,5,6,7,8)
>>> tuple1
(1, 2, 3, 4, 5, 6, 7, 8)
>>> tuple[1]
Traceback (most recent call last):
  File "<pyshe11#2>", line 1, in <module>
    tuple[1]
TypeError: 'type' object is not subscriptable
>>> tuple1[1]
2

```

6.4.2 元组的实质

- 非空元组的实质其实是逗号。

```

>>> temp=(1)
>>> type(temp)
<class 'int'>
>>> temp2=2,3,4
>>> type(temp2)
<class 'tuple'>
>>> temp=()
>>> type(temp)
<class 'tuple'>
>>> temp=(1,)
>>> type(temp)
<class 'tuple'>
>>> temp=1,
>>> type(temp)
<class 'tuple'>

```

6.4.3 元组和列表的不同之处

- 元组没有增加删除的操作，如果想在元组内部进行增加修改，只能通过重构的操作。

```

TypeError: can only concatenate tuple (not 'str') to tuple
>>> temp2=temp[:2]+('xiaohai',)+temp[2:]
>>> temp2
('zhishupoi', 'xuejunru', 'xiaohai', 'wangjianjun', 'miaomiao')
>>> temp
('zhishupoi', 'xuejunru', 'wangjianjun', 'miaomiao')
>>>

```

如果此处把temp2换成temp 就相当于增加修改了，回头Python会自动将原来的数据（没有标签指向时）给回收释放了。

硬要删除可以使用del +元组进行操作。

6.5 课后习题精选

6.5.1 上一题打印的样式不是很好，能不能修改一下代码打印成下图的样式呢？

```

>>> for each in range(len(member)):
    if each%2 == 0:
        print(member[each],member[each+1])
小甲鱼 88
黑夜 90
迷途 85
怡静 90
秋舞斜阳 88

```

7 字符串

7.1 字符串的一些内置函数用法

- `str2.capitalize()` 首字母大写
- `str3.casefold()` 大写转小写
- `str3.center(40)` 在总长40的字符中，将str3居中
- `str3.count('xi')` 查找str3中'xi'出现了几次
- `str3.endswith('xi')` 判断str3是否以'xi'作为结尾
- `str4.expandtabs()` 将 '\t' 转换成Tab(默认8个字符，是包含前面的非空字符，所以第二个才4个空格)
- `str4.find('com')` 查找str4中的'com'字符在第几位（从0开始），如果有返回位数，如果没有返回-1。

```
>>> str1='I love fishC.com'
>>> str1
'I love fishC.com'
>>> str1[6]
'.'
>>> str1[5]
'e'
>>> str2='xiaoxie'
>>> str2.capitalize()
'Xiaoxie'
>>> str3='DAXIExiaoxie'
>>> str3.casefold()
'daxiexiaoxie'
>>> str3.center(40)
'DAXIExiaoxie'
>>> str3.count('xi')
2
>>> str3.endswith('xi')
False
>>> str3.endswith('xie')
True
>>> str4='I\tlove\tFishC.com!'
>>> str4.expandtabs()
'I      love    FishC.com!'
>>> str4.find('etx')
-1
>>> str4.find(com)
Traceback (most recent call last):
  File "<pyshell#15>", line 1, in <module>
    str4.find(com)
NameError: name 'com' is not defined
>>> str4.find('com')
13
```

- `str5.islower()` 是否都是小写
- `str3.istitle()` 是否首字母大写
- `str7.join('12345')` 将'12345'用str7间隔开来
- `str8.lstrip()` 将左边的空格缩进掉
- `str9.partition('ov')` 将str9以 'ov' 为中心 分割成三元组
- `str9.split('i')` 每隔 'i' 分割成一个列表，'i' 不包含于其中
- `str10.strip('s')` 将前后的 's' 删除掉
- `str9.swapcase()` 交换大小写

```

>>> str5.islower()
False
>>> str2.islower()
True
>>> str3.istitle()
False
>>> str6='FISHC'
>>> str6.istitle()
False
>>> str7='Fishc'
>>> str7.istitle()
True
>>> str7.join('12345')
'1Fishc2Fishc3Fishc4Fishc5'
>>> str8='    I love you!'
>>> str8.lstrip()
Traceback (most recent call last):
  File "<pyshe11#27>", line 1, in <module>
    str8.lstrip()
AttributeError: 'str' object has no attribute 'lstrip'
>>> str8.lstrip()
'I love you!'
>>> str9='I love fishC.com!'
>>> str9.partition('ov')
('I l', 'ov', 'e fishC.com!')
>>> str9.replace('fishC', 'FishC')
'I love FishC.com!'
>>> str9.split()
['I', 'love', 'fishC.com!']
>>> str9.split('i')
['I love f', 'shC.com!'],
>>> str10='aaasss'
>>> str10.strip()
'aaasss'
>>> str10='ssssaaasss'
>>> str10.strip('s')
'aaa'
>>> str9.swapcase()
'i LOVE FISHc.COM!'

```

- `str10.translate(str.maketrans('s','b'))` 将's' 替换成'b'
- `str10.zfill(20)` 输出20个字符串, 不足在前面补0

```

>>> str10
'sssaaasss'
>>> str10.translate(str.maketrans('s','b'))
'bbbbaaabb'
>>> str.maketrans('s','b')
{115: 98}
>>> str10.zfill(5)
'sssaaasss'
>>> str10.zfill(20)
'0000000000ssssaaasss'

```

7.2 字符串的格式化方法

7.2.1 位置参数和关键字参数 (format)

位置参数{0}, {1}, {2}

关键字参数{a}, {b}, {c}, 关键字参数必须指定。

两者可以混合使用。

在混合使用时要注意, 位置参数要在关键字之前。

```

type help, copyright, credits or license() for more information
>>> '{0} love {1}. {2}'.format('I', 'FishC', 'com')
'I love FishC.com'
>>> '{a} love {b}. {c}'.format(a='I', b='FishC', c='com')
'I love FishC.com'
>>> '{a} love {b}. {0}'.format(a='I', b='FishC', c='com')
SyntaxError: positional argument follows keyword argument

```

位置参数不能在关键字参数的后面

如果'{0}'像这样使用了两个'{'则原本的'{'参数表示将被转义了。例如：

```

>>> '{{0}}'.format('不打印')
'{0}'

```

7.2.2 参数保留一位小数的方法

'{0:.1f}{1}'.format(27.658, 'GB') 四舍五入 最后出现27.7

```

type help, copyright, credits or license()
>>> '{0:.1f}{1}'.format(27.658, 'GB')
'27.7GB'

```

7.2.2 格式化参数的介绍

- '%c %c %c' % (97,98,99) 将97, 98, 99 转化为其对应的ASCII码
- '%s' % 'I love FishC.com!' 格式化字符串
- '%d + %d = %d' % (4,5,3+5) 格式化整数
- '%o' % 10 将10 格式化对应的8进位数
- '%x' % 10 将10 格式化对应的16进位数，并小写表示
- '%X' % 160 将10 格式化对应的16进位数，并大写表示
- '%f' % 27.658 格式化定点数，可以指定小数点后面的精度，默认6个
- '%e' % 27.658 科学计数法，小写e表示
- '%E' % 27.658 科学计数法，大写E表示
- '%g' % 27.658 科学技术和定点数，根据值的大小灵活选用。

```

21.100
>>> '%c' % 97
'a'
>>> '%c %c %c' % (97,98,99)
'a b c'
>>> '%s' % 'I love FishC.com!'
'I love FishC.com!'
>>> '%d' + '%d' = '%d' % (4,5,3+5)
SyntaxError: can't assign to operator
>>> '%d + %d = %d' % (4,5,3+5)
'4 + 5 = 8'
>>> '%o' % 10
'12'
>>> '%x' % 10
'a'
>>> '%X' % 10
'A'
>>> '%X' % 160
'A0'
>>> '%f' % 27.658
'27.658000'
>>> '%e' % 27.658
'2.765800e+01'
>>> '%E' % 27.658
'2.765800E+01'
>>> '%g' % 27.658
'27.658'

```

7.2.3 格式化操作符辅助指令

- '%10.5f' % 23.567 总字符数10，小数点后面字符数5
- '%-10d' % 5 左对齐

- '%+10d' % -5 增加+表示正数，如果是负数，则显示-5。
- '%#o' % 10 八进制表示10，并在前面显示零（'0'）
- '%#X' % 108 十六进制表示108，并在前面显示'0X'（此处为大写）
- '%010d' % 5 在显示5的数字前填充0，取代空格

```
type help, copyright, credits
>>> '%10.5f' % 23.567
'23.56700'
>>> '%10d' % 5
'5'
>>> '%-10d' % 5
'-5'
>>> '%-10d' % 5
'-5'
>>> '%+10d' % 5
'+5'
>>> '%+10d' % -5
'-5'
>>> '%#o' % 10
'0o12'
>>> '%#X' % 108
'0X6C'
>>> '%010d' % 5
'0000000005'
>>> '%-010d' % 5
'-5'
>>>
```

- 格式化的转义字符就不详细介绍了。

7.3 序列

7.3.1 list 基本使用

list方法的过程大概是：

- 1) 新建一个列表。
- 2) 将需要循环的对象，循环迭代入列表中。
- 3) 循环完成，生成一个列表。

```
>>> a=list()
>>> a
[]
>>> b='I love fishC.com!'
>>> b
'I love fishC.com!'
>>> list(b)
['I', ' ', 'l', 'o', 'v', 'e', ' ', 'f', 'i', 's', 'h', 'C', '.', 'c', 'o', 'm', '']
>>> c=(1, 1, 2, 3, 5, 8, 13, 21, 34)
>>> c
(1, 1, 2, 3, 5, 8, 13, 21, 34)
>>> list(c)
[1, 1, 2, 3, 5, 8, 13, 21, 34]
>>>
```

7.3.2 与list相关的一些内置函数

- tuple 将一个可迭代对象转化为元组，方法和list一样
- str(obj) 将一个参数转化为字符串
- len()返回有多少个字符

```
11, 1, 2, 3, 5, 8, 13, 21, 34]
>>> len(list(b))
17
>>> max(1, 2, 3, 4, 5)
5
>>> max(list(b))
'v'
>>> numbers=[1, 18, 13, 0, -79, 32]
```


- max() 返回序列或者参数集中的最大值 需要保持集合内部数据类型一致
- min() 返回序列或者参数集中的最小值 需要保持集合内部数据类型一致
- sum() 返回求和值 需要保持集合内部数据类型一致
- sorted() 从小到大排列 需要保持集合内部数据类型一致
- reversed 倒置 需要配合list使用
- enumerate 枚举 将对象变成带有索引值的元组。需要配合list使用。

```
>>> max(numbers)
32
>>> chars='1234567890'
>>> min(chars)
'0'
>>> tuple2=(3.1,3.2,3.4)
>>> sum(tuple2)
9.7000000000000001
>>> sum(tuple2)
9.7000000000000001
>>> tuple2=(3.1,2.3,3.4)
>>> sum(tuple2)
8.8
>>> tuple2=(3.1,3.2,3.3)
>>> sum(tuple2)
9.6000000000000001
>>> sum(numbers)
-15
>>> sorted(numbers)
[-79, 0, 1, 13, 18, 32]
>>> reversed(numbers)
<list_reverseiterator object at 0x037E87D0>
>>> list(reversed(numbers))
[32, -79, 0, 13, 18, 1]
>>> list(enumerate(numbers))
[(0, 1), (1, 18), (2, 13), (3, 0), (4, -79), (5, 32)]
```

8 函数

8.1 Python 函数的基本介绍

8.1.1 函数的创建和调用

```
SyntaxError: invalid character in identifier
>>> def MyFirstFunction():
    print('这是我创建的第一个函数!')
    print('我表示很激动')

>>>
>>> MyFirstFunction()
这是我创建的第一个函数!
我表示很激动
>>>
```

8.1.2 函数的参数

- 函数的参数让函数更加具有泛用性

```
>>> def MySecondFunction(name):
    print(name+'我爱你!')

>>> MySecondFunction('植树')
植树我爱你!
>>> MySecondFunction('小甲鱼')
小甲鱼我爱你!
```

- 调用2个以上的函数参数

```
>>> def add1(num1,num2):
    result=num1+num2
    print(result)

>>> add1(1,2)
3
```

- 具有函数返回值函数

```
3
>>> def add1(num1, num2):
    return (num1+num2)

>>> add1(5, 6)
11
...

```

8.2 函数的参数类型

8.2.1 形参和实参

```
>>> MyFisrtFunction('植树')
传递进来的植树叫做实参，因为Ta是具体的数值！
>>> def MyFisrtFunction(name):
    '函数定义过程中的name是叫形参'
    #因为Ta只是一个形式，表示占据一个参数位置
    print('传递进来的'+ name + '叫做实参，因为Ta是具体的数值！')

>>> MyFisrtFunction('植树')
传递进来的 植树 叫做实参，因为Ta是具体的数值！
>>> MyFisrtFunction.__doc__
'函数定义过程中的name是叫形参'
>>>
```

8.2.2 查阅函数文档

可以通过查阅函数文档，更方便快捷的了解该函数的作用

两种方法：1) help 2) .__doc__

```
>>> help(MyFisrtFunction)
Help on function MyFisrtFunction in module __main__:

MyFisrtFunction(name)
    函数定义过程中的name是叫形参

>>> print.__doc__
"print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)\n\nPrints t
he values to a stream, or to sys.stdout by default.\n\nOptional keyword arguments:
\nfile: a file-like object (stream); defaults to the current sys.stdout.\nsep:
  string inserted between values, default a space.\nend:   string appended after
the last value, default a newline.\nflush: whether to forcibly flush the stream
"
```

8.2.3 关键字参数

作用：可以防止因为函数参数太多而产生调用顺序的Bug。

```
>>> def SaySome(name, words):
    print(name+ ' -> ' + words)

>>> SaySome('植树', '让编程改变世界！')
植树->让编程改变世界！
>>> SaySome('让编程改变世界！', '植树')
让编程改变世界！->植树
>>> SaySome(words='让编程改变世界！', name='植树')
植树->让编程改变世界！
>>>
```

8.2.4 默认参数

给参数设置一个初始值。

```
>>> def SaySome(name='植树', words='让编程改变世界！'):
    print(name+ ' -> ' + words)

>>> SaySome()
植树->让编程改变世界！
>>> SaySome('王建军', '12345')
王建军->12345
>>>
```

8.2.5 收集参数 (牛逼)

可变参数，实际作者也不清楚到底需要几个参数，就设计了一个收集参数。

相当于把输入的参数用元组打包起来，放进去。

```
>>> def test1(*params):
    print('参数的长度是:', len(params))
    print('第3个参数是:', params[2])

>>> test1(1, '植树', 3.14, '王建军')
参数的长度是: 4
第3个参数是: 3.14
>>> test1(1, '植树')
参数的长度是: 2
Traceback (most recent call last):
  File "<pyshell#29>", line 1, in <module>
    test1(1, '植树')
  File "<pyshell#27>", line 3, in test1
    print('第3个参数是:', params[2])
IndexError: tuple index out of range
>>>
```

- 收集函数在使用时，如果后面还有参数，则后面的参数必须指定。最好也还是设置个默认参数

```
Traceback (most recent call last):
  File "<pyshell#32>", line 1, in <module>
    test1(1, '植树', 3.14, '王建军')
TypeError: test1() missing 1 required keyword-only argument: 'exp'
>>> test1(1, '植树', 3.14, '王建军', exp=8)
参数的长度是: 4 exp= 8
第3个参数是: 3.14
>>> test1(1, '植树', 3.14, '王建军', 8)
Traceback (most recent call last):
  File "<pyshell#34>", line 1, in <module>
    test1(1, '植树', 3.14, '王建军', 8)
TypeError: test1() missing 1 required keyword-only argument: 'exp'
>>>
```

必须指定出exp, 否则报错

8.3 函数变量的作用域/函数的返回值

8.3.1 函数和过程

- Python 的所有函数都会返回某个东西的，Python没有严格意义的过程。

Type "help", "copyright", "credits" or "license()" for more inform

```
>>> def hello():
    print('Hello, 植树poi!')
hello 没有返回值

>>> temp=hello()
Hello, 植树poi!
>>> temp
>>> print(temp)
None
>>> type(temp)
<class 'NoneType'>
此处的temp是一个NoneType 的类型
>>>
```

8.3.2 Python是没有变量的，只有名字

- 在函数内部声明时，不需要对变量加以限定，Python会自动地根据你输入的数据，判断其属于什么类型的变量。
- Python也可以返回一个列表，或者元组

```

Python 3.10.11: Interactive Shell
>>> def back1():
    return [1, '小甲鱼', 3.14]

>>> back1()
[1, '小甲鱼', 3.14]
>>>

```

8.3.3 Python 函数变量的作用域问题

局部变量和全局变量

当你想要去修改全局变量时，Python会自动创建一个跟全局变量重名的局部变量。

```

def discounts(price, rate):
    final_price = price * rate
    # print('这是试图打印全局变量old_price的值:', old_price)
    old_price = 50
    print('修改后全局变量old_price的值是1:', old_price)
    return final_price

old_price = float(input('请输入原价: '))
rate = float(input('请输入折扣率: '))
new_price = discounts(old_price, rate)
print('打折后的价格是:', new_price)
print('修改后全局变量old_price的值2:', old_price)
# print('这是试图打印局部变量final_price的值:', final_price)

```

此处的old_price是一个跟全局变量重名的局部变量。

final price是局部变量

8.4 全局和闭包的概念

8.4.1 global 关键字

- 用法：当你想要修改全局变量时，可在局部函数内部，**首先进行声明**，你要修改的这个变量时全局变量。

```

type, help, copyright, credit
>>> count=5
>>> def Myfunc():
    global count
    count=10
    print(count)

>>> count
5
>>> Myfunc()
10
>>> count
10
>>>

```

8.4.2 内嵌函数

- Python支持内嵌函数，但注意，如果嵌套的函数2完全定义于某函数1内部，则出了这个函数1，则无法调用。

```
>>> def fun1():
    print('fun1正在被调用...')
    def fun2():
        print('fun2正在被调用...')
    fun2()
```

8.4.3 闭包

如果在一个内部函数里，对外部作用域（但不是全局作用域）的变量进行引用，那内部函数就被认为是闭包。

ALL INFORMATION. Nonetype object has no attribute to

8.4.4 nonlocal 声明

```
>>> Fun1()
25
>>>
```

8.5.1 lambda 函数单参数使用

可以使代码更加精简，对于一些不需要多次调用的函数，使用lambda 也不用考虑函数的命名问题了。

lambda 函数在使用结束后，会自动释放掉。

```
>>> def ds(x):
    return 2*x+1

>>> ds(5)
11
>>> lambda x: 2*x+1
<function <lambda> at 0x00C09780>
>>> g=lambda x: 2*x+1
>>> g(5)
11
```

8.5.2 双参数的lambda函数的使用

```
>>> def add1(x,y):
    return x+y

>>> add1(3,4)
7
>>> lambda x,y:x+y
<function <lambda> at 0x03824E88>
>>> g=lambda x,y:x+y
>>> g(3,4)
7
>>>
```

8.5.3 filter 过滤器

filter (函数 (判断条件) , iteration (迭代器))

filter 会将iteration的值逐步带入条件中，并过滤掉值为0的值。

```
>>> filter(None,[1,0,False,True])
<filter object at 0x038178D0>
>>> list(filter(None,[1,0,False,True]))
[1, True]
>>> def odd(x):
    return x%2

>>> temp=range(10)
>>> show=filter(odd,temp)
>>> list(show)
[1, 3, 5, 7, 9]
>>> list(filter(lambda x:x%2,temp))
[1, 3, 5, 7, 9]
>>>
```

8.5.4 map 函数 映射

map (函数, iteration (迭代器))

两个内置参数：一个函数和一个可迭代的序列。

将序列的每一个元素作为函数的参数进行运算加工，直到所有元素加工完毕，返回所有加工后的元素构成的新的序列

```
>>> list(map(lambda x:x%2,temp))
[0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
>>> list(map(lambda x:x*2,temp))
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
>>>
```

9 递归算法/字典

9.1 递归的基本介绍

9.1.1 递归举例

```

SyntaxError: invalid syntax
>>> def recursion():
        return recursion()

>>> recursion()
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    recursion()
  File "<pyshell#3>", line 2, in recursion
    return recursion()
  File "<pyshell#3>", line 2, in recursion
    return recursion()
  File "<pyshell#3>", line 2, in recursion
    return recursion()
[Previous line repeated 991 more times]
RecursionError: maximum recursion depth exceeded
>>>

```

新版本会直接报错

9.1.2 递归求阶乘

- 非递归版本

```

102001020020
>>> def jiecheng(x):
        for i in range(1,x):
            x*=i
        return x

>>> jiecheng(5)
120
>>> jiecheng(10)
3628800
>>>

```

- 递归版本

```

1020000
>>> def factorial(n):
        if n==1:
            return 1
        else:
            return n*factorial(n-1)

>>> factorial(5)
120
>>>

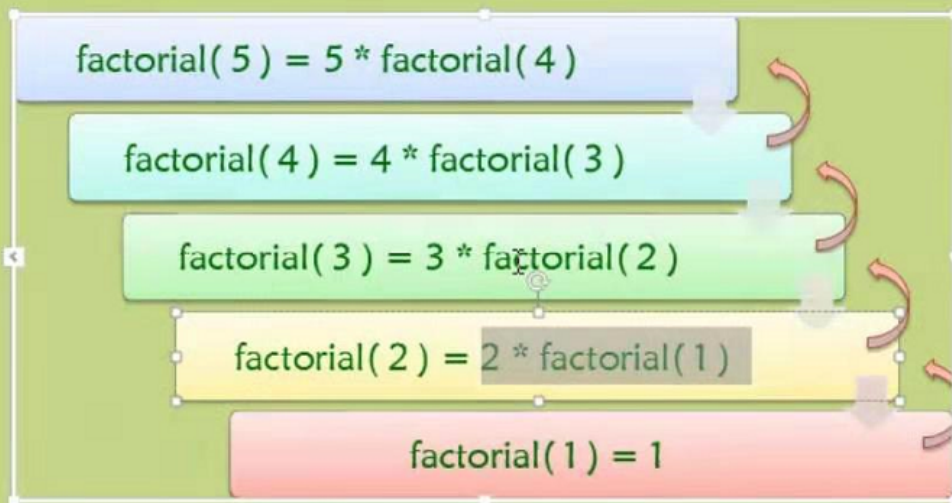
```

- 递归的两个要点:

- 1、设置自身。
- 2、要设置正确的返回值。

递归求阶乘

- 假设我们n的值传入是5，那么：



9.1.3 斐波那契数列 Fibonacci

作用：经常使用斐波那契数列替代黄金分割比例。

斐波那契数列的迭代实现

- 我们都知道兔子繁殖能力是惊人的，如下图：

所经过的月数	1	2	3	4	5	6	7	8	9	10	11	12
兔子的总对数	1	1	2	3	5	8	13	21	34	55	89	144

- 我们可以用数学函数来定义：

$$F(n) = \begin{cases} 1, & \text{当 } n=1 \\ 1, & \text{当 } n=2 \\ F(n-1)+F(n-2), & \text{当 } n>2 \end{cases}$$

- 课间练习：假设我们需要求出经历了20个月后，总共有多少对小兔崽子？（迭代 VS 递归）

迭代解法


```

>>> def fab(n):
    if n<1:
        print('输入有误')
        return -1
    if n==1 or n==2:
        return 1
    else:
        return fab(n-2)+fab(n-1)

>>> fab(0)
输入有误
-1
>>> fab(1)
1
>>> fab(2)
1
>>> fab(10)
55
>>>

```

9.1.4 汉诺塔游戏

- 思路（插入图片）

```

>>> def hanoi(n, x, y, z):
    if n==1:
        print(x, '--->', z)
    else:
        hanoi(n-1, x, z, y) #将n-1个盘子从x移动到y上
        print(x, '--->', z) # 将最底下的盘子移动到z上
        hanoi(n-1, y, x, z) #将n1个盘子从y移动到z上

>>> hanoi(3, 'X', 'Y', 'Z')
X ---> Z
X ---> Y
Z ---> Y
X ---> Z
Y ---> X
Y ---> Z
X ---> Z
\\

```

9.2 字典

9.2.1 创建和访问字典

字典的本质其实就是一种映射类型

```

SyntaxError: invalid syntax
>>> dict1={'李宁':'一切皆有可能','耐克':'Just do it','阿迪达斯':'Nothing is impos-
sible','鱼C工作室':'让编程改变世界'}
\\

```

查询时用dict1[]

```

SyntaxError: EOL while scanning string literal
>>> print('鱼C工作室的口号是:', dict1['鱼C工作室'])
鱼C工作室的口号是: 让编程改变世界
\\

```

dict () 括号里面只能有一个元素，在完成对应关系后，要用括号，把多组对应关系变成一个大元组，才能放入dict()使用。

此处的 ()，也可以变成[]。

```

TypeError: dict expected at most 1 arguments, got 4
>>> dict3=dict((( 'F', 70), ('i', 105), ('s', 115), ('h', 104), ('C', 67)))
>>> dict3
{'F': 70, 'i': 105, 's': 115, 'h': 104, 'C': 67}
\\

```

dict () 也可以通过 '=' 来确定对应关系，用 = 时，要注意关键字不能是字符串，也不能是数字开头。

dict()修改里面值的方法，可以通过dict4['关键字1']= '新的值' 进行修改。

get可以自定义查询不到时，返回什么。

默认返回是none。

```
>>> dict1.get(32)
>>> print(dict1.get(32))
None
>>> dict1.get(32, '木有!')
'木有!'
>>> dict1.get(31, '木有!')
'赞'
>>>
```

9.2.5 通过in 和 not in 检查字典中的成员存在与否

```
>>> 32 in dict1
False
>>> 31 in dict1
True
>>> 32 not in dict1
True
>>> 31 not in dict1
False
>>>
```

9.2.6 通过clear() 清空字典

```
raise
>>> dict1.clear()
>>> dict1
{}
>>>
```

9.2.7 通过copy()浅拷贝

字典里的copy()是浅拷贝。

赋值并不是拷贝，在Python里其只是贴了一个相同的标签。

```
>>> a = {1: 'one', 2: 'two', 3: 'three'}
>>> a
{1: 'one', 2: 'two', 3: 'three'}
>>> b=a.copy()
>>> b
{1: 'one', 2: 'two', 3: 'three'}
>>> c=a
>>> c
{1: 'one', 2: 'two', 3: 'three'}
>>> id(a)
58559104
>>> id(b)
47112336
>>> id(c)
58559104
>>> c[4]='123'
>>> a
{1: 'one', 2: 'two', 3: 'three', 4: '123'}
>>> b
{1: 'one', 2: 'two', 3: 'three'}
>>> c
{1: 'one', 2: 'two', 3: 'three', 4: '123'}
>>>
```

9.2.8 字典里的pop选项，和popitem选项

字典没有先后顺序 如果没有指定值，其会随机弹出一个关键字。

popitem () 没有参数。

```

1: one, 2: two, 3: three, 4: 123 }
>>> a.pop(2)
'two'
>>> a
{1: 'one', 3: 'three', 4: '123'}
>>> a.popitem()
(4, '123')
>>> a
{1: 'one', 3: 'three'}
>>> c.popitem(4)
Traceback (most recent call last):
  File "<pyshell#54>", line 1, in <module>
    c.popitem(4)
TypeError: popitem() takes no arguments (1 given)

```

9.2.9 字典通过setdefault 增加项目

```

SyntaxError: invalid syntax
>>> a.setdefault('小白')
>>> a
{1: 'one', 3: 'three', '小白': None}
>>> a.setdefault(5, 'five')
'five'
>>> a
{1: 'one', 3: 'three', '小白': None, 5: 'five'}
>>>

```

9.2.10 通过另一个字典 更新当前字典里面的选项 (update)

```

>>> b={'小白': '狗'}
>>> a.update(b)
>>> a
{1: 'one', 3: 'three', '小白': '狗', 5: 'five'}
>>>

```

9.3 字典的表亲 集合

9.3.1 集合的特点 (集合里面的元素的唯一性)

- 唯一性 不支持索引

```

>>> num={}
>>> type(num)
<class 'dict'>
>>> num2={1, 2, 3, 4, 5}
>>> type(num2)
<class 'set'>
>>> num2={1, 2, 3, 4, 5, 1, 2, 3, 4, 5}
>>> num2
{1, 2, 3, 4, 5}
>>> num2[2]
Traceback (most recent call last):
  File "<pyshell#6>", line 1, in <module>
    num2[2]
TypeError: 'set' object is not subscriptable

```

9.3.2 如何创建一个集合

- 用花括号直接括一堆元素
- 用set工厂函数

```

>>> num2={1, 2, 3, 4, 5, 1, 2, 3, 4, 5}
>>> num2
{1, 2, 3, 4, 5}
>>> num2[2]
Traceback (most recent call last):
  File "<pyshell#6>", line 1, in <module>
    num2[2]
TypeError: 'set' object is not subscriptable
>>> set1=set([1, 2, 3, 4, 5, 6, 7, 2, 3])
>>> set1
{1, 2, 3, 4, 5, 6, 7}
>>>

```

- set得到的集合是无序的。

9.3.3 可以使用in / not in 判断集合内元素是否存在

```
>>> 7 in set1
True
>>> 8 in set1
False
>>> 7 not in set1
False
>>>
```

9.3.4 用add 和remove 增加删减集合中的元素

```
>>> set1.add(5)
>>> set1
{1, 2, 3, 4, 5, 6, 7}
>>> set1.remove(5)
>>> set1
{1, 2, 3, 4, 6, 7}
>>> set1.add(5)
>>> set1
{1, 2, 3, 4, 5, 6, 7}
>>>
```

9.3.5 frozen 不可变集合

```
>>> set2=frozenset([1,2,3,4,5])
>>> set2
frozenset({1, 2, 3, 4, 5})
>>> set2.add(0)
Traceback (most recent call last):
  File "<pysHELL#20>", line 1, in <module>
    set2.add(0)
AttributeError: 'frozenset' object has no attribute 'add'
>>>
```

10 文件

10.1 文件的基本介绍

10.1.1 文件的打开

Python 里面用open () 函数，打开文件。

打开模式	执行操作
'r'	以只读方式打开文件（默认）
'w'	以写入的方式打开文件，会覆盖已存在的文件
'x'	如果文件已经存在，使用此模式打开将引发异常
'a'	以写入模式打开，如果文件存在，则在末尾追加写入
'b'	以二进制模式打开文件
't'	以文本模式打开（默认）
'+'	可读写模式（可添加到其他模式中使用）
'U'	通用换行符支持

10.1.2文件对象方法

文件对象方法	执行操作
f.close()	关闭文件
f.read(size=-1)	从文件读取size个字符，当未给定size或给定负值的时候，读取剩余的所有字符，然后作为字符串返回
f.readline()	以写入模式打开，如果文件存在，则在末尾追加写入
f.write(str)	将字符串str写入文件
f.writelines(seq)	向文件写入字符串序列seq，seq应该是一个返回字符串的可迭代对象
f.seek(offset, from)	在文件中移动文件指针，从from（0代表文件起始位置，1代表当前位置，2代表文件末尾）偏移offset个字节
f.tell()	返回当前在文件中的位置

f.readline() 从文件中读取一整行字符串。 图片有错误

- f.read() 在读取时，会将指针指到末尾

```
>>> f=open('E:\\record.txt')
>>> f
<_io.TextIOWrapper name='E:\\record.txt' mode='r' encoding='cp936'>
>>> f.read()
'11111111111111111111\n2222222222222222\n3333333333333333'
>>> f.read()
''
>>>
```

```

>>> f=open('E:\\record.txt')
>>> f.read(5)
'11111'
>>> f.tell() → 告诉当前read指针的位置
5
>>> f.seek(18, 0)
18
>>> f.readline()
'2222222222222222\n'
>>> list(f) → 可以直接转化为列表
['3333333333333333']
>>> f.seek(0, 0)
0
>>> list(f)
['1111111111111111\n', '2222222222222222\n', '3333333333333333']
>>> f.readline() → 读取一行，此时read指针已经在最后了
'3333333333333333'
>>> f.seek(0, 0)
0
>>> for each_line in f:
>>>     print(each_line) → 直接打印文本内容
1111111111111111
2222222222222222
3333333333333333
>>> f.write('I love FishC.com')
Traceback (most recent call last):
  File "<pyshell#20>", line 1, in <module>
    f.write('I love FishC.com')
io.UnsupportedOperation: not writable
>>> f=open('E:\\record.txt', 'w')
>>> f.write('I love FishC.com') → write必须和close一起使用
16
>>> f.close()
>>>

```

10.2 模块概念

10.2.1 OS模块

Python 中的OS 模块 可以使得Python 在不同的平台上，使用相同的语言进行操作描述。

有点类似Linux 的一些指令操作符。

- import os 导入os模块
- os.getcwd() 返回当前路径
- os.chdir('E:\\') 改变当前路径到E盘
- os.listdir() 列出当前路径下面的文件，并以列表的形式展现
- os.mkdir('E:\\A') 单层创建文件目录

```
>>> import os
>>> os.getcwd()
'C:\Program Files (x86)\Microsoft Visual Studio\Shared\Python37_64'
>>> os.chdir('E:\\')
>>> os.getcwd()
'E:\\'
>>> os.listdir()
['$RECYCLE.BIN', '2017人工智能全套', '2345Downloads', '360Rec', 'BaiduNetdiskDownload', 'Config.ini', 'Crazy tree.docx', 'DN347', 'EXCEL练习', 'MyDrivers', 'pagefile.sys', 'Program Files', 'python exercise', 'QMDownload', 'QQMusicCache', 'qqcmgr_docpro', 'QQ文件', 'QQ文件分享', 'record.txt', 'resources', 'System Volume Information', 'VGDownloads', 'Visio_V2010_xp85.com', 'zhuangjidashi', '业余爱好.rar', '动感新时代歌曲', '学习笔记', '工作文件', '幻日.docx', '微信', '文学收录', '日语学习', '星火六级晨读英语美文100篇', '机器语言学习', '研究生学习资料', '老妈文件', '计算机学习资料', '迅雷下载', '黑袜和白袜.docx']
```

```
>>> os.mkdir('E:\\A')
>>> os.mkdir('E:\\A\\B')
>>> os.mkdir('E:\\C\\B')
```

Traceback (most recent call last):

File "<pyshell#7>", line 1, in <module>

```
os.mkdir('E:\\C\\B')
```

FileNotFoundError: [WinError 3] 系统找不到指定的路径。: 'E:\\C\\B'

- os.makedirs('E:\\C\\B') 多层创建文件目录
- os.remove('E:\\A\\B\\test.txt') 删除文本
- os.rmdir('E:\\A\\B') 单层删除文件目录

AttributeError: module 'os' has no attribute 'makedirs'

```
>>> os.makedirs('E:\\C\\B')
```

```
>>> os.rmdir('E:\\A\\B')
```

Traceback (most recent call last):

File "<pyshell#10>", line 1, in <module>

```
os.rmdir('E:\\A\\B')
```

OSError: [WinError 145] 目录不是空的。: 'E:\\A\\B'

```
>>> os.remove('E:\\A\\B\\test.txt')
```

```
>>> os.rmdir('E:\\A\\B')
```

- os.removedirs('E:\\A\\B') 删除多层文件目录

OSError: [WinError 145] 目录不是空的

```
>>> os.removedirs('E:\\A\\B')
```

- os.rename('E:\\C\\B\\test1.txt', 'E:\\C\\B\\test2.txt') 重命名 (源文件, 目的文件)

TypeError: rename() takes at most 2 positional arguments (3 given)

```
>>> os.rename('E:\\C\\B\\test1.txt', 'E:\\C\\B\\test2.txt')
```

- os.system('cmd') os.system('calc') 打开cmd 打开计算器
- os.getcwd() 当前目录, 相当于Linux 里面的 '.'。
- os.pardir 上一级目录。相当于Linux里面的 '..'。


```

>>> os.system('cmd')
-1073741510
>>> os.system('calc')
0
>>> os.getcwd()
'...'
>>> os.listdir('.')
['$RECYCLE.BIN', '2017人工智能全套', '2345Downloads', '360Rec', 'BaiduNetdiskDownload', 'C', 'Config.ini', 'Crazy tree.docx', 'DN347', 'EXCEL练习', 'MyDrivers', 'pagefile.sys', 'Program Files', 'python exercise', 'QMDownload', 'QQMusicCache', 'qqpcmgr_docpro', 'QQ文件', 'QQ文件分享', 'record.txt', 'resources', 'System Volume Information', 'test2.txt', 'VGDownloads', 'Visio_V2010_xp85.com', 'zhuang jidashi', '业余爱好.rar', '动感新时代歌曲', '学习笔记', '工作文件', '幻日.docx', '微信', '文学收录', '日语学习', '星火六级晨读英语美文100篇', '机器语言学习', '研究生学习资料', '老妈文件', '计算机学习资料', '迅雷下载', '黑袜和白袜.docx']
>>> os.listdir(os.getcwd())
['$RECYCLE.BIN', '2017人工智能全套', '2345Downloads', '360Rec', 'BaiduNetdiskDownload', 'C', 'Config.ini', 'Crazy tree.docx', 'DN347', 'EXCEL练习', 'MyDrivers', 'pagefile.sys', 'Program Files', 'python exercise', 'QMDownload', 'QQMusicCache', 'qqpcmgr_docpro', 'QQ文件', 'QQ文件分享', 'record.txt', 'resources', 'System Volume Information', 'test2.txt', 'VGDownloads', 'Visio_V2010_xp85.com', 'zhuang jidashi', '业余爱好.rar', '动感新时代歌曲', '学习笔记', '工作文件', '幻日.docx', '微信', '文学收录', '日语学习', '星火六级晨读英语美文100篇', '机器语言学习', '研究生学习资料', '老妈文件', '计算机学习资料', '迅雷下载', '黑袜和白袜.docx']
>>> os.sep()

```

- `os.sep` 输出操作系统特定的路径分隔符 (Win下为'\\', Linux下为'/')
- `os.linesep` 当前平台使用的行终止符 (Win下为'\r\n', Linux下为'\n')
- `os.name` 指代当前使用的操作系统 (包括: 'posix', 'nt', 'mac', 'os2', 'ce', 'java')

```

>>> os.sep
'\\'
>>> os.linesep
'\r\n'
>>> os.name
'nt'
>>>

```

10.2.2 os.path 模块

- `os.path.basename('E:\\A\\B\\sex.avi')` 去掉目录路径, 单独返回文件名
- `os.path.dirname('E:\\A\\B\\sex.avi')` 去掉文件名, 单独返回目录路径
- `os.path.join('C:\\', 'A', 'B', 'C')` 将各部分组合成一个路径名
- `os.path.split('E:\\A\\B\\sex.avi')` 将路径和文件名分开
- `os.path.split('E:\\A\\B\\C')` 默认会把C当做文件名分开
- `os.path.splitext('E:\\A\\B\\sex.avi')` 分离文件名与扩展名, 返回 (f_name, f_extension) 元组

```

type help, copyright, credits or license()
>>> import os
>>> os.path.basename('E:\\A\\B\\sex.avi')
'sex.avi'
>>> os.path.dirname('E:\\A\\B\\sex.avi')
'E:\\A\\B'
>>> os.path.join('C:\\', 'A', 'B', 'C')
'C:\\A\\B\\C'
>>> os.path.split('E:\\A\\B\\sex.avi')
('E:\\A\\B', 'sex.avi')
>>> os.path.split('E:\\A\\B\\C')
('E:\\A\\B', 'C')
>>> os.path.splitext('E:\\A\\B\\sex.avi')
('E:\\A\\B\\sex', '.avi')
>>> os.path.splitext('E:\\A\\B\\C.txt')
('E:\\A\\B\\C', '.txt')

```

- `os.path.getsize('E:\\C\\B\\test2.txt')` 返回指定文件的尺寸，单位是字节
- `time.localtime(os.path.getmtime('E:\\C\\B\\test2.txt'))` 返回指定文件最新的修改时间 (浮点型秒数，可用time模块的gmtime()或localtime()函数换算)
- `time.localtime(os.path.getatime('E:\\C\\B\\test2.txt'))` 返回指定文件最近的访问时间 (浮点型秒数，可用time模块的gmtime()或localtime()函数换算)
- `time.localtime(os.path.getctime('E:\\C\\B\\test2.txt'))` 返回指定文件的创建时间 (浮点型秒数，可用time模块的gmtime()或localtime()函数换算)

```

>>> os.path.getsize('E:\\C\\B\\test2.txt')
0
>>> import time
>>> time.gmtime(os.path.getmtime('E:\\C\\B\\test2.txt'))
time.struct_time(tm_year=2019, tm_mon=6, tm_mday=15, tm_hour=3,
=34, tm_wday=5, tm_yday=166, tm_isdst=0)
>>> os.path.getatime('E:\\C\\B\\test2.txt')
1560568054.0278146
>>> time.localtime(os.path.getmtime('E:\\C\\B\\test2.txt'))
time.struct_time(tm_year=2019, tm_mon=6, tm_mday=15, tm_hour=11,
c=34, tm_wday=5, tm_yday=166, tm_isdst=0)
>>> time.localtime(os.path.getmtime('E:\\C\\B\\test2.txt'))
time.struct_time(tm_year=2019, tm_mon=6, tm_mday=15, tm_hour=13,
ec=30, tm_wday=5, tm_yday=166, tm_isdst=0)
>>> time.localtime(os.path.getctime('E:\\C\\B\\test2.txt'))
time.struct_time(tm_year=2019, tm_mon=6, tm_mday=15, tm_hour=11,
c=34, tm_wday=5, tm_yday=166, tm_isdst=0)

```

以下为函数返回 True 或 False

exists(path)	判断指定路径（目录或文件）是否存在
isabs(path)	判断指定路径是否为绝对路径
isdir(path)	判断指定路径是否存在且是一个目录
isfile(path)	判断指定路径是否存在且是一个文件
islink(path)	判断指定路径是否存在且是一个符号链接
ismount(path)	判断指定路径是否存在且是一个挂载点
samefile(path1, path2)	判断path1和path2两个路径是否指向同一个文件

10.3 pickle 模块（重要）

10.3.1 举例说明

- 使用pickle 模块的原因：

如果使用之前的文件保存的方法，会自动把列表里面的元素都转换成字符串。无论是存储还是调用起来都相当麻烦。而pickle完美地解决了类似的问题。

- pickle 存储

```
11: ( , 0: \ 梨子 , 0 )
>>> import pickle
>>> my_list=[123,3.14,'植树poi',['another list']]
>>> pickle_file=open('my_list.pkl','wb')
>>> pickle.dump(my_list,pickle_file)
>>> pickle_file.close()
\\ import os
```

- pickle 调用

```
>>> os.getcwd()
'C:\\Users\\finlay.wang\\AppData\\Local\\Programs\\Python\\Python37-32'
>>> print('C:\\Users\\finlay.wang\\AppData\\Local\\Programs\\Python\\Py
')
C:\\Users\\finlay.wang\\AppData\\Local\\Programs\\Python\\Python37-32
>>> pickle_file=open('my_list.pkl','rb')
>>> my_list2=pickle.load(pickle_file)
>>> print(my_list2)
[123, 3.14, '植树poi', ['another list']]
\\ import os
```

- 文件格式的存储调用，相当麻烦

```
>>> f=open('my_list.txt','w')
>>> f.write(my_list)
Traceback (most recent call last):
  File "<pyshell#18>", line 1, in <module>
    f.write(my_list)
TypeError: write() argument must be str, not list
>>> f.write(str(my_list))
38
>>> f.readline()
Traceback (most recent call last):
  File "<pyshell#20>", line 1, in <module>
    f.readline()
io.UnsupportedOperation: not readable
>>> f.close()
>>> f=open('my_list.txt','r')
>>> f.readline()
"[123, 3.14, '植树poi', ['another list']]"
>>> f.readline()
''
>>> f.seek(0,0)
0
>>> my_list3=f.read()
>>> mylist3
Traceback (most recent call last):
  File "<pyshell#27>", line 1, in <module>
    mylist3
NameError: name 'mylist3' is not defined
>>> my_list3
"[123, 3.14, '植树poi', ['another list']]"
>>> type(my_list3)
<class 'str'>
>>> list(my_list3)
['[', '1', '2', '3', ',', ',', ',', '3', ',', ',', ',', '1', '4', ',', ',', ',', '"', '"', '植', '树', 'p', 'o', 'i', ',', '[', '"', '"', 'a', 'n', 'o', 't', 'h', 'e', 'r', ' ', 'l', 'i', 's', 't', '"', '"', ']', ']', '']']
>>>
```

10.4.1 常用異常

```

>>> assert len(my_list) > 0
>>> my_list.pop()
'小甲鱼是帅哥'
>>> assert len(my_list) > 0
Traceback (most recent call last):
  File "<pyshell#3>", line 1, in <module>
    assert len(my_list) > 0
AssertionError
>>> my_list.fishc
Traceback (most recent call last):
  File "<pyshell#4>", line 1, in <module>
    my_list.fishc
AttributeError: 'list' object has no attribute 'fishc'
>>> my_list = [1, 2, 3]
>>> my_list[3]
Traceback (most recent call last):
  File "<pyshell#6>", line 1, in <module>
    my_list[3]
IndexError: list index out of range
>>> my_list[2]
3
>>> my_dict = {'one':1, 'two':2, 'three':3}
>>> my_dict['one']
1
>>> my_dict['four']
Traceback (most recent call last):
  File "<pyshell#10>", line 1, in <module>
    my_dict['four']
KeyError: 'four'
>>> my_dict.get('four')
>>> fishc
Traceback (most recent call last):
  File "<pyshell#12>", line 1, in <module>
    fishc
NameError: name 'fishc' is not defined
>>> print 'I love FishC.com'
SyntaxError: invalid syntax
>>>

```

Ln: 41 Col: 1

10.4.2 try-except 语句 (检测异常)

- try-except 语句

try:

检测范围

except Exception 【as reason】:

出现异常 (Exception) 后的处理代码

```

>>> try:
    f=open("why I am a file?.txt")
    print(f.read())
    f.close()
except OSError as reason:
    print('文件出错啦! ==\n 错误的原因是: '+str(reason))
文件出错啦! ==
错误的原因是: [Errno 22] Invalid argument: 'why I am a file?.txt'
>>>

```

- 如果先出现某个异常，则会立即中断处理（短路原则）：

```
>>> try:
    sum=1+'1'
    f=open("why I am a file?.txt")
    print(f.read())
    f.close()
except OSError as reason:
    print('文件出错啦! ==\n 错误的原因是: '+str(reason))
except TypeError as reason:
    print('类型出错啦! ==\n 错误的原因是: '+str(reason))

类型出错啦! ==
错误的原因是: unsupported operand type(s) for +: 'int' and 'str'
>>>
```

- 如果出现了后续except并不包含的异常，则会直接报错处理：

```
>>> try:
    int('abc')
    sum=1+'1'
    f=open("why I am a file?.txt")
    print(f.read())
    f.close()
except OSError as reason:
    print('文件出错啦! ==\n 错误的原因是: '+str(reason))
except TypeError as reason:
    print('类型出错啦! ==\n 错误的原因是: '+str(reason))
```

```
Traceback (most recent call last):
  File "<pyshell#22>", line 2, in <module>
    int('abc')
ValueError: invalid literal for int() with base 10: 'abc'
>>>
```

10.4.3 try-finally 语句

try:

检测范围

except Exception 【as reason】：

出现异常（Exception）后的处理代码

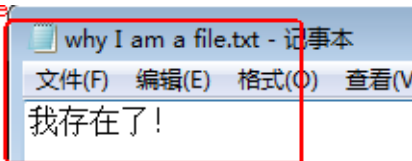
finally:

无论如何都会被执行的代码

- 如果没有finally语句，文件也就不可能会被保存了。

```
AttributeError: _io.TextIOWrapper object has no attribute 'write'
>>> try:
    f=open("why I am a file.txt", 'w')
    print(f.write('我存在了!'))
    int('abc')
    sum=1+'1'
except OSError as reason:
    print('文件出错啦! ==\n 错误的原因是: '+str(reason))
except TypeError as reason:
    print('类型出错啦! ==\n 错误的原因是: '+str(reason))
finally:
    f.close()

5
Traceback (most recent call last):
  File "<pyshell#34>", line 4, in <module>
    int('abc')
ValueError: invalid literal for int() with base 10: 'abc'
>>>
```



10.4.4 raise 语句（自己引发异常）

- raise + 异常名

```

ValueError: invalid literal for int() with base 10:
>>> raise ZeroDivisionError('除数为零的异常')
Traceback (most recent call last):
  File "<pyshell#35>", line 1, in <module>
    raise ZeroDivisionError('除数为零的异常')
ZeroDivisionError: 除数为零的异常
>>> |

```

10.5 else语句搭配

10.5.1 while - else举例说明

《插入图片》

10.5.2 Try-else 语句搭配

```

>>> try:
    int('abc')
except ValueError as reason:
    print('出错啦! 原因是: ' + str(reason))
else:
    print('没有任何异常!')

出错啦! 原因是: invalid literal for int() with base 10: 'abc'
>>> try:
    int('123')
except ValueError as reason:
    print('出错啦! 原因是: ' + str(reason))
else:
    print('没有任何异常!')

123
没有任何异常!
>>>

```



10.6 with 语句

10.6.1 with 语句作用

可以减少代码量, 会自动帮你考虑到文件关闭的问题, 此处就不用f.close()了。

```

>>> try:
    f=open("why I am a file.txt",'w')
    for each_line in f:
        print(each_line)
except OSError as reason:
    print('文件出错啦! ==\n 错误的原因是: ' + str(reason))
except TypeError as reason:
    print('类型出错啦! ==\n 错误的原因是: ' + str(reason))
finally:
    f.close()

文件出错啦! ==
错误的原因是: not readable
>>> try:
    with open("why I am a file.txt",'w') as f:
        for each_line in f:
            print(each_line)
except OSError as reason:
    print('文件出错啦! ==\n 错误的原因是: ' + str(reason))
except TypeError as reason:
    print('类型出错啦! ==\n 错误的原因是: ' + str(reason))

文件出错啦! ==
错误的原因是: not readable
>>>

```

11 easygui 和类

11.1 easygui 学习

11.1.1 easygui 学习资料

[https://fishc.com.cn/forum.php?](https://fishc.com.cn/forum.php?mod=viewthread&tid=46069&extra=page%3D1%26filter%3Dtypeid%26typeid%3D403)

[mod=viewthread&tid=46069&extra=page%3D1%26filter%3Dtypeid%26typeid%3D403](https://fishc.com.cn/forum.php?mod=viewthread&tid=46069&extra=page%3D1%26filter%3Dtypeid%26typeid%3D403)

- 使用pip3 install easygui 进行安装

```
C:\Users\finlay.wang>pip3 install easygui
Collecting easygui
  Using cached https://files.pythonhosted.org/packages/89/b5/fd22bb3eb36085aeb7781670bbc59cf8b641b1774f77578ec06368865aa3/easygui-0.98.1-py2.py3-none-any.whl
Installing collected packages: easygui
Successfully installed easygui-0.98.1
You are using pip version 19.0.3, however version 19.1.1 is available.
You should consider upgrading via the 'python -m pip install --upgrade pip' command.

C:\Users\finlay.wang>
```

安装的地址:

C:\Users\finlay.wang\AppData\Local\Programs\Python\Python37-32\Lib\site-packages里面

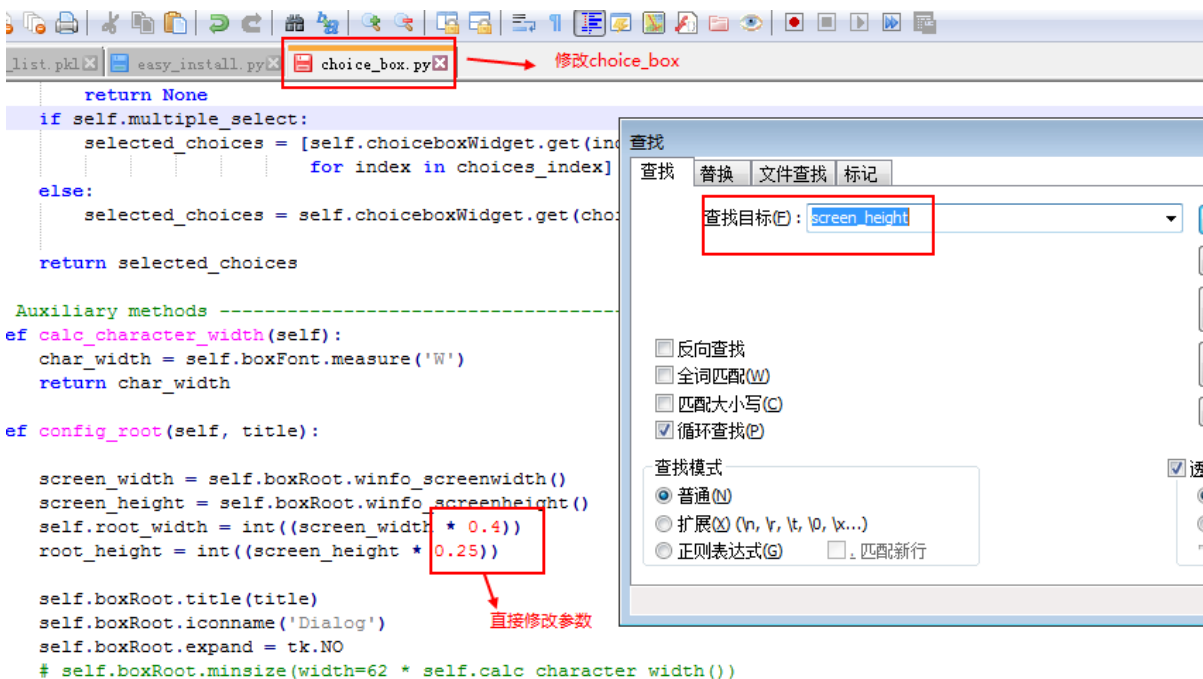
(默认Python3 的site-packages地址)

11.1.2 easygui的使用

```
>>> import easygui
>>> import easygui as g
>>> g.msgbox('hi! 租树poi')
'OK'
>>>
```

- easygui最好不要在IDLE中使用, 因为这个也有类似Shell的程序, 在调用时, 可能会发生事件冲突。

11.1.3 修改默认的宽高



The screenshot shows a Python IDE with a file named `choice_box.py` open. The code defines a class with methods for calculating character width and configuring the root window. The `config_root` method sets `screen_width` and `screen_height` based on the screen dimensions, multiplied by 0.4 and 0.25 respectively. A search dialog is open, showing the search target `screen_height` and the search mode set to '普通(N)'.

```
return None
if self.multiple_select:
    selected_choices = [self.choiceboxWidget.get(index)
                        for index in choices_index]
else:
    selected_choices = self.choiceboxWidget.get(choices_index)
return selected_choices

Auxiliary methods -----
def calc_character_width(self):
    char_width = self.boxFont.measure('W')
    return char_width

def config_root(self, title):
    screen_width = self.boxRoot.winfo_screenwidth()
    screen_height = self.boxRoot.winfo_screenheight()
    self.root_width = int((screen_width * 0.4))
    root_height = int((screen_height * 0.25))

    self.boxRoot.title(title)
    self.boxRoot.iconname('Dialog')
    self.boxRoot.expand = tk.NO
    # self.boxRoot.minsize(width=62 * self.calc_character_width())
```

11.2 面向对象编程

11.2.1 继承

Mylist()继承了list的性质

- python还支持多重继承，多重继承容易造成代码混乱，不建议用。

```
type help, copyright, credits or license() for more information
>>> class Mylist(list):
    pass

>>> list1=Mylist()
>>> list1.append(3)
>>> list1.append(4)
>>> list1.append(8)
>>> list1.append(2)
>>> list1
[3, 4, 8, 2]
>>> list1.sort()
>>> list1
[2, 3, 4, 8]
```

11.2.2 多态

比如func函数，在不同的类别中，有不同的表述意义：

```
>>> class A:
    def func(self):
        print('我是小A.....')

>>> class B:
    def func(self):
        print('我是小B.....')

>>> a=A()
>>> b=B()
>>> a.func()
我是小A.....
>>> b.func()
我是小B.....
>>>
```

11.2.3 self 是什么?

self 表示当前类的对象（一般py将其隐藏了，self指向的是实例对象本身）

```
type help, copyright, credits or license() for more information
>>> class Ball:
    def setName(self,name):
        self.name=name
    def kick(self):
        print('我叫%s,该死的，谁踢我.....' % self.name)

>>> a=Ball()
>>> a.setName('球A')
>>> b=Ball()
>>> b.setName('球B')
>>> c=Ball()
>>> c.setName('土豆')
>>> a.kick()
我叫球A,该死的，谁踢我.....
>>> b.kick()
我叫球B,该死的，谁踢我.....
>>> c.kick()
我叫土豆,该死的，谁踢我.....
>>>
```

此处self表示a这个对象的,在传递时被隐藏了

11.2.3 __init__(self)构造方法（自动调用）

一般类默认参数是__init__(self)，修改后，则必须增加一个name才能调用：

```
>>> class Ball:
    def __init__(self, name):
        self.name = name
    def kick(self):
        print('我叫%s, 该死的, 谁踢我……' % self.name)

>>> a = Ball()
Traceback (most recent call last):
  File "<pyshell#17>", line 1, in <module>
    a = Ball()
TypeError: __init__() missing 1 required positional argument: 'name'
>>> a = Ball('小A')
>>> a.kick()
我叫小A, 该死的, 谁踢我……
>>>
```

11.2.4 公有和私有

在Python中定义私有变量只需要在变量名或函数名前加上 ‘ ’ 两个下划线, 那么这个变量就会变成私有的了。

防止外部修改内部数据

```
>>> class Person:
    name = '小甲鱼'

>>> p = Person()
>>> p.name
小甲鱼
>>> class Person:
    __name = '小甲鱼'

>>> p.name
小甲鱼
>>> p = Person()
>>> p.name
Traceback (most recent call last):
  File "<pyshell#29>", line 1, in <module>
    p.name
AttributeError: 'Person' object has no attribute 'name'
>>> p.__name
Traceback (most recent call last):
  File "<pyshell#30>", line 1, in <module>
    p.__name
AttributeError: 'Person' object has no attribute '__name'
>>>
```

————— 变成私有后重新赋值, 就无法访问了

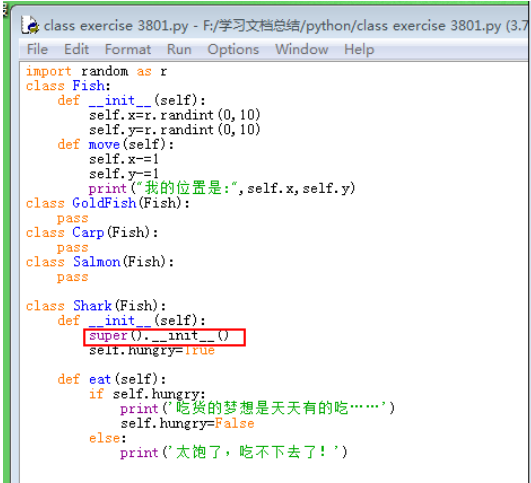
实际上python只是改了个名字:

你可以通过: p._Person__name 进行访问

```
AttributeError: 'Person' object has no attribute 'name'
>>> p._Person__name
小甲鱼
>>>
```

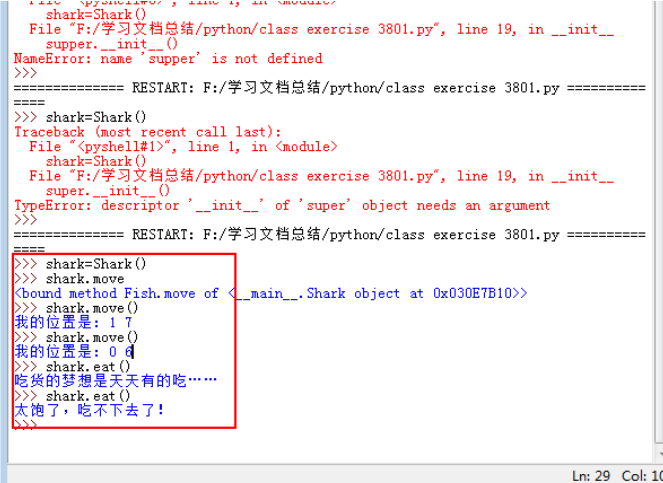
11.2.5 super()直接调用

super().__init__()



```
class exercise 3801.py - F:/学习文档总结/python/class exercise 3801.py (3.7)
File Edit Format Run Options Window Help

import random as r
class Fish:
    def __init__(self):
        self.x = r.randint(0, 10)
        self.y = r.randint(0, 10)
    def move(self):
        self.x -= 1
        self.y -= 1
        print('我的位置是:', self.x, self.y)
class GoldFish(Fish):
    pass
class Carp(Fish):
    pass
class Salmon(Fish):
    pass
class Shark(Fish):
    def __init__(self):
        super().__init__()
        self.hungry = True
    def eat(self):
        if self.hungry:
            print('吃货的梦想是天天有吃的……')
            self.hungry = False
        else:
            print('太饱了, 吃不下去了!')
```



```
==== RESTART: F:/学习文档总结/python/class exercise 3801.py =====
>>> shark = Shark()
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    shark = Shark()
  File "F:/学习文档总结/python/class exercise 3801.py", line 19, in __init__
    super().__init__()
NameError: name 'super' is not defined
>>>
==== RESTART: F:/学习文档总结/python/class exercise 3801.py =====
>>> shark = Shark()
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    shark = Shark()
  File "F:/学习文档总结/python/class exercise 3801.py", line 19, in __init__
    super().__init__()
TypeError: descriptor '__init__' of 'super' object needs an argument
>>>
==== RESTART: F:/学习文档总结/python/class exercise 3801.py =====
>>> shark = Shark()
>>> shark.move()
Bound method Fish.move of <_main_.Shark object at 0x030E7B10>
>>> shark.move()
我的位置是: 1 7
>>> shark.move()
我的位置是: 0 6
>>> shark.eat()
吃货的梦想是天天有的吃……
>>> shark.eat()
太饱了, 吃不下去了!
>>>
```

11.2.6 组合

self.turtle /self.fish 就相当于一个对象名

```
>>> class Turtle:
    def __init__(self,x):
        self.x=x

>>> class Fish:
    def __init__(self,x):
        self.x=x

>>> class Pool:
    def __init__(self,x,y):
        self.turtle=Turtle(x)
        self.fish=Fish(y)

>>> class Pool:
    def __init__(self,x,y):
        self.turtle=Turtle(x)
        self.fish=Fish(y)
    def print_num(self):
        print('水池里有乌龟%d只，鱼%d只'%(self.turtle.x,self.fish.x))

>>> pool=Pool(1,10)
>>> pool.print_num()
水池里有乌龟1只，鱼10只
>>>
```

[https://fishc.com.cn/forum.php?](https://fishc.com.cn/forum.php?mod=viewthread&tid=48888&extra=page%3D1%26filter%3Dtypeid%26typeid%3D403)

[mod=viewthread&tid=48888&extra=page%3D1%26filter%3Dtypeid%26typeid%3D403](https://fishc.com.cn/forum.php?mod=viewthread&tid=48888&extra=page%3D1%26filter%3Dtypeid%26typeid%3D403)

类的属性名可以覆盖方法名

11.3 一些与类相关的BIF

- `issubclass(D,S)` 判断D是否是S的子类，继承关系

```
type neip, copyright,
>>> class A:
    pass

>>> class B(A):
    pass

>>> issubclass(B,A)
True
>>>
```

- `isinstance(object,classinfo)` 判断是否是后面类的实例化对象

- 1、第一传递的不是object类型，就永远会返回一个False。
- 2、第二个参数如果不是由类组成或者由类对象组成的元组，就会抛出一个TypeError的异常。

```
>>> class C:
    pass

>>> b1=B()
>>> isinstance(b1,C)
False
>>> isinstance(b1,B)
True
>>>
```

- `hasattr(object,name)` 判断对象是否有内置name这个属性
- `getattr(object,name,'默认返回')` 返回目标object里面name的属性值，如果没有返回后面语句。

- setattr(object,name,values) 如果object没有对应属性，则新建一个name属性，并贴于values的值上。
- delattr (object, name) 删除object对应的name属性，如果没有name属性则报错。

```
>>> class C:
    def __init__(self,x=0):
        self.x=x

>>> c1=C()
>>> hasattr(c1,'x')
True
>>> getattr(c1,'x')
0
>>> getattr(c1,'y','您所访问的属性不存在')
'您所访问的属性不存在'
>>> setattr(c1,'y','您所访问的属性不存在')
>>> c1.y
'您所访问的属性不存在'
>>> delattr(c1,'y')
>>> delattr(c1,'y')
Traceback (most recent call last):
  File "<pyshell#29>", line 1, in <module>
    delattr(c1,'y')
AttributeError: y
\\>>>
```

11.3.1 property 函数

通过属性去设置属性，可以方便设置用户的调用接口，用户用X，我们自己函数内部该动就不会影响用户使用了。

```
>>> class C:
    def __init__(self,size=10):
        self.size=size
    def getSize(self):
        return self.size
    def setSize(self,value):
        self.size=value
    def delSize(self):
        del self.size
    x=property(getSize,setSize,delSize)

>>> c1=C()
>>> c1.getSize()
10
>>> c1.x
10
>>> c1.x=18
>>> c1.x
18
>>> c1.size
18
>>> del c1.x
>>> c1.size
Traceback (most recent call last):
  File "<pyshell#48>", line 1, in <module>
    c1.size
AttributeError: 'C' object has no attribute 'size'
\\>>>
```

12 魔法方法

12.1 魔法方法的概述

- 魔法方法总是被双下划线包围，例如__init__

- 魔法方法是面向对象的Python的一切，如果你不知道魔法方法，说明你还没有意识到面向对象的Python的强大
- 魔法方法的“魔力”体现在他们总能够在适当的时候自动被调用。

12.1.1 `__init__` (`self[,...]`)构造和析构

- `__init__()`这个魔法方法必须返回一个none

```
>>> class Rectangle:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def getPeri(self):
        return (self.x + self.y) * 2
    def getArea(self):
        return self.x * self.y

>>> rect = Rectangle(3, 4)
>>> rect.getPeri
<bound method Rectangle.getPeri of <__main__.Rectangle object>
>>> rect.getPeri()
14
>>> rect.getArea()
12
>>>
```

- `__new__(cls[,...])`方法 极少数会重写 也是一个类第一个调用的方法

继承一个不可变class的时候，提供给你一些自定义这些类的实例化过程的途径，这时就可以通过改动`__new__()`这个魔法方法，来完成

```
>>> class CapStr(str):
    def __new__(cls, string):
        string = string.upper()
        return str.__new__(cls, string)

>>> a = CapStr('I love FishC.com!')
>>> a
'I LOVE FISHC.COM!'
>>>
```

- `__del__(self)`

```
>>> class C:
    def __init__(self):
        print("我是init方法 我被调用了……")
    def __del__(self):
        print("我是del方法，我被调用了……")

>>> c1 = C()
我是init方法 我被调用了……
>>> c2 = c1
>>> c3 = c2
>>> del c3
>>> del c2
>>> del c1
我是del方法，我被调用了……
```

12.2 类的算术运算

12.2.1 如何看出类可以算术运算？

```

type help, copyright, credits or license() for more information.
>>> type(len)
<class 'builtin_function_or_method'>
>>> type(int)
<class 'type'>
>>> class C:
    pass
>>> type(C)
<class 'type'>
>>> type(int('123'))
<class 'int'>
>>> a=int('123')
>>> b=int('456')
>>> a+b
579
>>>

```

原来int是一个类啊!

原来两个类是可以相加的啊!

12.2.2 把加法改写成减法

- 如何改写类里面的内置方法?

```

>>> class New_int(int):
    def __add__(self, other):
        return int.__sub__(self, other)
    def __sub__(self, other):
        return int.__add__(self, other)

>>> a=New_int(3)
>>> b=New_int(5)
>>> a+b
-2
>>> a-b
8
>>>

```

12.2.3 内置方法列表

<code>__add__(self, other)</code>	定义加法的行为：+
<code>__sub__(self, other)</code>	定义减法的行为：-
<code>__mul__(self, other)</code>	定义乘法的行为：*
<code>__truediv__(self, other)</code>	定义真除法的行为：/
<code>__floordiv__(self, other)</code>	定义整数除法的行为：//
<code>__mod__(self, other)</code>	定义取模算法的行为：%
<code>__divmod__(self, other)</code>	定义当被 <code>divmod()</code> 调用时的行为
<code>__pow__(self, other[, modulo])</code>	定义当被 <code>power()</code> 调用或 <code>**</code> 运算时的行为
<code>__lshift__(self, other)</code>	定义按位左移位的行为：<<
<code>__rshift__(self, other)</code>	定义按位右移位的行为：>>
<code>__and__(self, other)</code>	定义按位与操作的行为：&
<code>__xor__(self, other)</code>	定义按位异或操作的行为：^
<code>__or__(self, other)</code>	定义按位或操作的行为：

订正一下： `divmod(a, b)`
返回的值是一个元祖：
(`a//b`, `a%b`)

12.2.4 反运算

举例说明：

`a+b` 按常理来说应该先执行`a`的`add`运算，但如果`a`的没有执行，系统会找到`b`的执行，`b`的`add`就是反运算。

```
===== RESTART: Shell =====
>>> class Nint(int):
        def __radd__(self, other):
            return int.__sub__(self, other)

>>> a=Nint(5)
>>> b=Nint(3)
>>> a+b
8
>>> 1+b
2
>>>
```

1没有add方法，调用b的故为2

- 重写反运算时要注意顺序问题

12.2.5 属性访问

`__getattr__(self, name)`

-定义当用户试图获取一个不存在的属性时的行为

`__getattr__(self,name)`

-定义当该类的属性被访问是的行为

`__setattr__(self,name,value)`

-定义当一个属性设置时的行为

`__delattr__(self,name)`

-定义当一个属性被删除时的行为

```
>>> class C:
    def __getattr__(self, name):
        print('getattr')
        return super().__getattr__(name)
    def __getattr__(self, name):
        print('getattr')
    def __setattr__(self, name, value):
        print('setattr')
        super().__setattr__(name, value)
    def __delattr__(self, name):
        print('delattr')
        super().__delattr__(name)

>>> c=C()
>>> c.x
getattr
>>> c.x=1
setattr
>>> del c.x
delattr
>>>
```

利用收集参数变动的去定义一个传入参数（自己想的）

```
>>>
===== RESTART: P
>>> r1=Rectangle(4)
>>> r1.height
4
>>> r1.width
4
>>> r1.getArea()
16
>>> r2=Rectangle(6, 7)
>>> r2.height
6
>>> r2.width
7
>>> r2.getArea()
42
>>>
```

```
class Rectangle:
    def __init__(self, *params):
        if len(params)==1:
            self.square=params[0]
        elif len(params)==2:
            self.height=params[0]
            self.width=params[1]
        else:
            print('请按正确的格式输入')

    def __setattr__(self, name, value):
        if name=='square':
            self.width=value
            self.height=value
        else:
            super().__setattr__(name, value)

    def getArea(self):
        return self.width*self.height
```

描述符知识点

描述符就是将某种特殊类型的类的实例指派给另一个类的属性

`__get__(self,instance,owner)`

-用于访问属性，它返回属性的值

`__set__(self,instance,value)`

-将在属性分配操作中调用，不返回任何内容

`__delete__(self,instance)`

-控制删除操作，不返回任何内容

- 用例子展示先关用法


```

SyntaxError: unexpected indent
>>> class MyDescriptor:
    def __get__(self, instance, owner):
        print('getting...', self, instance, owner)
    def __set__(self, instance, owner):
        print('setting...', self, instance, owner)
    def __delete__(self, instance):
        print('delete...', self, instance)

>>> class Test:
    x=MyDescriptor()

>>> test=Test()
>>> test.x
getting... <__main__.MyDescriptor object at 0x037E8E10> <__main__.Test object at 0x037E8D90> <class '__main__.Test'>
>>> test
<__main__.Test object at 0x037E8D90>
>>> Test
<class '__main__.Test'>
>>> test.x='X-man'
setting... <__main__.MyDescriptor object at 0x037E8E10> <__main__.Test object at 0x037E8D90> X-man
>>>
>>> del test.x
delete... <__main__.MyDescriptor object at 0x037E8E10> <__main__.Test object at 0x037E8D90>
>>> class MyPropertyDescriptor:

AttributeError: 'C' object has no attribute '_x'
>>> MyDescriptor()
<__main__.MyDescriptor object at 0x0359D870>
>>>

```

- 自行设计property函数

```

...
>>> class Myproperty:
    def __init__(self, fget=None, fset=None, fdel=None):
        self.fget=fget
        self.fset=fset
        self.fdel=fdel
    def __get__(self, instance, owner):
        return self.fget(instance)
    def __set__(self, instance, value):
        self.fset(instance, value)
    def __delete__(self, instance):
        self.fdel(instance)

>>> class C:
    def __init__(self):
        self._x=None
    def getX(self):
        return self._x
    def setX(self, value):
        self._x=value
    def delX(self):
        del self._x
    x=Myproperty(getX, setX, delX)

>>> c=C()
>>> c.x='X-man'
>>> c.x
'X-man'
>>> c._x
'X-man'
>>> del c.x
>>> c.x
Traceback (most recent call last):
  File "<pyshell#97>", line 1, in <module>
    _

```

这里的实例应该是c，内部以c._x作为传递，Myproperty将三个函数getX，setX，delX整合在一起了。

容器类型的协议

- 如果你希望定制的容器是不可变的话，你只需要定义__len__()和__getitem__()方法。

- 如果你希望定制的容器是可变的话，除了

`__len__()`和`__getitem__()`方法，你还需要定义`__setitem__()`和`__delitem__()`两个方法。

```
>>> class Countlist:
    def __init__(self, *args):
        self.values=[x for x in args]
        self.count={}.fromkeys(range(len(self.values)),0)

>>> class Countlist:
    def __init__(self, *args):
        self.values=[x for x in args]
        self.count={}.fromkeys(range(len(self.values)),0)
    def __len__(self, values):
        return len(self.values)
    def __getitem__(self, key):
        self.count[key]+=1
        return self.values[key]

>>> c1=Countlist(1,3,5,7,9)
>>> c2=Countlist(2,4,6,8,10)
>>> c1[1]
3
>>> c1[1]+c2[1]
7
>>> c1.count
{0: 0, 1: 2, 2: 0, 3: 0, 4: 0}
>>> c2.count
{0: 0, 1: 1, 2: 0, 3: 0, 4: 0}
>>> c1[1]+c2[1]
7
>>> c1.count
{0: 0, 1: 3, 2: 0, 3: 0, 4: 0}
>>> ce.count
Traceback (most recent call last):
  File "<pyshell#21>", line 1, in <module>
    ce.count
NameError: name 'ce' is not defined
>>> c2.count
{0: 0, 1: 2, 2: 0, 3: 0, 4: 0}
>>>
```

迭代

- Python中自带的迭代操作

`iter()` `__iter__()` 返回迭代器本身

`next()` `__next__()` 返回迭代器的迭代规则

```
type help, copyright, credits or license() i
>>> string='FishC'
>>> it=iter(string)
>>> next(it)
'F'
>>> next(it)
'i'
>>> next(it)
's'
>>> next(it)
'h'
>>> next(it)
'C'
>>> next(it)
Traceback (most recent call last):
  File "<pyshell#7>", line 1, in <module>
    next(it)
StopIteration
>>>
```

费布拉奇数列：

```

StopIteration
>>> class Fibs:
    def __init__(self, n=20):
        self.a=0
        self.b=1
        self.n=n
    def __iter__(self):
        return self
    def __next__(self):
        self.a, self.b = self.b, self.a+self.b
        if self.a>self.n:
            raise StopIteration
        return self.a

>>>
>>> fbs=Fibs()
>>> for each in fbs:
    print(each)

1
1
2
3
5
8
13
>>>

```

生成器

yield 所谓的协同程序就是可以运行的独立函数调用，函数可以暂停或者挂起，并在需要的时候从程序离开的地方继续或者重新开始。

```

SyntaxError: invalid syntax
>>> def myGen():
    print('生成器被执行')
    yield 1
    yield 2

>>> myG=myGen()
>>> next(myG)
生成器被执行
1
>>> next(myG)
2
>>> next(myG)
Traceback (most recent call last):
  File "<pyshell#9>", line 1, in <module>
    next(myG)
StopIteration
>>> for i in myGen():
    print(i)

生成器被执行
1
2

```

for语句在Python 里就是一个生成器

```

True, 9: False}
>>> e=(i for i in range(10))
>>> e
<generator object <genexpr> at 0x03763A30>
>>>

```

模块

- 容器----->对数据的封装 列表 元组，字符串
- 函数----->语句的封装
- 类 ----->方法和属性的封装
- 模块----->模块就是程序


```

/ht<1>'
>>> html=html.decode("UTF-8")
>>> print(html)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.1
  <meta name="keywords" content="鱼C工作室|免费编程视频教程教学|Python教学
教学|全栈开发教学|C语言教学|汇编教学|Win32开发|加密与解密|Linux教学">
  <meta name="description" content="鱼C工作室为大家提供最有趣的编程视:
>
  <meta name="author" content="鱼C工作室">
  <title>鱼C工作室-免费编程视频教程教学|Python教学|Web开发教学|全栈开发教:
教学|汇编教学|Win32开发|加密与解密|Linux教学</title>
  <link rel="shortcut icon" type="image/x-icon" href="img/favicon.ico"

```

从网站上爬取一张小猫的照片

```

import urllib.request

req=urllib.request.Request('http://placekitten.com/200/300')
response=urllib.request.urlopen(req)
#response=urllib.request.urlopen('http://placekitten.com/200/300')
cat_img=response.read()

with open('cat_200_300.jpg','wb') as f:
    f.write(cat_img)

```

```

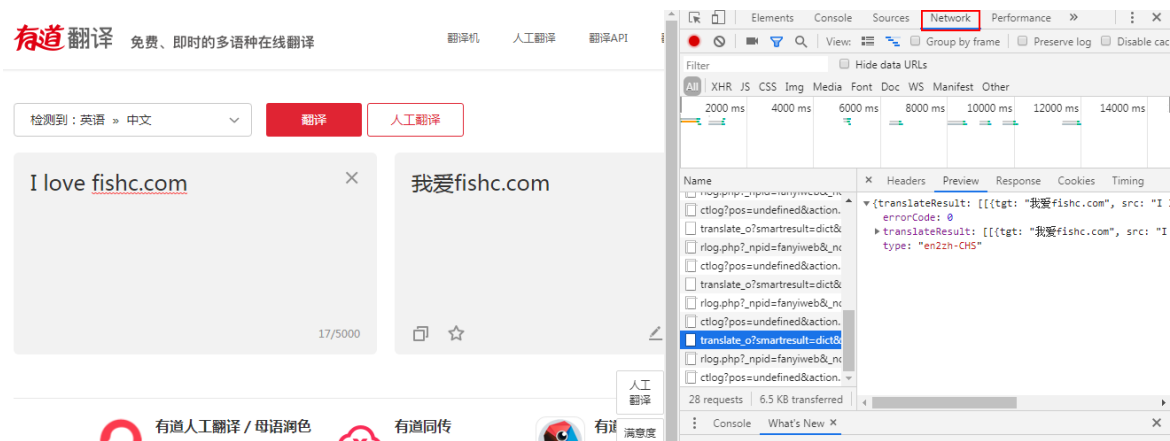
===== RESTART: D:\finlay.wang\桌面\downloadpet.py =====
>>> response.geturl()
'http://placekitten.com/200/300'
>>> response.info
<bound method HTTPResponse.info of <http.client.HTTPResponse object at 0x038841F0>
>>> response.info()
<http.client.HTTPMessage object at 0x03884F30>
>>> print(response.info())
Date: Mon, 08 Jul 2019 06:38:39 GMT
Content-Length: 6545
Connection: close
Set-Cookie: __cfduid=d84827b16d249aba04c89b375d58118321562567919; expires=Tue, 0
7-Jul-20 06:38:39 GMT; path=/; domain=.placekitten.com; HttpOnly
Access-Control-Allow-Origin: *
Cache-Control: public, max-age=86400
Expires: Tue, 09 Jul 2019 06:38:39 GMT
CF-Cache-Status: HIT
Age: 35297
Accept-Ranges: bytes
Vary: Accept-Encoding
Server: cloudflare
CF-RAY: 4f300e7b3c7135c4-LHR

>>> response.getcode() 返回http的状态
200
>>>

```

利用有道词典翻译文本

按下F12打开下面工具栏，选择network



有道词典F12的简单介绍



此处为个人的访问信息处



此为为翻译的内容



代码阅读：

```

import urllib.request
import urllib.parse
import json

content=input('请输入需要翻译的内容:')

url='http://fanyi.youdao.com/translate?_osmartresult=dict&smartresult=rule'

data={}
data['i']=content
data['from']='AUTO'
data['to']='AUTO'
data['smartresult']='dict'
data['client']='fanyideskweb'
data['salt']='15625684503083'
data['sign']='d7cf30a414ea256b605480a7015e42e3'
data['ts']='1562568450308'
data['bv']='6463522ba46bac94c96fd37965fadc8d'
data['doctype']='json'
data['version']='2.1'
data['keyfrom']='fanyi.web'
data['action']='FY_BY_REALTIME'

#encode 将其他格式转换成utf-8的格式。
data=urllib.parse.urlencode(data).encode('utf-8')

response=urllib.request.urlopen(url,data)
#decode 将utf-8的编码格式解码成其他格式。
html=response.read().decode('utf-8')

target=json.loads(html)
print('翻译结果: %s'% (target['translateResult'][0][0]['tgt']))

```

结果查询:

```

'''
===== RESTART: D:\finlay.wang\桌面\translation.py ===
请输入需要翻译的内容: 你好啊
翻译结果: How are you?
>>>
'''

```

修改header操作

1、直接新建一个列表head

```

head={}
head['User-Agent']='Mozilla/5.0 (Windows NT 6.1; Win64; x64) AppleWebKit/537.36

```

然后再req里面追加一个head

req=urllib.request.Request(url,data,head) 即可。

2、通过req.add_header('User-Agent','Mozilla/5.0 (Windows NT 6.1; Win64; x64).....)来直接添加header即可。

结果查询验证:

```

===== RESTART: F:\子习义何思结\python\translation00.py =====
=====
请输入需要翻译的内容: love
翻译结果: 爱
>>> req.headers
{'User-agent': 'Mozilla/5.0 (Windows NT 6.1; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/75.0.3770.100 Safari/537.36'}
>>>

```

如何避开检查

1、使用时间模块:


```
import time
while True:
    content=input('请输入需要翻译的内容(q!退出程序):')
    if content=='q!':
        break
```

设置每5秒重新查询一次:

```
time.sleep(5)
```

查询的结果:

```
>>>
===== RESTART: F:\学习文档总结\python\translation55.py =====
=====
请输入需要翻译的内容(q!退出程序): ai
翻译结果: ai
请输入需要翻译的内容(q!退出程序): 你好啊
翻译结果: How are you?
请输入需要翻译的内容(q!退出程序): 小甲鱼是乌龟
翻译结果: The little turtle is the tortoise
请输入需要翻译的内容(q!退出程序): q!
>>>
```

2、调用代理

代理

- 步骤：
 1. 参数是一个字典 { '类型': '代理ip:端口号' }
`proxy_support = urllib.request.ProxyHandler({})`
 2. 定制、创建一个 opener
`opener = urllib.request.build_opener(proxy_support)`
 - 3a. 安装 opener
`urllib.request.install_opener(opener)`
 - 3b. 调用 opener 
`opener.open(url)`

增加了改动header和随机选择代理的功能，未来可以用pickle打包


```

import urllib.request
import random

url='http://www.whatismyip.com.tw'
iplist=['119.6.144.73:81','183.203.208.166:8118','111.1.32.28:81']
proxy_support=urllib.request.ProxyHandler({'http':random.choice(iplist)})

opener=urllib.request.build_opener(proxy_support)
opener.addheaders=[('User-Agent','Mozilla/5.0 (Windows NT 6.1; Win64; x64) App

urllib.request.install_opener(opener)

response=urllib.request.urlopen(url)
html=response.read().decode('utf-8')

print(html)

```

```

import urllib.request
import random

```

```

url='http://www.whatismyip.com.tw'
iplist=['119.6.144.73:81','183.203.208.166:8118','111.1.32.28:81']
proxy_support=urllib.request.ProxyHandler({'http':random.choice(iplist)})

opener=urllib.request.build_opener(proxy_support)
opener.addheaders=[('User-Agent','Mozilla/5.0 (Windows NT 6.1; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/75.0.3770.100 Safari/537.36')]

urllib.request.install_opener(opener)

response=urllib.request.urlopen(url)
html=response.read().decode('utf-8')

print(html)

```

如何爬取网页上的图片:

```

import urllib.request
import os
import random

def url_open(url):
    req=urllib.request.Request(url)
    req.add_header('User-Agent','Mozilla/5.0 (Windows NT 6.1; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/75.0.3770.100 Safari/537.36')

```

```
proxies=['119.16.88:81']
proxy=random.choice(proxies)
```

```
proxy_support=urllib.request.ProxyHandler({'http':proxy})
opener=urllib.request.build_opener(proxy_support)
urllib.request.install_opener(opener)
```

```
response=urllib.request.urlopen(url)
html=response.read().decode('utf-8')
```

```
return html
```

```
def get_page(url):
    html=url_open(url).decode('utf-8')

    a=html.find('current-comment-page')+23
    b=html.find(']',a)
    return html[a:b]
```

```
def find_imgs(url):
    html=url_open(url).decode('utf-8')
    img_addrs=[]

    a= html.find('img src=')
    while a != -1:
        b= html.find('.jpg',a,a+255)
        if b !=-1:
            img_addrs.append(html[a+9,b+4])
        else:
            b=a+9
        a=html.find('img src=',b)

    return img_addrs
```

```
def save_imgs(folder,img_addrs):
    for each in img_addrs:
```

```
filename=each.split('/')[-1]
with open(filename,'wb') as f:
    img=url_open(each)
    f.write(img)
```

```
def down_load_mm(folder='OOXX',pages=10):
    os.mkdir(folder)
    os.chdir(folder)
```

```
url='http://http://jandan.net/ooxx'
```

```
page_num=int(get_page(url))
```

```
for i in range(pages):
    page_num-=1
    page_url=url+'/page-'+str(page_num)+'#comments'
    img_addrs=find_imgs(page_url)
    save_imgs(img_addrs)
```

```
if __name__=='__main__':
    download_mm()
```

使用正则表达式：

```

type: http, Copyright, Credits of Alibaba for more information
>>> import re
>>> re.search('FishC', 'I love FishC.com!')
<re.Match object; span=(7, 12), match='FishC'>
>>> re.search(r'FishC', 'I love FishC.com!')
<re.Match object; span=(7, 12), match='FishC'>
>>> re.search(r'.', 'I love FishC.com!')
<re.Match object; span=(0, 1), match='I'>
>>> re.search(r'Fish.', 'I love FishC.com!')
<re.Match object; span=(7, 12), match='FishC'>
>>> # .号配合任何字符，需要匹配自身 需加反斜杠
>>> re.search(r'\.', 'I love FishC.com!')
<re.Match object; span=(12, 13), match='.'>
>>> re.search(r'\d', 'I love 123FishC.com!')
<re.Match object; span=(7, 8), match='1'>
>>> re.search(r'\d\d\d', 'I love 123FishC.com!')
<re.Match object; span=(7, 10), match='123'>
>>> #匹配IP地址
>>> re.search(r'\d\d\d\.\d\d\d\.\d\d\d\.\d\d\d', '192.168.111.123')
<re.Match object; span=(0, 15), match='192.168.111.123'>
>>> re.search(r'\d\d\d\.\d\d\d\.\d\d\d\.\d\d\d', '192.168.1.01')
>>> #为了解决上述的问题，我们发明了一个字符类，字符类表示，只要匹配里面的任何一个字符都算匹配成功。
>>> re.search(r'[aeiou]', 'I love 123FishC.com!')
<re.Match object; span=(3, 4), match='o'>
>>> #正则表达式区分大小写
>>> re.search(r'[a-z]', 'I love 123FishC.com!')
<re.Match object; span=(2, 3), match='l'>
>>> re.search(r'[2-9]', 'I love 123FishC.com!')
<re.Match object; span=(8, 9), match='2'>
>>> #这样就可以限制IP地址的范围了0-255
>>> #如何限定重复匹配的次数
>>> re.search(r'ab{3}c', 'abbbbc')
<re.Match object; span=(0, 5), match='abbbc'>
>>> re.search(r'ab{3}c', 'abbbbbbc')
>>> re.search(r'ab{3,10}c', 'abbbbbbc')
<re.Match object; span=(0, 8), match='abbbbbbc'>
>>> #上述表示匹配重复次数范围为3-10次
>>> #正则表达式只匹配字符串
>>> re.search(r'[01]\d\d|2[0-4]\d|25[0-5]', '188')
<re.Match object; span=(0, 3), match='188'>
>>> #百位0或1则十位，个位为任意数；百位为2，十位为0-4，则个位为任意；百位为2，十位为5，个位只能为0-5。正则表达式表示IP地址的方法。
>>>

```

IP地址用正则表达式表示：

```

0-4]\d|2[0-5])', '192.168.1.1')
>>> re.search(r'((01){0,1}\d{0,1}\d|2[0-4]\d|25[0-5])\.){3}([01]{0,1}\d{0,1}\d|2[0-4]\d|25[0-5])', '192.168.1.1')
<re.Match object; span=(0, 11), match='192.168.1.1'>
>>>

```

正则表达式部分用法：

```

type help, copyright, credits or license() for more informat
>>> import re
>>> re.search(r'Fish(0|1)', 'FishC')
<re.Match object; span=(0, 5), match='Fish1'>
>>> re.search(r'Fish(0|1)', 'FishU')
<re.Match object; span=(0, 5), match='Fish0'>
>>> re.search(r'FishC', 'FishC.com!')
<re.Match object; span=(0, 5), match='FishC'>
>>> re.search(r'FishC', 'I love FishC.com!')
<re.Match object; span=(0, 5), match='FishC'>
>>> re.search(r'FishC$', 'I love FishC')
<re.Match object; span=(7, 12), match='FishC'>
>>> re.search(r'(FishC)\1', 'FishC.com!')
<re.Match object; span=(0, 10), match='FishCFishC'>
>>> re.search(r'(FishC)\1', 'FishCFishC')
<re.Match object; span=(0, 10), match='FishCFishC'>
>>> re.search(r'(FishC)\060', 'FishCFishC0')
<re.Match object; span=(5, 11), match='FishC0'>
>>> re.search(r'(FishC)\141', 'FishCFishCa')
<re.Match object; span=(5, 11), match='FishCa'>
>>> re.search(r'[.]', 'I love FishC.com!')
<re.Match object; span=(12, 13), match='.'>
>>> re.findall(r'[a-z]', 'I love FishC.com!')
['l', 'o', 'v', 'e', 'i', 's', 'h', 'c', 'o', 'm']
>>> re.findall(r'[\n]', 'I love FishC.com\n')
['\n']
>>> re.findall(r'[^a-z]', 'I love FishC.com\n')
['I', ' ', ' ', ' ', ' ', 'F', ' ', 'C', ' ', ' ', '\n']
>>> #上面表示取反
>>> re.search(r'fishC{3}', 'I love FishCCC.com!')
>>> re.search(r'fishC{3}', 'I love fishccc.com!')
<re.Match object; span=(7, 14), match='fishccc'>
>>> #所以必须加上括号, 作为一个整体括起来
>>> # * 表示匹配{0, } + 表示匹配{1, } ? 表示匹配{0, 1}
>>> #正则表达式启动了贪心 只要符合的条件下 会尽可能地多匹配

```

\+ 普通字母构成的正则表达式:

此处的\B表示py后面接字母有几个, 如果后面接.或者空格, 则会忽略, 反义词为\b

```

['py']
>>> re.findall(r'py\B', 'python!pythonpy.py ')
['py', 'py']
>>> re.findall(r'\w' '我等角CT作宏')

```

正则表达式的编译:

此时p就相当于一个正则表达式了。

```

>>> p=re.compile(r'[A-Z]')
>>> type(p)
<class 're.Pattern'>
>>> p.search('I love FishC.com!')
<re.Match object; span=(0, 1), match='I'>
>>> p.findall('I love FishC.com!')
['I', 'F', 'C']

```

正则表达式开启详细注释模式, verbose

```

1. charref = re.compile(r"""
2.    &[#]                # 开始数字引用
3.    (
4.        0[0-7]+         # 八进制格式
5.        | [0-9]+        # 十进制格式
6.        | x[0-9a-fA-F]+ # 十六进制格式
7.    )
8.    ;                    # 结尾分号
9. """, re.VERBOSE)

```

复制代码

正则表达式使用方法:

使用group可以显示查找的元素:

使用start, end, span 可以显示查找的开始, 结尾, 范围:

```
TypeError: search() missing 1 required positional argument:
>>> result=re.search(r' (\w+) (\w+)', 'I love fishC.com!')
>>> result
<re.Match object; span=(1, 12), match=' love fishC'>
>>> result.group()
' love fishC'
>>> result.group(1)
'love'
>>> result.group(2)
'fishC'
>>> result.start()
1
>>> result.end()
12
<built-in method end of re.Match object at 0x0381AAD0>
>>> result.end()
12
>>> result.span()
(1, 12)
>>>
```

- findall 会自动地把小括号里面的作为一个元素括起来发送给使用者:

```
import urllib.request
import re

def url_open(url):
    req=urllib.request.Request(url)
    req.add_header('User-Agent', 'Mozilla/5.0 (Windows NT 6.1; Win64; x64) A
    page=urllib.request.urlopen(req)
    html=page.read().decode('utf-8')

    return html

def get_img(html):
    #src[]内部的双引号表示, 除了双引号的数重复1次或多次
    p=r'((?:(?![0-9]|?d\d|2[0-4]|d|25[0-5])\.) ){3} (?:(?![0-9]|?d\d|2[0-4]|d|25[0-5])'
    iplist=re.findall(p,html)

    for each in iplist:
        print(each)

if __name__=='__main__':
    url='http://www.89ip.cn/'
    get_img(url_open(url))
```

```
Python 3.7.3 (v3.7.3:ef4ec6ed12, Mar
1) on win32
Type "help", "copyright", "credits" o
>>>
===== RESTART: F:\学习文档总
=====
('11.', '11', '119')
('182.', '182', '24')
('86.', '86', '158')
('203.', '203', '62')
('177.', '177', '120')
('16.', '16', '20')
('90.', '90', '23')
('174.', '174', '22')
('158.', '158', '34')
('247.', '247', '86')
('152.', '152', '139')
('42.', '42', '179')
>>>
```

改进之后的, 增加?: 表示扩展语法: 这时就不会自动包装成元组了

```
def get_img(html):
    #src[]内部的双引号表示, 除了双引号的数重复1次或多次
    p=r'((?:(?![0-9]|?d\d|2[0-4]|d|25[0-5])\.) ){3} (?:(?![0-9]|?d\d|2[0-4]|d|25[0-5])'
    iplist=re.findall(p,html)

    for each in iplist:
        print(each)

if __name__=='__main__':
    url='http://www.89ip.cn/'
    get_img(url_open(url))
```

```
=====
123.207.11.119
118.24.182.24
42.238.86.158
114.252.203.62
119.29.177.120
117.127.16.20
111.59.90.23
119.179.174.22
103.233.158.34
123.207.247.86
123.195.152.139
106.15.42.179
>>>
```

import urllib.error 导入错误库

urllib.error.HTTPError 网页错误库

一般处理异常的写法:

```

from urllib.request import Request, urlopen
from urllib.error import URLError
req = Request(someurl)
try:
    response = urlopen(req)
except URLError as e:
    if hasattr(e, 'reason'):
        print('We failed to reach a server.')
        print('Reason: ', e.reason)
    elif hasattr(e, 'code'):
        print('The server couldn\'t fulfill the request.')
        print('Error code: ', e.code)
else:
    # everything is fine

```

scrapy 安装

<https://www.cnblogs.com/simpleBlue3/p/10616771.html>

安装语句:

python -m pip install --upgrade pip 升级pip版本 否则会安装不成功

pip install D:\TDownloads\Twisted-18.9.0-cp37-cp37m-win_amd64.whl 安装 Twisted 后面为路径, 具体看你自己下载的地址

pip install Scrapy 安装Scrapy

成功安装Twisted

```

9.2.1) <2.8>
Installing collected packages: Twisted
Successfully installed Twisted-19.2.1
C:\Users\finlay.wang>

```

成功安装Scrapy

```

py>=2.3->pyOpenSSL->scrapy> <2.19>
Installing collected packages: cssselect, parsel, scrapy
Successfully installed cssselect-1.0.3 parsel-1.5.1 scrapy-1.6.0

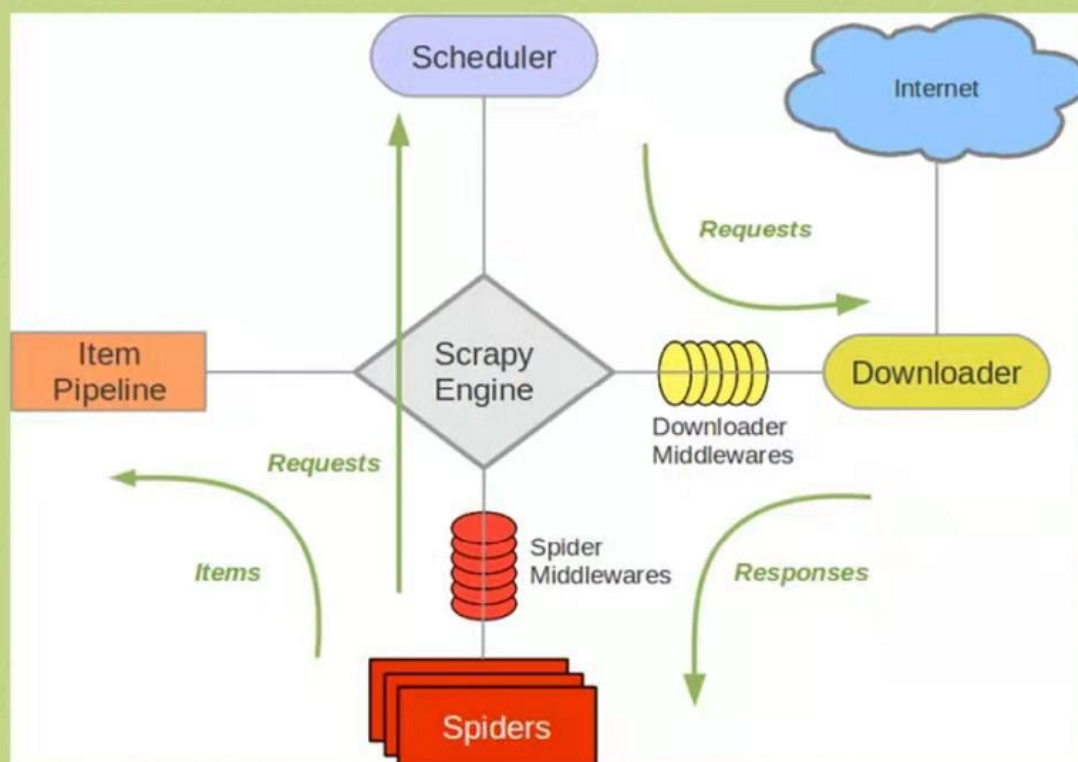
```

scrapy抓取步骤:

- 1、创建一个scrapy项目
- 2、定义Item容器
- 3、编写爬虫
- 4、存储内容

scrapy框架

Scrapy框架之初窥门径



Scrapy 控制核心，控制流动

Scheduler 调度器

Downloader 下载器

Spider 用户

ItemPipeline 存储容器

Middlewares 中间键 允许你扩展代码

scrapy 实例应用

1、创建一个新的项目：

```
.config
.idlerc
.oracle_jre_usage
.ssh
AppData
mcafee dlp quarantined
tutorial
.bash_history
.gitconfig
.viminfo
```

```
C:\Users\finlay.wang>cd ..
C:\Users>cd desktop
系统找不到指定的路径。
C:\Users>cd finlay.wang
C:\Users\finlay.wang>scrapy startproject tutorial
New Scrapy project 'tutorial', using template directory
ppdata\local\programs\python\python37-32\lib\site-packag
ect', created in:
C:\Users\finlay.wang\tutorial
You can start your first spider with:
```


2、如图所示定义好Item

定义三个参数，title、link、desc

```
# -*- coding: utf-8 -*-

# Define here the models for your scraped items
#
# See documentation in:
# https://doc.scrapy.org/en/latest/topics/items.html

import scrapy

class MeilukeItem(scrapy.Item):
    # define the fields for your item here like:
    # name = scrapy.Field()
    title = scrapy.Field()
    link = scrapy.Field()
    desc = scrapy.Field()
```

3、编写爬虫，在spider里面。新建一个Meiluke_spider.py的项目

先按照如下格式编写爬虫：

```
import scrapy

class MeilukeSpider(scrapy.Spider):
    #定义名字，区分你是什么爬虫
    name = "Meiluke"
    #定义一个范围，这样就不会跑到其他网页上面去了
    allowed_domains = ['Meiluke.com']
    #定义在哪里下载
    start_urls = [
        'http://tieba.baidu.com/p/5862595424/'
    ]

    def parse(self, response):
        filename = response.url.split('/')[-2]
        with open(filename, 'wb') as f:
            f.write(response.body)
```

4、在cmd命令行里面

cd 到之前建立的tutorial项目中，然后运行 scrapy crawl Meiluke (class里面定义的名字)

```
ModuleNotFoundError: No module named 'win32api'

C:\Users\finlay.wang\tutorial>scrapy crawl Meiluke
```

5、此处的失败时因为没有安装win32api

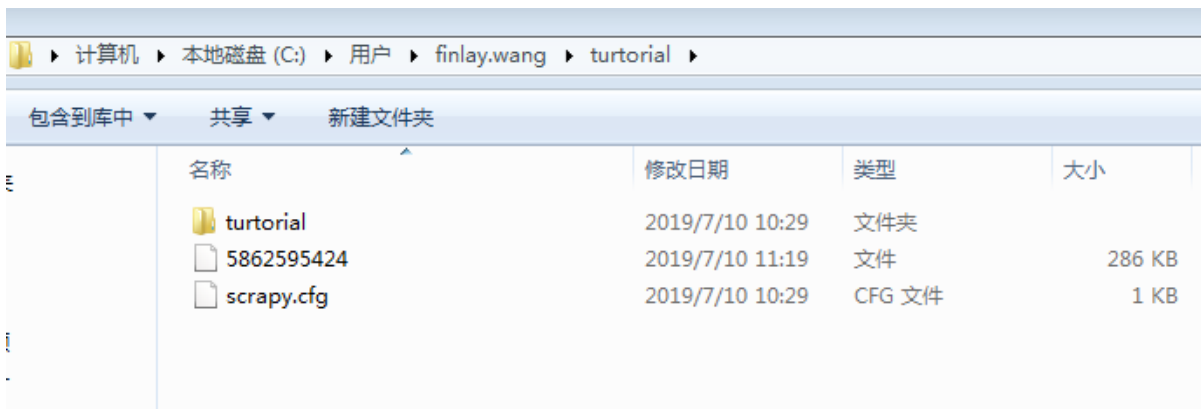
输入语句：pip install pypiwin32

成功安装win32api

```
5kB/s
Installing collected packages: pywin32, pypiwin32
Successfully installed pypiwin32-223 pywin32-224
```

6、再度测试：

成功爬取到文件



7、接下来就是将抓取的网页内容，提取出所需要的数据出来

在Scrapy中使用的是一种基于XPath和CSS的表达式机制：Scrapy Selectors。

Selector是一个选择器，它有四个基本的方法：

- xpath(): 传入xpath表达式，返回该表达式所对应的所有节点的selector list列表。
- css(): 传入CSS表达式，返回该表达式所对应的所有节点的selector list列表。
- extract(): 序列化该节点为unicode字符串并返回list。
- re(): 根据传入的正则表达式对数据进行提取，返回unicode字符串list列表。

8、启动Scrapy shell

此时出现>>>的箭头表示你已经进入shell了。

```

[s] Request <GET https://ti
[s] response <200 https://ti
[s] settings <scrapy.setting
[s] spider <DefaultSpider
[s] Useful shortcuts:
[s] fetch(url[, redirect=True]
, redirects are followed)
[s] fetch(req)
s
[s] shelp(<) Shell he
[s] view(response) View res
>>>

```

此时可以通过response.headers 查看网页的标头 或者 response.body 查看整个网页。

```

>>> response.headers
{'Content-Type': [b'text/html; charset=UTF-8'],
:29:18 GMT'], b'Server': [b'Apache'], b'Set-Cook
eaf55a1978c0a4e; expires=Thu, 31-Dec-2020 15:59:
du.com', b'wise_device=0; path=/' ], b'Tracecode'
, b'17575201430322395146071011' ], b'Vary': [b'Ac
ion': [b'1; mode=block' ]}

```

9、select 选择器 XPath 是一门在网页中查找特定信息的语言。所以用这个比正则表达式简单很多。

/html/head/title: 选择HTML文档中<head>标签内的
<title>元素

 /html/head/title/text(): 选择上面提到的<title>元素的文字

//td: 选择所有的<td>元素

//div[@class="mine"]: 选择所有具有class="mine"属性的div元素

- 通过xpath查找title 必须有//

```

ion': [b'1; mode=block' ]}
>>> response.xpath('//title')
[<Selector xpath='//title' data='<title>梅露可物语吧公会招募集中贴_梅露可物语吧_
百度贴吧</title>'>]
>>>

```

将title提取出来并字符串化

```

百度贴吧</title>'>]
>>> response.xpath('//title').extract()
['<title>梅露可物语吧公会招募集中贴_梅露可物语吧_百度贴吧</title>']
>>>

```

将title里面的文字提取出来:

```

['<title>梅露可物语吧公会招募集中贴_梅露可物语吧_百度贴吧</
>>> response.xpath('//title/text()').extract()
['梅露可物语吧公会招募集中贴_梅露可物语吧_百度贴吧']
>>>

```

10、查找出所有的图片元素

sel.xpath('//img') img 可以用extract 但无法用text ()

```
>>> sel.xpath('//img')
2019-07-10 11:47:55 [py.warnings] W
"sel" shortcut is deprecated. Use
onse.selector" instead
```

sel.xpath('//ul/li/a/text()).extract() 可以搜出百度贴吧上面的用户名。

```
result
ValueError: XPath error: Invalid expression in //img/str()
> sel.xpath('//ul/li/a/text()').extract()
看贴', '图片', '精品', '玩乐', '久叹怜曩昔', '甜甜0o布丁',
ami', '离开这里0', '米斯特儿法克', '米斯特儿法克', '希洛',
颤抖啊', '人生', '回旋木马', '0n呱呱', '玉镜如明月', '萝
liecky', '米斯特儿法克', '旺仔小酥脆', '策问', '安提戈涅',
克', '去爱丽舍', '沉香亭北倚阑杆', '沉香亭北倚阑杆', '章鱼
替课月入数千', '20年后打老师者获刑一年半', '武艺恋情', '小
物', '咸鱼上发死亡威胁', '不拴狗绳取消奖学金', '全网首个警
开班教学', '喜多川去世', '贴吧页面意见反馈', '违规贴吧举报
息处理公示',
```

1) 要注意缩进 2) shift+enter 表示换行不返回结果。

```
SyntaxError: invalid syntax
>>> sites=sel.xpath('//ul/li')
>>> for site in sites:
...     title=site.xpath('a/text()').extract()
...     print(title)
...
['看贴']
[]
['图片']
['精品']
['玩乐']
[]
[]
[]
```

11、通过【@ “字符串”】这样就可以进一步地筛选

```
ValueError: XPath error: Invalid expression in //img[@class="BDE_Image"]
>>> sel.xpath('//img[@class="BDE_Image"]').extract()
['', '<img class="BDE_I
="https://imgsa.baidu.com/forum/w%3D580/sign=fdf5980ab3096b6381195e583c
7cb8178a82b901bd239c167e8da9773812ef8f.jpg" size="67519" changedsize="t
h="560" height="314">', '', '<img class="BDE_Image"
s://imgsa.baidu.com/forum/w%3D580/sign=ba7d9652281f95caa6f592bef9167fc5
76094b3681d888c1aecc7cd98c109dd1.jpg" size="274732" changedsize="true"
0" height="315">', ' scrapy crawl Meiluke -o items.json -t json
```

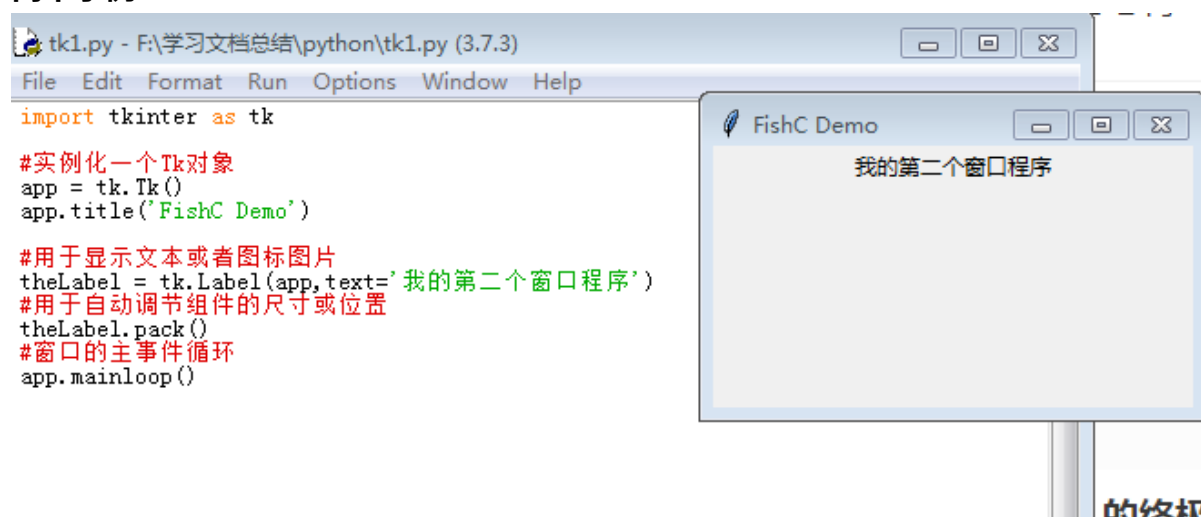
英语写作连接词笔记 Moreover 更加，用

14、爬取成功，成功创建json：

tutorial	2019/7/10 10:29	文件夹
5862595424	2019/7/10 11:19	文件
items.json	2019/7/10 14:50	JSON 文件
scrapy.cfg	2019/7/10 10:29	CFG 文件

GUI 的终极选择 Tkinter

简单示例



示例2：

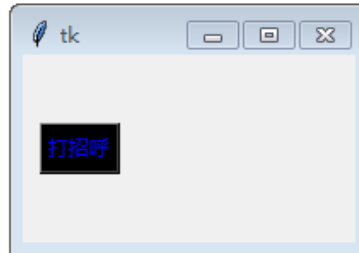
```
import tkinter as tk

class APP:
    def __init__(self, master):
        frame=tk.Frame(master)
        #设置按钮的位置，让其出现在左边，并肩X,Y间距为10
        frame.pack(side=tk.LEFT, padx=10, pady=10)
        #设置按钮的属性，frame是框架，bg是背景，fg是字体颜色，command是调用方法。
        self.hi_there = tk.Button(frame, text='打招呼', bg='black', fg='blue', command=self.say_hi)
        self.hi_there.pack()

    def say_hi(self):
        print('大家好！我是植树！')

root = tk.Tk()
app = APP(root)

root.mainloop()
```



点击过后会调用say_hi()函数



如何导入一个图片，Label的用法

Label选项，可以在file里面指定路径来表明插入图片的位置。


```
from tkinter import *

root=Tk()

textLabel = Label(root,text='您所下载的内容含有限制内容，请满18周岁再点击进去')
textLabel.pack()

photo = PhotoImage(file='刀剑神域_亚丝娜.gif')
imgLabel= Label(root,image=photo)
imgLabel.pack()

mainloop()
```



Label 也支持转义符。

```
from tkinter import *

root=Tk()

textLabel = Label(root,
    text='您所下载的内容含有限制内容，\n请满18周岁再点击进去',
    justify=LEFT,
    padx=10)
textLabel.pack()

photo = PhotoImage(file='刀剑神域_亚丝娜.gif')
imgLabel= Label(root,image=photo)
imgLabel.pack()

mainloop()
```

→ 表示文字方面左对齐



将文字插入到图片中间


```
from tkinter import *  
  
root=Tk()  
  
photo = PhotoImage(file='刀剑神域_亚丝娜.gif')  
theLabel = Label(root,  
    text='您所下载的内容含有限制内容，\n请满18周岁再点击进去',  
    justify=LEFT,  
    image=photo,  
    compound=CENTER,  
    font=('华义行楷', 20),  
    fg='white')  
  
theLabel.pack()  
  
mainloop()
```



button 组件

通过button组件实现内容的变化:

```

from tkinter import *

def callback1():
    var.set('吹吧你，我才不信哩！')

root=Tk()

frame1=Frame(root)
frame2=Frame(root)

#将原来不变的text设置成textvariable变量形势，这里设置其的参数信息
var =StringVar()
var.set('您所下载的内容含有限制内容，\n请满18周岁再点击进去')

photo = PhotoImage(file='刀剑神域_亚丝娜.gif')
theLabel = Label(frame1,
                  textvariable=var,
                  justify=LEFT,
                  image=photo,
                  compound=CENTER,
                  font=('华文行楷',20),
                  fg='white')

theLabel.pack()

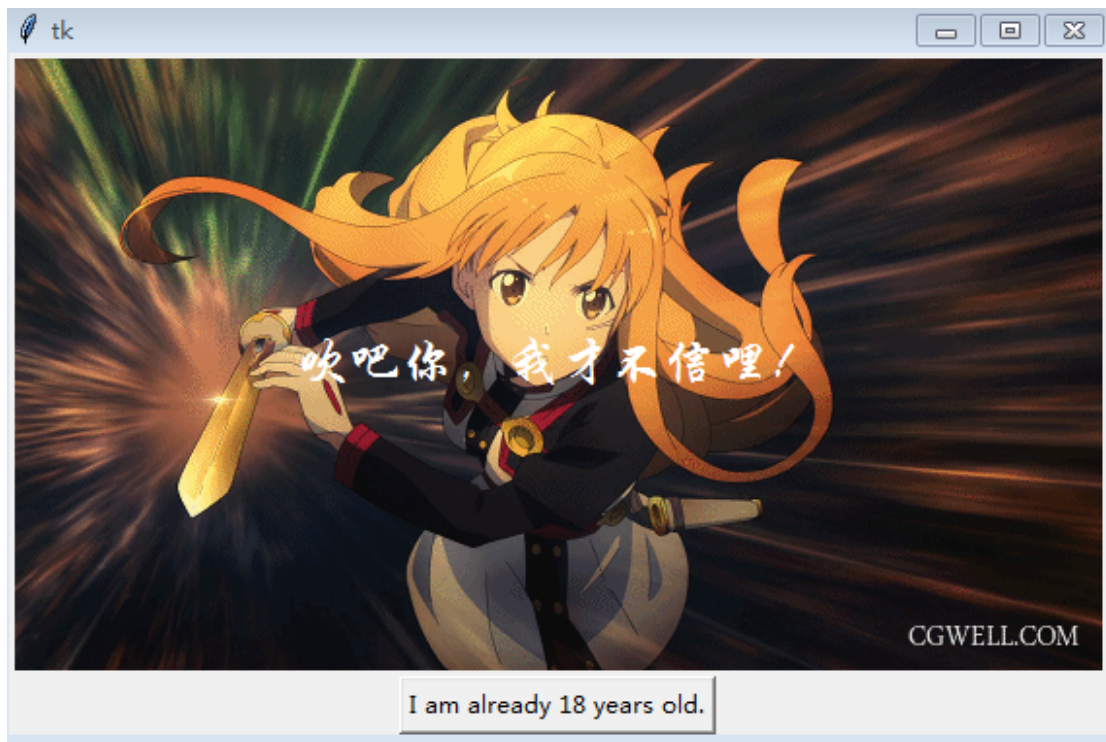
theButton = Button(frame2,text='I am already 18 years old.',command=callback1)
theButton.pack()

frame1.pack()
frame2.pack()

mainloop()

```

点击按钮有变化。主要利用继承相同root的frame框架来完成。



具体组件的介绍可以到网站上了解：

<https://fishc.com.cn/forum.php?>

[mod=forumdisplay&fid=243&filter=typeid&typeid=604](https://fishc.com.cn/forum.php?mod=forumdisplay&fid=243&filter=typeid&typeid=604)

checkboxbutton 介绍使用

此处的v赋值一个int变量 当选中时就变成1。表明选中变化就为1

```
from tkinter import *

root = Tk()

v= IntVar()

c=Checkbutton(root,text='测试一下',variable=v)
c.pack()

l=Label(root,textvariable=v)
l.pack()

mainloop()
```



选择4个选项:

```
from tkinter import *

root = Tk()

Girls= ['西施','貂蝉','王昭君','杨玉环']

v=[]

for girl in Girls:
    v.append(IntVar())
    #v[-1]表示每次都要拿到它最后一个元素
    b=Checkbutton(root,text=girl,variable=v[-1])
    #向西对齐，左对齐
    b.pack(anchor=W)
l=Label(root,textvariable=v[-1])
l.pack()

mainloop()
```

radiobutton的使用方法:

```
from tkinter import *

root = Tk()

v=IntVar()

#根据value的值进行单选，点中two，则value=2，不等于1，3其他退出不选。
Radiobutton(root,text='one',variable=v,value=1).pack(anchor=W)
Radiobutton(root,text='two',variable=v,value=2).pack(anchor=W)
Radiobutton(root,text='three',variable=v,value=3).pack(anchor=W)

mainloop()
```

多重单选可以使用列表包含元素的形式实现

将单选嵌套到group（框架）里面去

```

from tkinter import *

root=Tk()

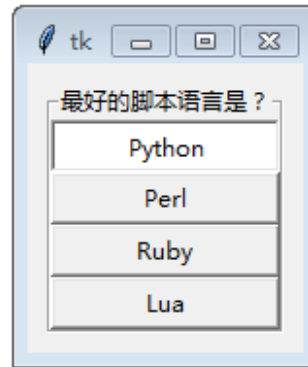
group=LabelFrame(root,text='最好的脚本语言是?')
group.pack(padx=10,pady=10)

LANGS=[
    ('Python',1),
    ('Perl',2),
    ('Ruby',3),
    ('Lua',4)]

v=IntVar()
#给其设置一个初始值1，即光标选中Python的情况
v.set(1)

for lang,num in LANGS:
    #indicatoron 指示器(小圆点),fill=X表示横向填充
    Radiobutton(group,text=lang,variable=v,value=num,indicatoron=False).pack(fill=
mainloop()

```



Python中的for语句可以设置2个变量，尤其在列表中是元组表示时表示一前一后。

```

type help, copyright, credits or license() for more information.
>>> list1=[('王建军','wangjianjun'),('植树','zhishupoi'),('白雨鑫','xiaobai')]
>>> for l1,l2 in list1:
    print(l1,end=" ")
    print(l2,end=" ")

王建军 wangjianjun 植树 zhishupoi 白雨鑫 xiaobai
>>> for l1,l2 in list1:
    print(l1,end=" ")

王建军 植树 白雨鑫
>>> for l1,l2 in list1:
    print(l2,end=" ")

wangjianjun zhishupoi xiaobai
>>>

```

entry组件

简单介绍

```

from tkinter import *

root=Tk()

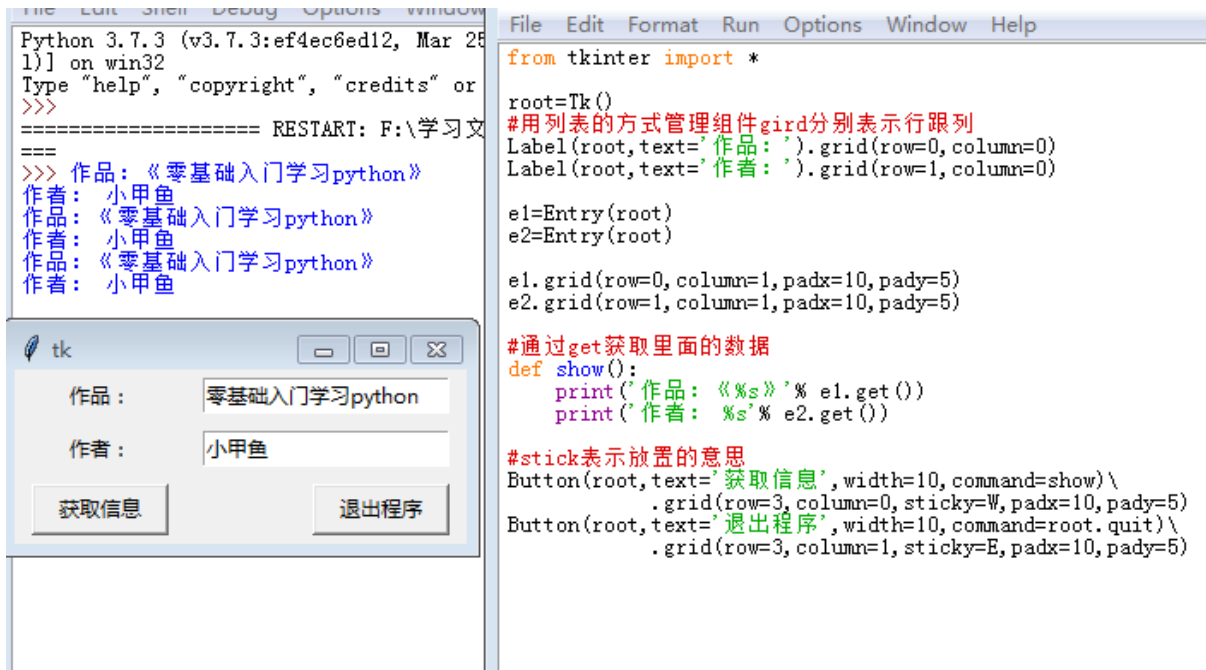
e=Entry(root)
e.pack(padx=20,pady=20)

#清空代码
e.delete(0,END)
#插入文本数字代码
e.insert(0,"默认文本...")

mainloop()

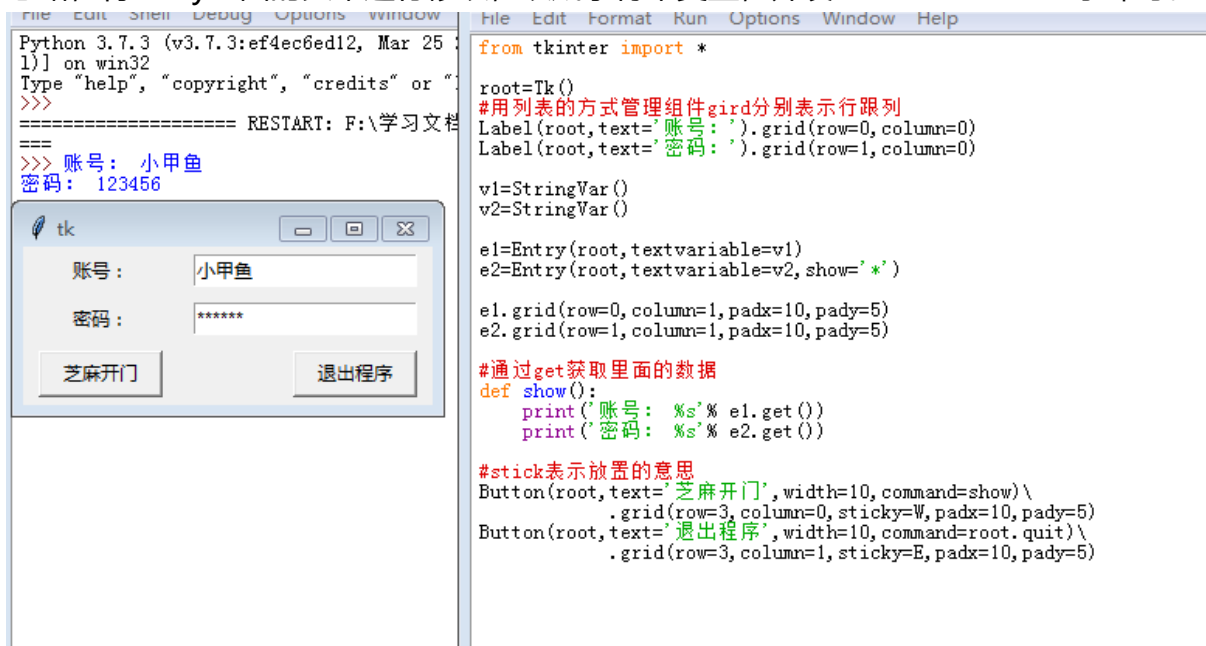
```

用列表的方式管理组件，非frame用法：



如何输入密码:

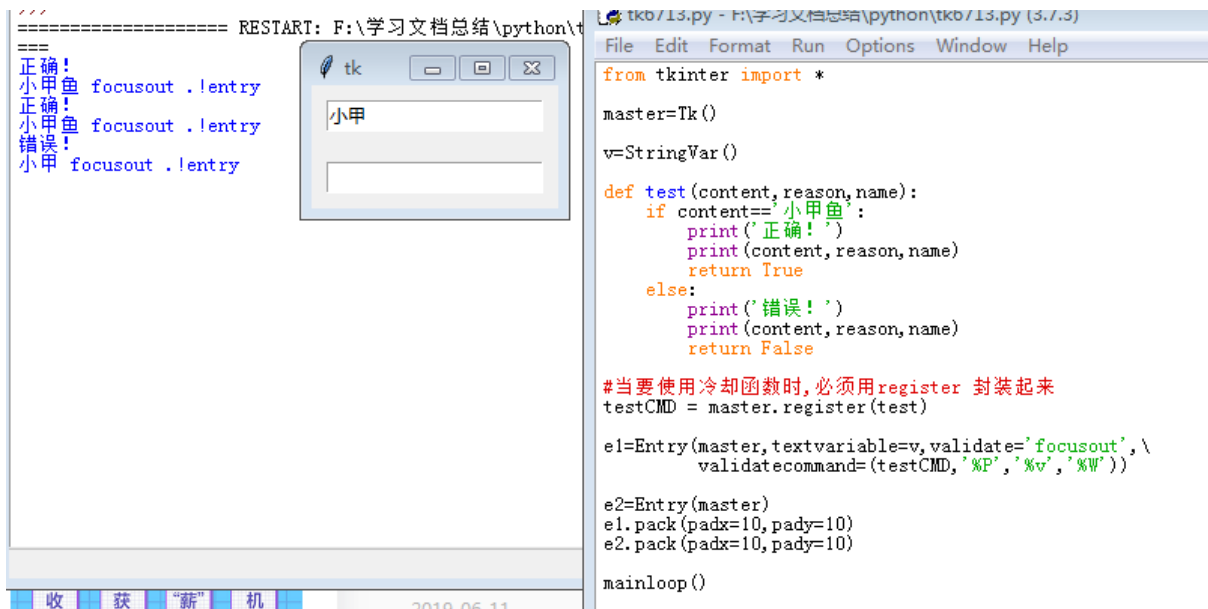
思路, 将Entry里面的文本进行修改, 改成字符串变量, 并设置show= '*' 显示即可。



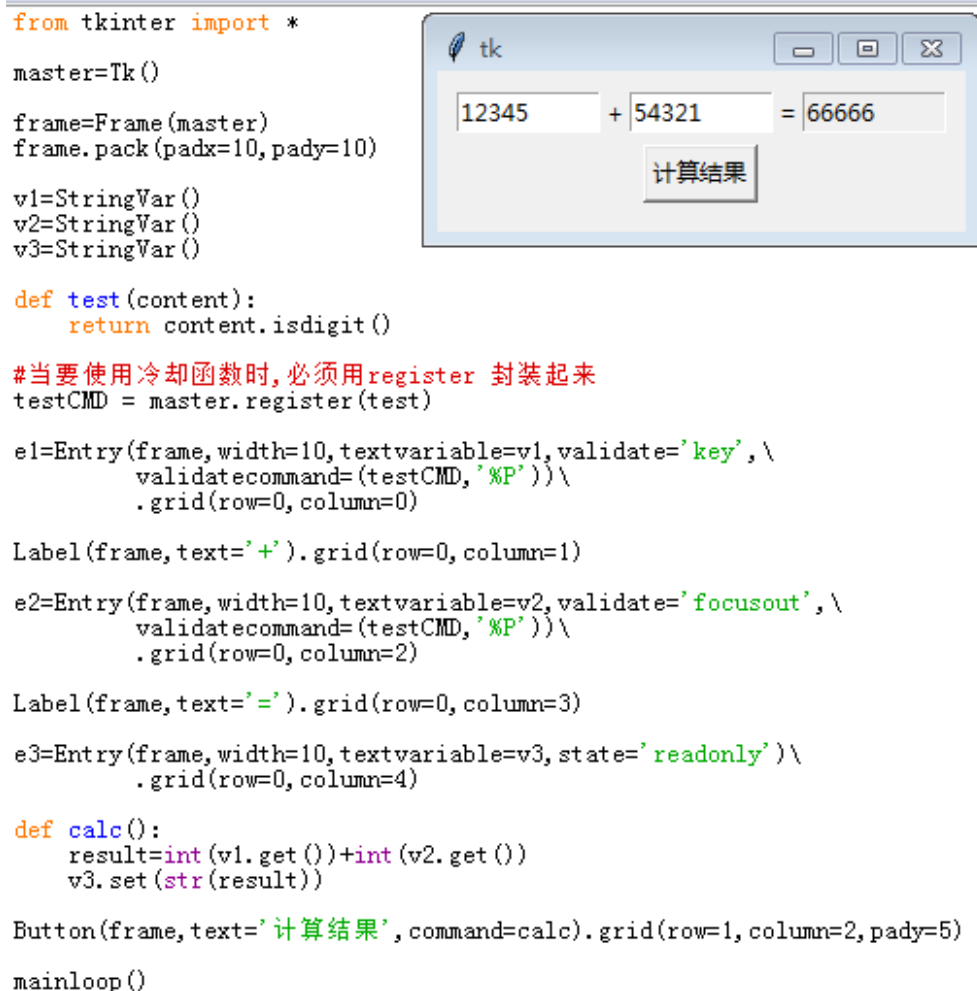
如何设置一个计算器

计算器的话就只能输入一个数字, 不能输入其他字符。可以通过entry自行验证。

1、练习, %P表示content内容, %v当前组件validate选项的值, %W表示该组件的名称



2、如果是数字，自动返回True，如果不是则返回False；register必须在master后面。



验证的函数应用：

首先启用验证的“开关”是 validate 选项，该选项可以设置的值有：失去焦点表示换行之类的操作

值	含义
'focus'	当 Entry 组件获得或失去焦点的时候验证

'focusin'	当 Entry 组件获得焦点的时候验证
'focusout'	当 Entry 组件失去焦点的时候验证
'key'	当输入框被编辑的时候验证
'all'	当出现上边任何一种情况的时候验证
'none'	1. 关闭验证功能 2. 默认设置该选项（即不启用验证） 3. 注意，是字符串的 'none'，而非 None

validatecommand 选项指定一个验证函数，该函数只能返回 True 或 False 表示验证的结果。然后，invalidcommand 选项指定的函数只有在 validatecommand 的返回值为 False 的时候才被调用。

冷却触发详解：

Tkinter 为验证函数提供一些额外的选项：

额外选项	含义
'%d'	操作代码：0 表示删除操作；1 表示插入操作；2 表示获得、失去焦点或 textvariable 变量的值被修改
'%i'	1. 当用户尝试插入或删除操作的时候，该选项表示插入或删除的位置（索引号） 2. 如果是由于获得、失去焦点或 textvariable 变量的值被修改而调用验证函数，那么该值是 -1
'%P'	1. 当输入框的值允许改变的时候，该值有效 2. 该值为输入框的最新文本内容
'%S'	该值为调用验证函数前输入框的文本内容
'%S'	1. 当插入或删除操作触发验证函数的时候，该值有效 2. 该选项表示文本被插入和删除的内容
'%v'	该组件当前的 validate 选项的值
'%V'	1. 调用验证函数的原因 2. 该值是 'focusin', 'focusout', 'key' 或 'forced'（textvariable 选项指定的变量值被修改）中的一个
'%W'	该组件的名字

listbox 提供大量的选项框


```

from tkinter import *

master=Tk()

theLB=Listbox(master)
theLB.pack()

for item in ['鸡蛋','鸭蛋','鸟蛋','李狗蛋']:
    #END相当于一个变量存放最后一个位置
    theLB.insert(END,item)

theButton = Button(master,text='删除它',\
                    #利用lambda函数，给x一个初始值theLB，ACTIVE和END一样
                    #属于内置的一个函数，表示激活的选项
                    command=lambda x=theLB:x.delete(ACTIVE))

theButton.pack()

mainloop()

```

Listbox 组件根据 selectmode 选项提供了四种不同的选择模式：**SINGLE**（单选）、**BROWSE**（也是单选，按住左键拖动鼠标或通过方向键可以直接改变选项）、**MULTIPLE**（多选）和 **EXTENDED**（也是多选，但需要同时按住 Shift 键或 Ctrl 键或拖拽鼠标实现）。默认是 BROWSE。

```

theLB=Listbox(master,selectmode=SINGLE)
theLB.pack()

```

滚动条组件

- 1、首先在实例化Tk对象内部建立一个Scrollbar(滚动条)
- 2、在theLB (Listbox) 内部建立一个竖直的滚动条回调函数yscrollcommand
- 3、设置滚动条的config即可。

当 Listbox 组件的可视范围发生改变的时候，Listbox 组件通过调用 set() 方法通知 Scrollbar 组件。而当用户操纵滚动条的时候，将自动调用 Listbox 组件的 yview() 方法。

```

from tkinter import *

master=Tk()

sb=Scrollbar(master)
sb.pack(side=RIGHT,fill=Y)

#当滚动条滚动后，就会修改里面的set方法，并带动更新显示出来
theLB=Listbox(master,selectmode=EXTENDED,height=10,yscrollcommand=sb.set)

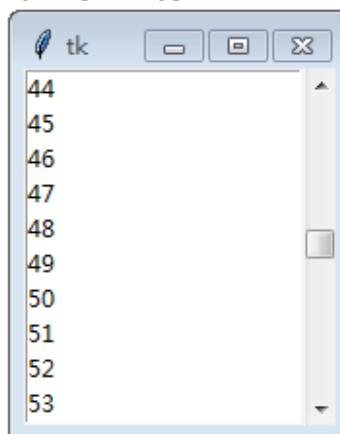
for item in range(100):
    #END相当于一个变量存放最后一个位置
    theLB.insert(END,item)

theLB.pack(side=LEFT,fill=BOTH)

#yview显示如何垂直滚动的操作
sb.config(command=theLB.yview)

mainloop()

```



滑块组件设置

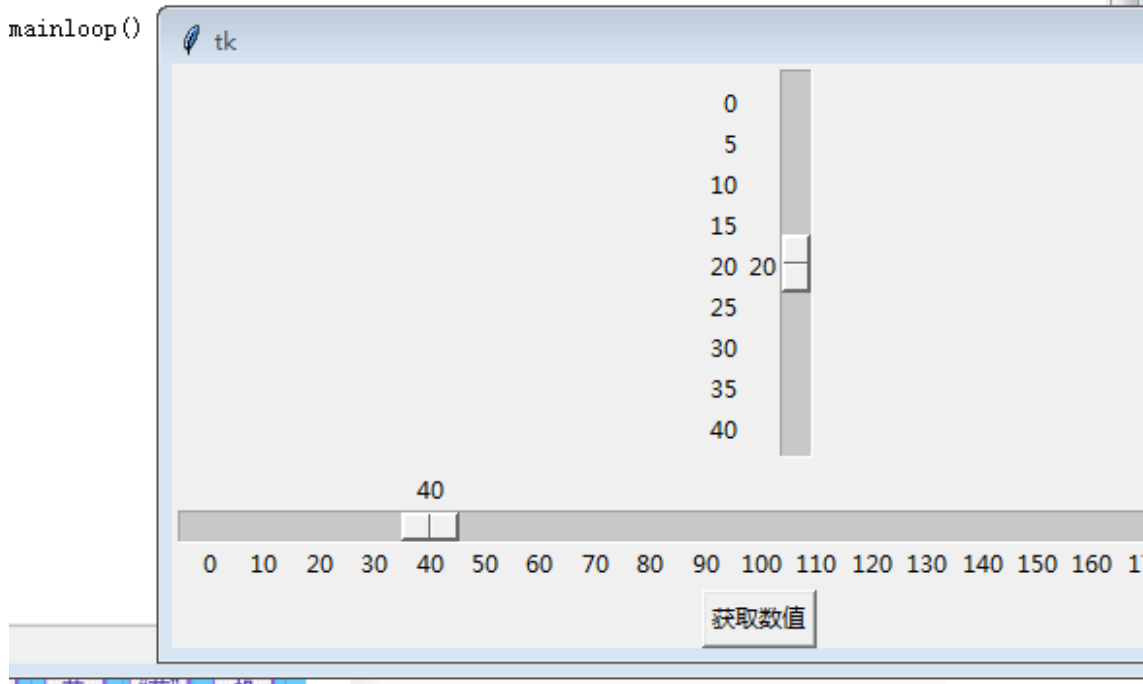
```
from tkinter import *

master=Tk()
#tickinterval表示刻度间隔，resolution表示移动的间隔（精度）
#length表示长度 这边scale是滑块
s1=Scale(from_=0,to=42,tickinterval=5,resolution=5,length=200)
s1.pack()
s2=Scale(from_=0,to=200,orient=HORIZONTAL,tickinterval=10,\
resolution=5,length=600)
s2.pack()

def show():
    print(s1.get(),s2.get())

Button(master,text='获取数值',command=show).pack()

mainloop()
```



text 组件 (IDLE就是一个text组件)

1、text可以插入button 同时也可以插入图片

```

from tkinter import *

root = Tk()

text1 = Text(root,width=80,height=30)
text1.pack()

text1.insert(INSERT,'I love \n')
text1.insert(END,"FishC.com!")

photo=PhotoImage(file='刀剑神域_亚丝娜.gif')

def show1():
    print('yo!我居然被你点了……')
def show2():
    text1.image_create(END,image=photo)

b1=Button(text1,text='点我点我',command=show1)
b2=Button(text1,text='点我插图',command=show2)
#在text中插入按钮组件，这样b1就不用pack了
text1.window_create(INSERT>window=b1)
text1.window_create(INSERT>window=b2)

mainloop()

```



text组件中的索引 (Mark, Tags) :

mark只有通过mark_unset解除mark封印 (具体用到再学吧)

mark_gravity() mark 偏向哪边? 一般默认为右边。

[https://fishc.com.cn/forum.php?](https://fishc.com.cn/forum.php?mod=viewthread&tid=59444&extra=page%3D1%26filter%3Dtypeid%26typeid%3D604)

[mod=viewthread&tid=59444&extra=page%3D1%26filter%3Dtypeid%26typeid%3D604](https://fishc.com.cn/forum.php?mod=viewthread&tid=59444&extra=page%3D1%26filter%3Dtypeid%26typeid%3D604)

```

from tkinter import *

root = Tk()

text1 = Text(root,width=30,height=10)
text1.pack()

text1.insert(INSERT,'I love fishC.com!')

#添加tag, tag名字, 1.7表示第一行的第8列 (从0开始) 到第一行的第13列
#然后一个单独的第一行的第15列 双数表示范围, 最后的单数代表单个元素
text1.tag_add("tag1","1.7","1.12","1.14")
text1.tag_config("tag1",background='yellow',foreground='red')

mainloop()

```



实战演练 tag

```

from tkinter import *
import webbrowser

root = Tk()

text1 = Text(root,width=30,height=10)
text1.pack()

text1.insert(INSERT,'I love fishC.com!')

#添加tag，tag名字，1.7表示第一行的第8列（从0开始）到第一行的第13列
#然后一个单独的第一行的第15列 双数表示范围，最后的单数代表单个元素
text1.tag_add('tag1','1.7','1.16')
text1.tag_config('tag1',foreground='blue',underline=True)

def show_arrow_cursor(event):
    #鼠标变成箭头
    text1.config(cursor='arrow')
def show_xterm_cursor(event):
    #离开后鼠标变成文本编辑xterm模式
    text1.config(cursor='xterm')
def click(event):
    webbrowser.open('http://www.fishc.com')

#事件绑定
text1.tag_bind('tag1','<Enter>',show_arrow_cursor)
text1.tag_bind('tag1','<Leave>',show_xterm_cursor)
text1.tag_bind('tag1','<Button-1>',click)

mainloop()

```



如何检查内容是否有过改动

```

from tkinter import *
import hashlib

root = Tk()

text1 = Text(root,width=30,height=10)
text1.pack()

text1.insert(INSERT,'I love fishC.com!')

#内容为第1行第0列到结束。
contents=text1.get('1.0',END)

def getSig(contents):
    m = hashlib.md5(contents.encode())
    return m.digest()

#将内容先保存到sig中
sig=getSig(contents)

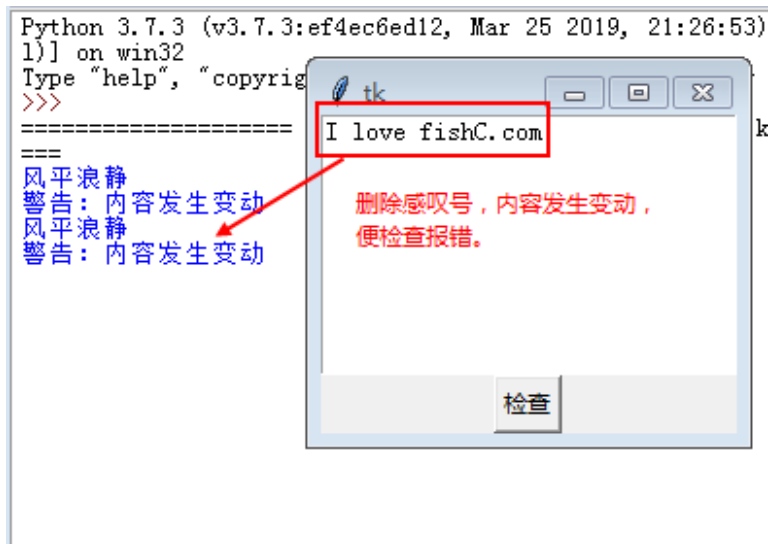
def check():
    #重新获取当前页面的content内容。
    contents=text1.get('1.0',END)
    if sig !=getSig(contents):
        print('警告：内容发生变动')
    else:
        print('风平浪静')

Button(root,text='检查',command=check).pack()

mainloop()

```

效果展示：



文本搜索

1) text.search返回的值是一个line.column形式, 是一个浮点数

```
from tkinter import *
import hashlib

root = Tk()

text1 = Text(root,width=30,height=10)
text1.pack()

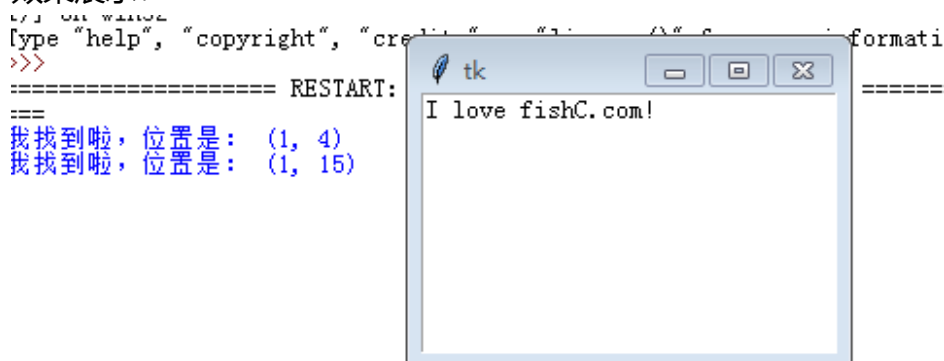
text1.insert(INSERT,'I love fishC.com!')

def getIndex(index):
    #text.index(index)将参数指定的位置(1.3)以line.column形式索引
    #返回基于给定索引的索引的绝对值。
    #str.split就是把1.3拆分成['1','3'],而map(int,['1','3'])就是把字符串
    #['1','3']变成数字[1,3]最后在tuple一下就变成元组了。
    #return tuple(map(int,str.split(text.index(index),'.')))
    temp=list(map(int,str.split(index,'.')))
    temp[1]+=1
    return tuple(temp)

start='1.0'
while True:
    #search的返回值是以'line.column'的索引形式返回,此时返回的pos=1.3
    pos=text1.search('o',start,stopindex=END)
    if not pos:
        break
    print('我找到啦,位置是:',getIndex(pos))
    #将start指向下一个字符
    start= pos+ '1c'

mainloop()
```

效果展示:



text组件还有恢复, 撤销的操作undo:

```
from tkinter import *
import hashlib

root = Tk()

text1 = Text(root, width=30, height=10, undo=True)
text1.pack()

text1.insert(INSERT, 'I love fishC.com!')

def show():
    text1.edit_undo()

Button(root, text='撤销', command=show).pack()

mainloop()
```

开启undo

```
from tkinter import *
import hashlib

root = Tk()

text1 = Text(root, width=30, height=10, undo=True, autoseparators=False)
text1.pack()

text1.insert(INSERT, 'I love fishC.com!')

def callback(event):
    text1.edit_separator()

text1.bind('<Key>', callback)

def show():
    text1.edit_undo()

Button(root, text='撤销', command=show).pack()

mainloop()
```

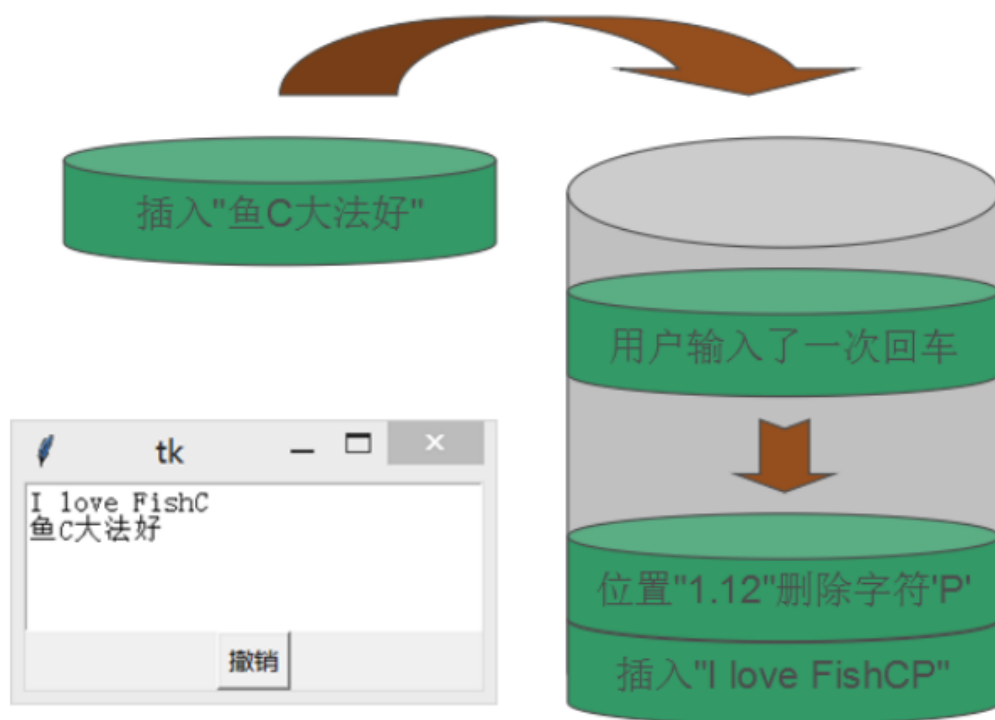
打开撤销功能

将自动分割关闭

修改一次分割符

每次输入，调用一次callback

原理示意：一般认为回车是一个完整的分隔符。



用Tkinter创建一个画布：

[https://fishc.com.cn/forum.php?](https://fishc.com.cn/forum.php?mod=viewthread&tid=60116&extra=page%3D1%26filter%3Dtypeid%26typeid%3D604)

[mod=viewthread&tid=60116&extra=page%3D1%26filter%3Dtypeid%26typeid%3D604](https://fishc.com.cn/forum.php?mod=viewthread&tid=60116&extra=page%3D1%26filter%3Dtypeid%26typeid%3D604)

```
from tkinter import *

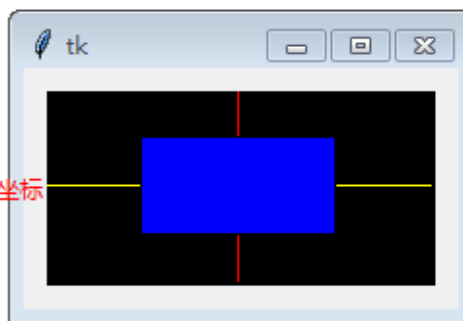
root = Tk()

w=Canvas(root,width=200,height=100,background='black')
w.pack(padx=10,pady=10)

w.create_line(0,50,200,50,fill='yellow')
w.create_line(100,0,100,100,fill='red')
w.create_rectangle(50,25,150,75,fill='blue')

mainloop()
```

直线时前两个参数表示开始点的坐标
矩阵的是对角线的坐标



黄线被覆盖了

移动里面的物体，同时创建一个文本对象

```

from tkinter import *

root = Tk()

w=Canvas(root,width=200,height=100,background='black')
w.pack(padx=10,pady=10)

#创建物体
line1=w.create_line(0,50,200,50,fill='yellow')
line2=w.create_line(100,0,100,100,fill='red')
rect1=w.create_rectangle(50,25,150,75,fill='blue')

#对里面的物体进行修改的方法
w.coords(line1,0,25,200,25)
w.itemconfig(rect1,fill='green')
#w.move()
w.delete(line2)

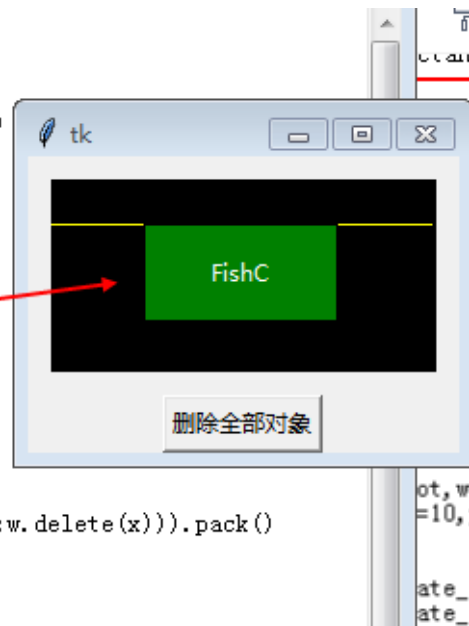
w.create_text(100,50,text='FishC',fill='white')

Button(root,text='删除全部对象',command=(lambda x=ALL:w.delete(x))).pack()

mainloop()

```

填充一个文字对象



画椭圆

tkinter是根据契合的矩形去画内部的椭圆，圆形是椭圆的一个特例

```

from tkinter import *

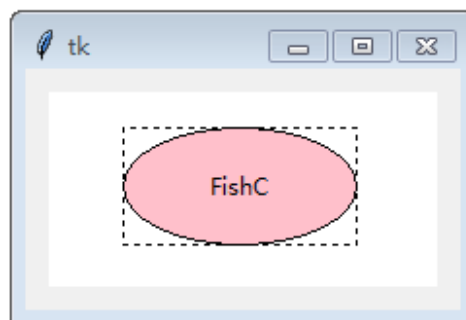
root = Tk()

w=Canvas(root,width=200,height=100,background='white')
w.pack(padx=10,pady=10)

#虚线的矩形
w.create_rectangle(40,20,160,80,dash=(4,4))
#这里oval的位置参数信息和矩形的一模一样
w.create_oval(40,20,160,80,fill='pink')
w.create_text(100,50,text='FishC')

mainloop()

```



画五角星（多边形）

```

from tkinter import *
import math as m

root = Tk()

w=Canvas(root,width=200,height=100,background='white')
w.pack(padx=10,pady=10)

center_x=100
center_y=50
r=50

```

```

points=[
    #左上点
    center_x-int(r*m.sin(2*m.pi/5)),
    center_y-int(r*m.cos(2*m.pi/5)),
    #右上点
    center_x+int(r*m.sin(2*m.pi/5)),
    center_y-int(r*m.cos(2*m.pi/5)),
    #左下点
    center_x-int(r*m.sin(m.pi/5)),
    center_y+int(r*m.cos(m.pi/5)),
    #顶点
    center_x,
    center_y-r,
    #右下点
    center_x+int(r*m.sin(m.pi/5)),
    center_y+int(r*m.cos(m.pi/5))
]

```

```

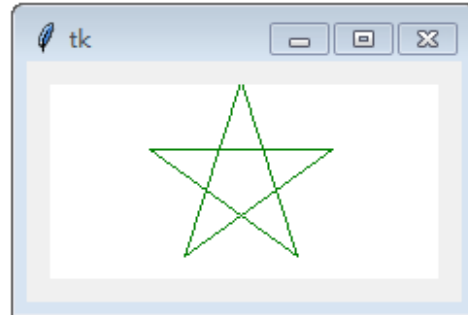
w.create_polygon(points, outline='green', fill='')

```

```

mainloop()

```



利用polygon画点，填充为空

利用canvas设置画板


```

from tkinter import *
import math as m

root = Tk()

w=Canvas(root,width=400,height=200,background='white')
w.pack(padx=10,pady=10)

def paint(event):
    x1,y1 = (event.x-1), (event.y-1)
    x2,y2 = (event.x+1), (event.y+1)
    #通过创建一个最小的椭圆表示点
    w.create_oval(x1,y1,x2,y2,fill='red')

w.bind('<B1-Motion>',paint)
Label(root,text=' 按住鼠标左键并移动，开始绘图吧....').pack(side=BOTTOM)

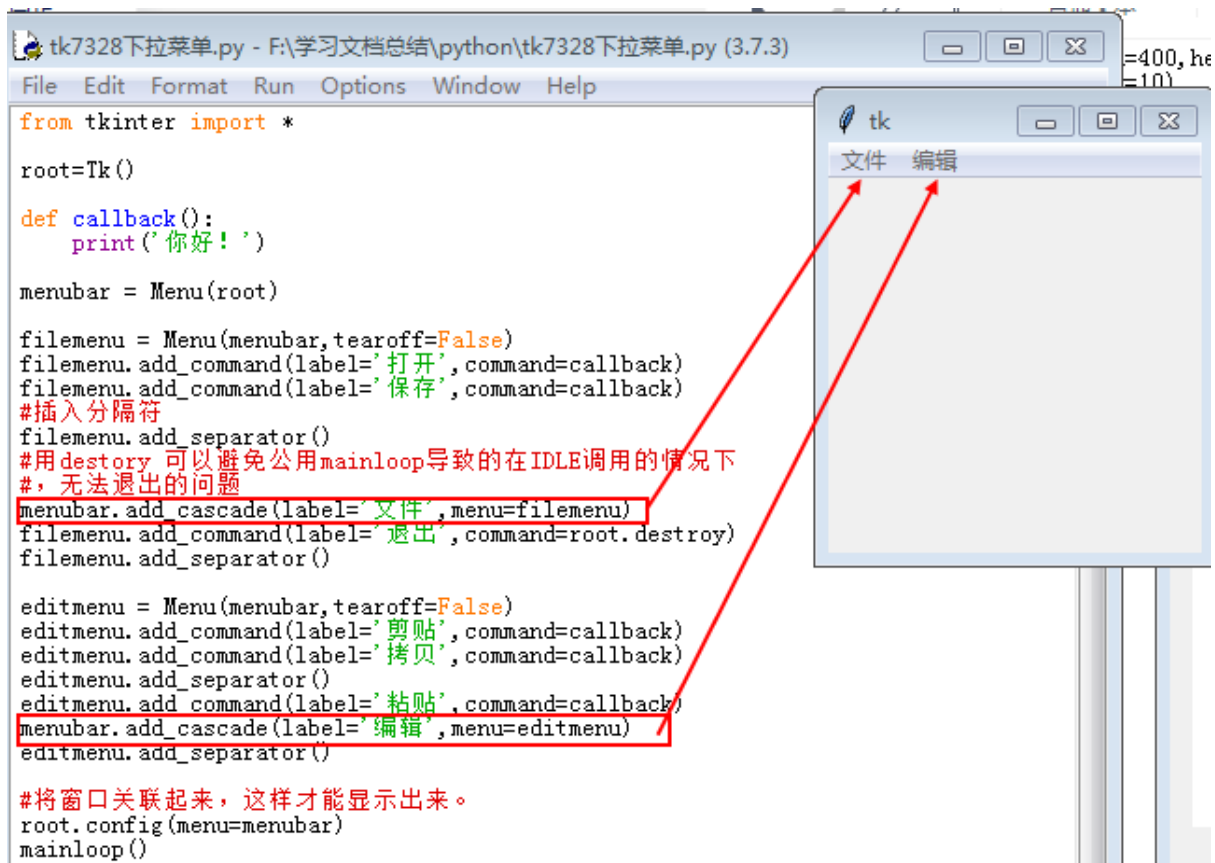
Button(root,text='重新开始绘画',command=(lambda x=ALL:w.delete(x))).pack(side=BO

mainloop()

```



画下拉菜单



tearoff (撕开) 如果不设置False会多一个虚线, 可以将菜单单独撕出来。

右键弹出菜单

frame设置了点击区域,

```

from tkinter import *

root=Tk()

def callback():
    print('你好!')

menubar = Menu(root)

filemenu=Menu(menubar)
filemenu.add_command(label='打开',command=callback)
filemenu.add_command(label='保存',command=callback)
#插入分隔符
filemenu.add_separator()
#用destroy 可以避免公用mainloop导致的在IDLE调用的情况下
#，无法退出的问题
menubar.add_cascade(label='文件',menu=filemenu)
filemenu.add_command(label='退出',command=root.destroy)
filemenu.add_separator()

editmenu = Menu(menubar,tearoff=False)
editmenu.add_command(label='剪贴',command=callback)
editmenu.add_command(label='拷贝',command=callback)
editmenu.add_separator()
editmenu.add_command(label='粘贴',command=callback)
menubar.add_cascade(label='编辑',menu=editmenu)
editmenu.add_separator()

frame = Frame(root,width=512,height=512)
frame.pack()

def popup(event):
    menubar.post(event.x_root,event.y_root)

frame.bind('<Button-3',popup)

#将窗口关联起来，这样才能显示出来。
#root.config(menu=menubar)
mainloop()

```

在下拉菜单中使用checkbutton (多选) 和radiobutton (单选)

```

root=Tk()

def callback():
    print('你好!')

menubar = Menu(root)

openVar=IntVar()
saveVar=IntVar()
quitVar=IntVar()

filemenu=Menu(menubar,tearoff=False)
filemenu.add_checkbutton(label='打开',command=callback,variable=openVar)
filemenu.add_checkbutton(label='保存',command=callback,variable=saveVar)
#插入分隔符
filemenu.add_separator()
#用destroy 可以避免公用mainloop导致的在IDLE调用的情况下
#，无法退出的问题
filemenu.add_checkbutton(label='退出',command=root.destroy,\
                        variable=quitVar)
filemenu.add_separator()
menubar.add_cascade(label='文件',menu=filemenu)

editVar=IntVar()

editmenu = Menu(menubar,tearoff=False)
editmenu.add_radiobutton(label='剪贴',command=callback,\
                        variable=editVar,value=1)
editmenu.add_radiobutton(label='拷贝',command=callback,\
                        variable=editVar,value=2)
editmenu.add_separator()
editmenu.add_radiobutton(label='粘贴',command=callback,\
                        variable=editVar,value=3)
menubar.add_cascade(label='编辑',menu=editmenu)
editmenu.add_separator()

#将窗口关联起来，这样才能显示出来。
root.config(menu=menubar)
mainloop()

```

设置下拉按钮

```

from tkinter import *

root=Tk()

def callback():
    print('你好!')

mb=Menubutton(root,text='点我',relief=RAISED)
mb.pack()

openVar=IntVar()
saveVar=IntVar()
quitVar=IntVar()

filemenu=Menu(mb,tearoff=False)
filemenu.add_checkbutton(label='打开',command=callback,variable=openVar)
filemenu.add_checkbutton(label='保存',command=callback,variable=saveVar)
#插入分隔符
filemenu.add_separator()
#用destroy 可以避免公用mainloop导致的在IDLE调用的情况下
#，无法退出的问题
filemenu.add_checkbutton(label='退出',command=root.destroy,\
                        variable=quitVar)
filemenu.add_separator()

#将窗口关联起来，这样才能显示出来。
mb.config(menu=filemenu)

mainloop()

```

选择菜单（涉及到*的用法） 形参+*表示打包，实参+*表示解包

```

from tkinter import *

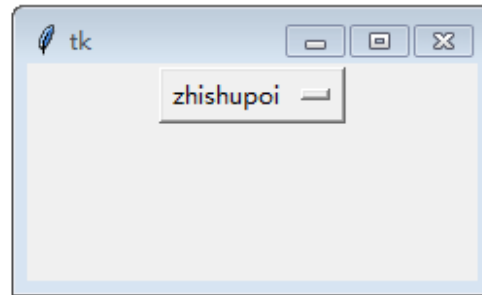
OPTIONS=[
    'zhishupoi',
    'wangjianjun',
    'FF',
    'xuejunru',
    'panjunjun'
]
root=Tk()

variable=StringVar()
variable.set(OPTIONS[0])

#这里的*表示解包的作用，如果加在形参上，则是打包的作用
##表示打包成列表，**表示打包成字典
w = OptionMenu(root,variable,*OPTIONS)
w.pack()

mainloop()

```



事件绑定

[https://fishc.com.cn/forum.php?](https://fishc.com.cn/forum.php?mod=viewthread&tid=61373&extra=page%3D1%26filter%3Dtypeid%26typeid%3D604)

[mod=viewthread&tid=61373&extra=page%3D1%26filter%3Dtypeid%26typeid%3D604](https://fishc.com.cn/forum.php?mod=viewthread&tid=61373&extra=page%3D1%26filter%3Dtypeid%26typeid%3D604)

```

from tkinter import *

root=Tk()

def callback(event):
    print(event.char)

frame=Frame(root,width=200,height=200)
frame.bind('<Key>',callback)
#通过设置focus设置焦点
frame.focus_set()

frame.pack()

mainloop()

```

通过事件绑定捕捉移动

```

from tkinter import *

root=Tk()

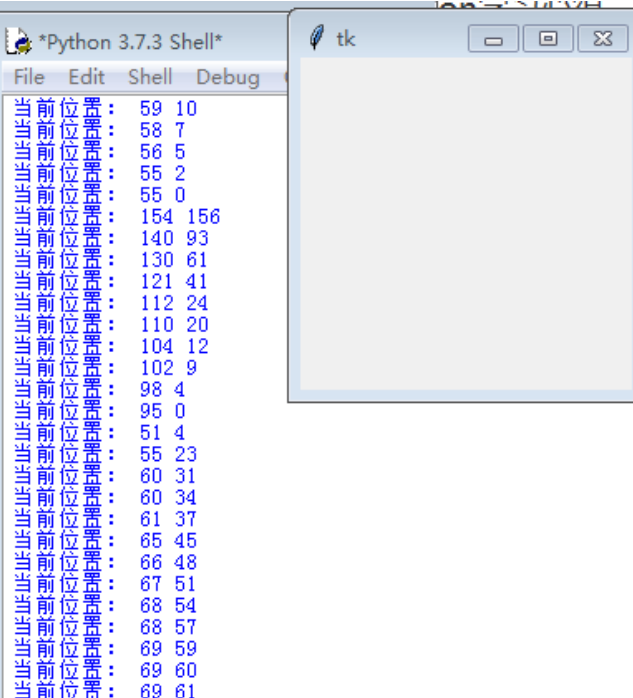
def callback(event):
    print('当前位置: ',event.x,event.y)

frame=Frame(root,width=200,height=200)
frame.bind('<Motion>',callback)
#通过设置focus设置焦点
#frame.focus_set()

frame.pack()

mainloop()

```



Message

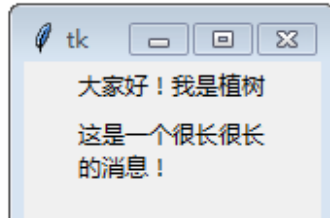
```
from tkinter import *

root=Tk()

w1 = Message(root,text='大家好！我是植树',width=100)
w2 = Message(root,text='这是一个很长很长的消息！',width=100)

w1.pack()
w2.pack()

mainloop()
```



Spinbox 从一些固定的值中选一个

```
from tkinter import *

root=Tk()

w=Spinbox(root,values=('小甲鱼','凤姐','张宇'))
w.pack()

mainloop()
```



展现窗格系统()

expand=True指，当我拉伸窗口时，里面的同步填充

```
from tkinter import *

#root=Tk()

#默认是左右窗格
#showhandle显示的是小方格，sashrelief是分割线，向下凹进去
m1 = PanedWindow(showhandle=True,sashrelief=SUNKEN)
#BOTH是垂直水平填充，
#expand指定是否填充父组件的额外空间
#默认是False
m1.pack(fill=BOTH,expand=1)

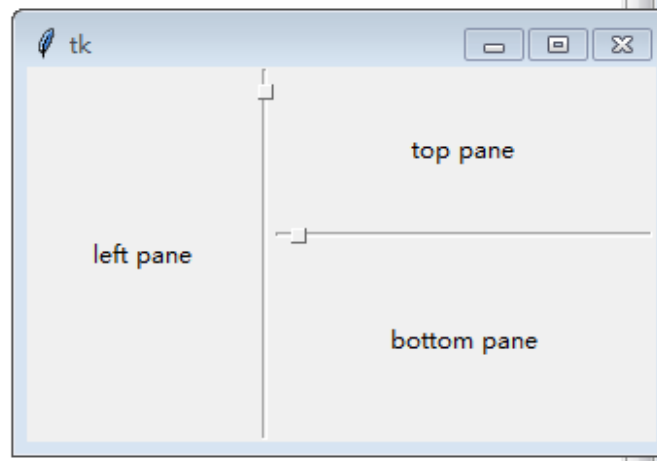
left = Label(m1,text='left pane')
m1.add(left)

#建立一个垂直方向的窗口
m2 = PanedWindow(orient=VERTICAL,showhandle=True,sashrelief=SUNKEN)
#将m2直接加到m1中
m1.add(m2)

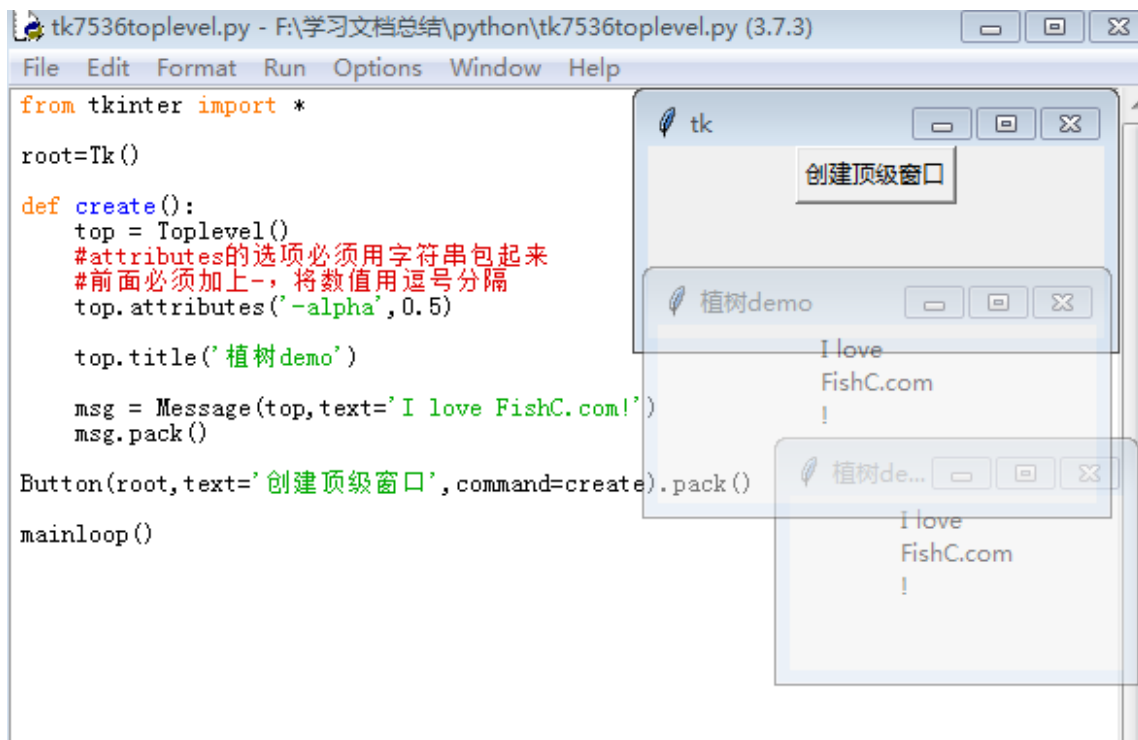
top=Label(m2,text='top pane')
m2.add(top)

bottom=Label(m2,text='bottom pane')
m2.add(bottom)

mainloop()
```



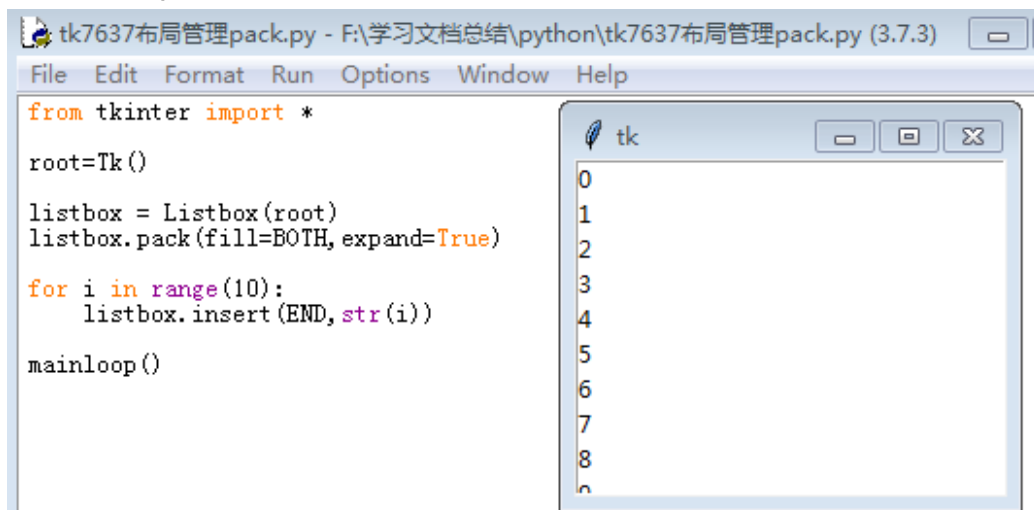
Toplevel



Tkinter 布局管理器

pack、grid 和 place 均用于管理同在一个父组件下的所有组件的布局，(这三种组件最好不要混用)其中：

- pack 是按添加顺序排列组件
- grid 是按行/列形式排列组件
- place 则允许程序员指定组件的大小和位置



pack是顺序填充的

```
from tkinter import *
```

```
root=Tk()
```

```
Label(root,text='red',bg='red',fg='white').pack(fill=X)
```

```
Label(root,text='green',bg='green',fg='black').pack(fill=X)
```

```
Label(root,text='blue',bg='blue',fg='white').pack(fill=X)
```

```
mainloop()
```



grid管理器首选（特别灵活）

tk7638grid排列布局.py - F:\学习文档总结\python\tk7638grid排列布局.py (3.7.3)

```
File Edit Format Run Options Window Help
```

```
from tkinter import *
```

```
root=Tk()
```

```
photo = PhotoImage(file='刀剑神域_亚丝娜.gif')
```

```
#rowspan表示跨10行
```

```
Label(root,image=photo).grid(row=2,column=2,rowspan=10,\  
                             columnspan=10,padx=5,pady=5)
```

```
#sticky指对齐方式，W表示左（西）对齐
```

```
Label(root,text='用户名').grid(row=0,sticky=W)
```

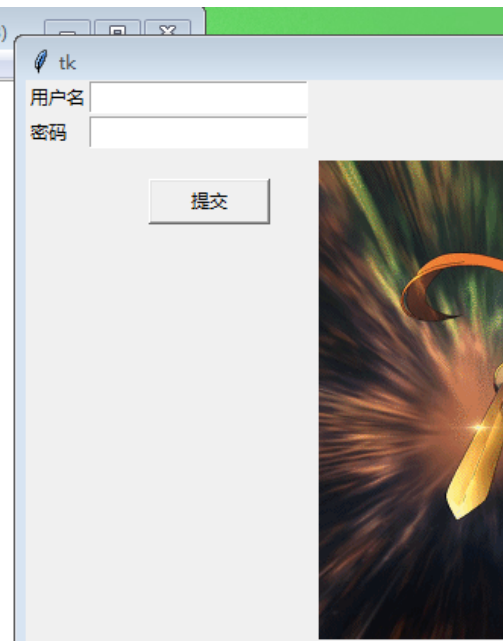
```
Label(root,text='密码').grid(row=1,sticky=W)
```

```
Entry(root).grid(row=0,column=1)
```

```
Entry(root,show='*').grid(row=1,column=1)
```

```
Button(root,text='提交',width=10).grid(row=2,columnspan=3,pady=5)
```

```
mainloop()
```



place通常不建议使用

```
from tkinter import *
```

```
root=Tk()
```

```
photo=PhotoImage(file='刀剑神域_亚丝娜.gif')
```

```
Label(root,image=photo).pack(expand=True)
```

```
def callback():  
    print('正中靶心')
```

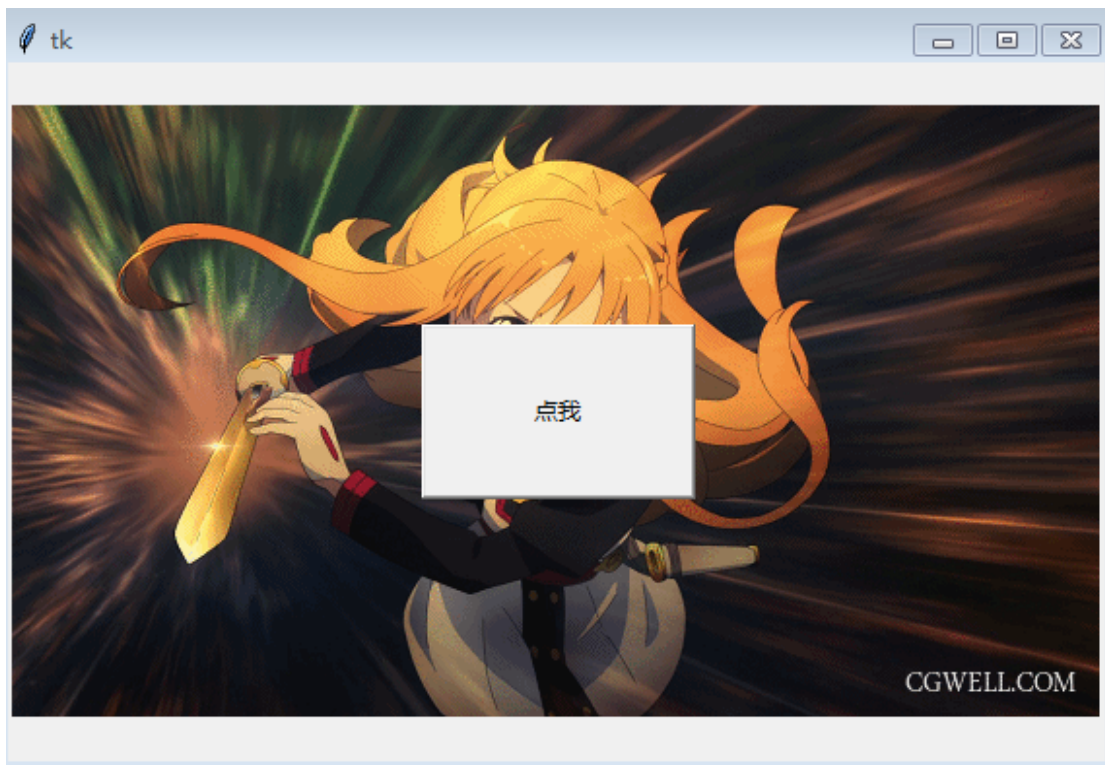
```
#0.5表示正中间，1表示最右边。relx表示相对父组件的距离
```

```
#relheight表示相对父组件的高度，relwidth相对父组件的尺寸
```

```
Button(root,text='点我',command=callback).\  
    place(relx=0.5,rely=0.5,relheight=0.25,\  
          relwidth=0.25,anchor=CENTER)
```

```
mainloop()
```

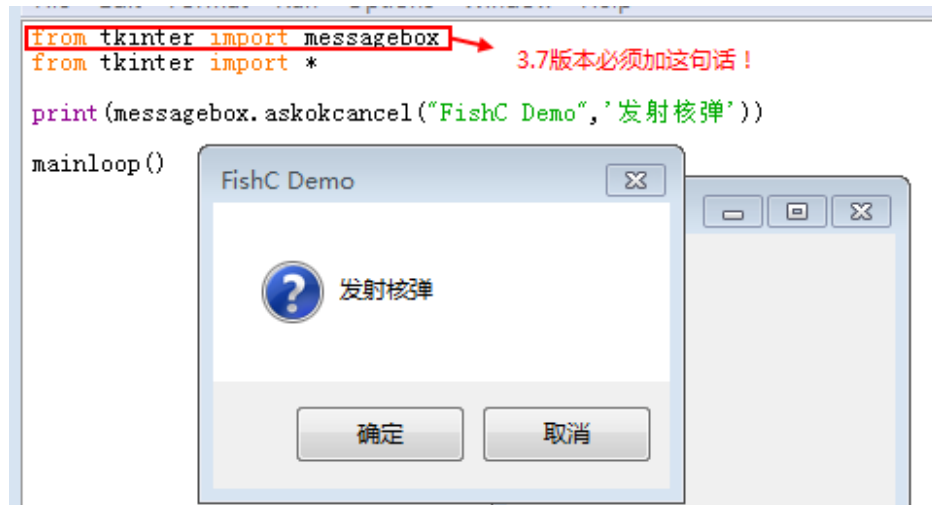
显示效果：



Tkinter 模块-messagebox

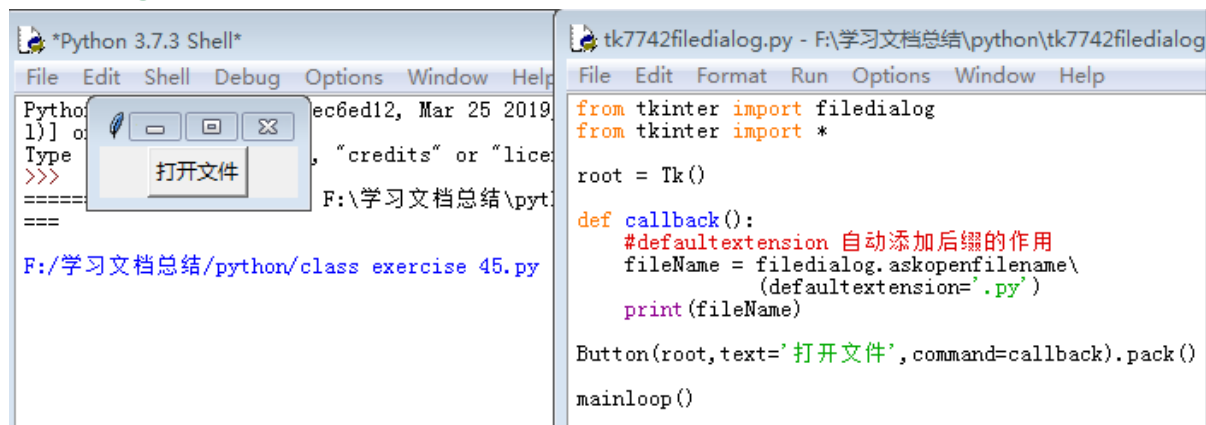
[https://fishc.com.cn/forum.php?](https://fishc.com.cn/forum.php?mod=viewthread&tid=61678&extra=page%3D1%26filter%3Dtypeid%26typeid%3D604)

[mod=viewthread&tid=61678&extra=page%3D1%26filter%3Dtypeid%26typeid%3D604](https://fishc.com.cn/forum.php?mod=viewthread&tid=61678&extra=page%3D1%26filter%3Dtypeid%26typeid%3D604)



具体有用到窗口再看吧，如果选择取消返回False，如果选择确定返回True。

filedialog (文件对话框)



可以根据内容进行筛选：

```
from tkinter import filedialog
from tkinter import *

root = Tk()

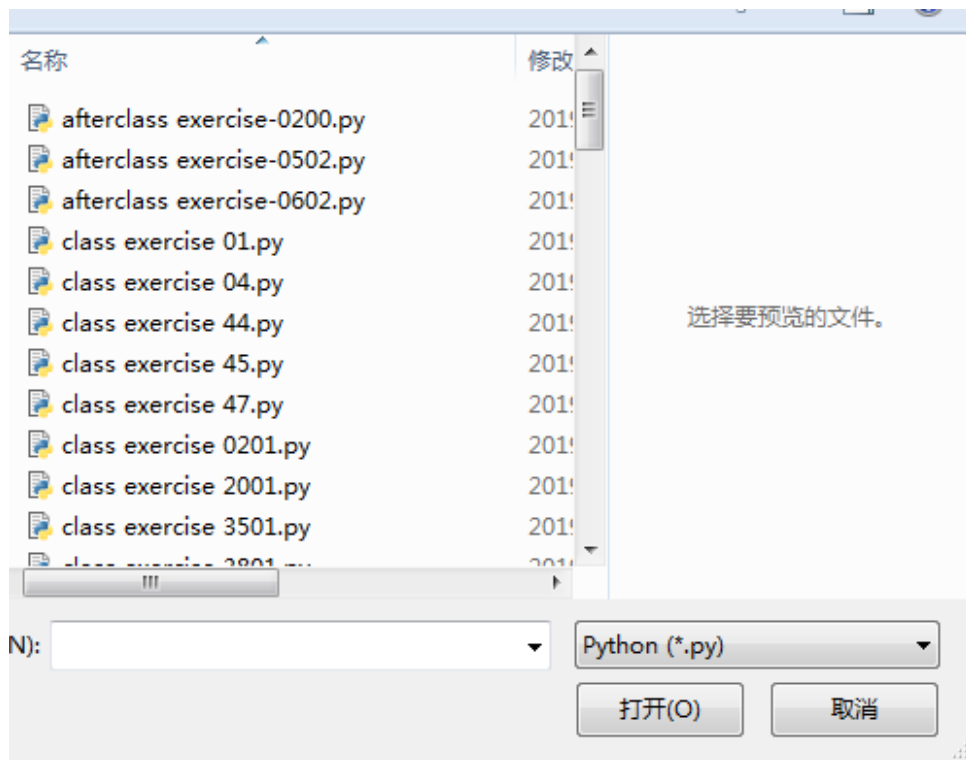
def callback():
    #defaultextension 自动添加后缀的作用
    #fileName = filedialog.askopenfilename\
    #    (defaultextension='.py')
    #可以根据filetypes的类型对文件进行筛选
    fileName = filedialog.askopenfilename\
        (filetypes=[('PNG', '.png'),
                     ('GIF', '.gif'),
                     ('JPG', '.jpg'),
                     ('Python', '.py')])

    print(fileName)

Button(root, text='打开文件', command=callback).pack()

mainloop()
```

筛选的效果展示：



colorchooser (颜色选择对话框)

```
from tkinter import colorchooser
from tkinter import *

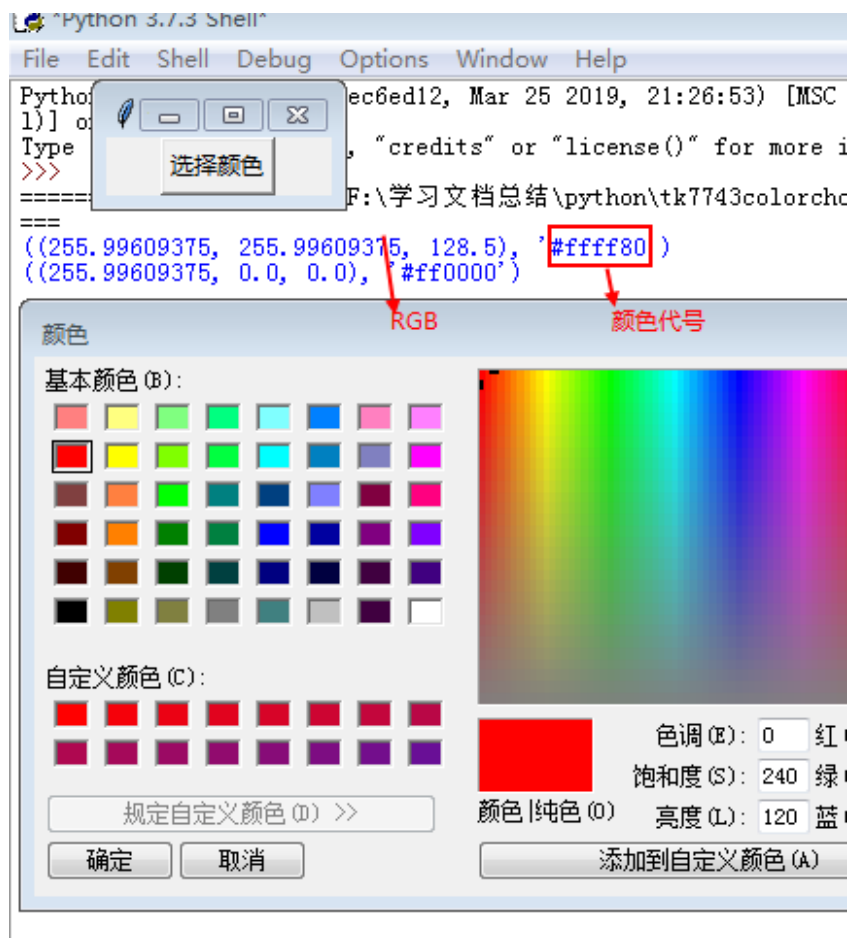
root = Tk()

def callback():
    #询问你需要什么颜色，打开颜色管理器
    fileName = colorchooser.askcolor()
    print(fileName)

Button(root, text='选择颜色', command=callback).pack()

mainloop()
```

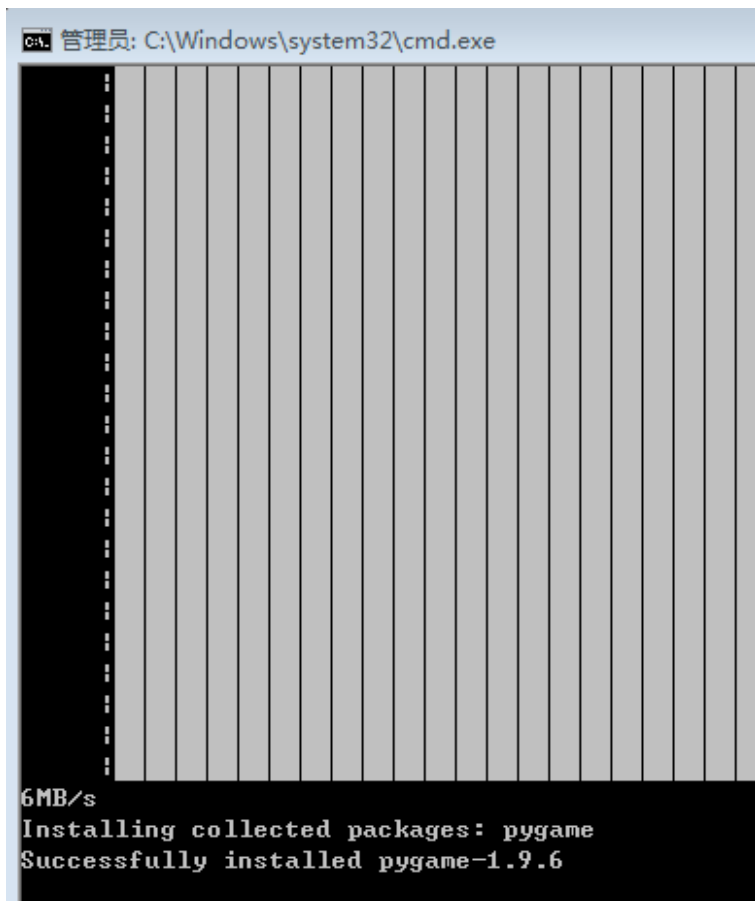
效果展示：



pygame 篇章

pygame 安装

直接 CMD: pip install pygame



作用：

- 1) 绘制图形
- 2) 显示图片
- 3) 动画效果
- 4) 键盘交互
- 5) 碰撞检测

具体资料：

<https://fishc.com.cn/forum.php?mod=forumdisplay&fid=243&filter=typeid&typeid=605>

pygame答疑

Q1、什么是Surface对象？

答：Surface就是图像

Q2、将一个图形绘制到另一个图像上面是怎么回事？

答：在pygame中，由于图像是由像素组成，也就是马赛克，其实没有在一个图形绘制到另一个图像上的说法，实际上就是改变该被覆盖的图像的像素，就是使用blit方法将图像这个一群像素覆盖到另一个图像的像素上面。

Q3、移动图像是怎么回事？

答：涉及到帧率的问题，指的是一秒钟可以切换多少次图像，调用get_rect对象的move方法，改变矩形范围的位置，对应代码的第29行，其中speed = [-2, 1]，相当于水平位置减少2，垂直位置加1来移动矩形的位置。再更新图像和界面，在内存中进行双缓冲。

Q4、如何控制游戏的速度？

答：使用了time模块的delay方法，延迟速度，也就是每隔10ms移动一次矩形也就是图像；也可以通过改变帧率的形式，注释掉delay方法，假如clock来通过设置帧率来控制速度，编写代码如下：

```
12 clock = pygame.time.Clock() # 实例化Clock对象
48 clock.tick(200) # 改变帧率为200, 图像1秒移动200次
```

Q5、pygame的效率高不高？

答：pygame的大部分模块都是由C语言编写和优化，效率很好，帧率是40-200帧/秒。

Q6、如何从pygame中获得帮助？

答：pygame的官网有很详细的文档和教程以及例子，小甲鱼论坛里面也有模块的中文翻译计划，也有找到一个很好的网站学习pygame，[博客地址](#)。

总结：虽然不从事游戏开发，但是了解东西总是多多益善，尤其是编程类，了解多了，也就能够互通了。

py里面事件的本质：

查询代码：

```
# coding=utf-8
import pygame
import sys

#初始化pygame
pygame.init()

size=width,height=1000,800
speed=[-2,1]
bg=(255,255,255)

#创建指定大小的窗口 Surface
screen = pygame.display.set_mode(size)

#设置窗口标题
pygame.display.set_caption('初次见面，请多多关照')

f=open('record.txt','w')

while True:
    #当在事件流中
    for event in pygame.event.get():
        f.write(str(event)+'\n')

        if event.type == pygame.QUIT:
            f.close()
            sys.exit()
```

结果显示：

```
<Event(17-VideoExpose {})>
<Event(16-VideoResize {'size': (1000, 800), 'w': 1000, 'h': 800})>
<Event(1-ActiveEvent {'gain': 0, 'state': 1})>
<Event(4-MouseMotion {'pos': (343, 325), 'rel': (344, 326), 'buttons': (0, 0, 0), 'window': None})>
<Event(1-ActiveEvent {'gain': 1, 'state': 1})>
<Event(4-MouseMotion {'pos': (342, 325), 'rel': (-1, 0), 'buttons': (0, 0, 0), 'window': None})>
<Event(4-MouseMotion {'pos': (342, 323), 'rel': (0, -2), 'buttons': (0, 0, 0), 'window': None})>
<Event(4-MouseMotion {'pos': (342, 318), 'rel': (0, -5), 'buttons': (0, 0, 0), 'window': None})>
<Event(4-MouseMotion {'pos': (343, 309), 'rel': (1, -9), 'buttons': (0, 0, 0), 'window': None})>
<Event(4-MouseMotion {'pos': (344, 297), 'rel': (1, -12), 'buttons': (0, 0, 0), 'window': None})>
<Event(4-MouseMotion {'pos': (347, 285), 'rel': (3, -12), 'buttons': (0, 0, 0), 'window': None})>
<Event(4-MouseMotion {'pos': (351, 269), 'rel': (4, -16), 'buttons': (0, 0, 0), 'window': None})>
<Event(4-MouseMotion {'pos': (356, 249), 'rel': (5, -20), 'buttons': (0, 0, 0), 'window': None})>
<Event(4-MouseMotion {'pos': (366, 226), 'rel': (10, -23), 'buttons': (0, 0, 0), 'window': None})>
<Event(4-MouseMotion {'pos': (380, 201), 'rel': (14, -25), 'buttons': (0, 0, 0), 'window': None})>
<Event(4-MouseMotion {'pos': (397, 176), 'rel': (17, -25), 'buttons': (0, 0, 0), 'window': None})>
<Event(4-MouseMotion {'pos': (426, 149), 'rel': (29, -27), 'buttons': (0, 0, 0), 'window': None})>
<Event(4-MouseMotion {'pos': (460, 127), 'rel': (34, -22), 'buttons': (0, 0, 0), 'window': None})>
<Event(4-MouseMotion {'pos': (508, 102), 'rel': (48, -25), 'buttons': (0, 0, 0), 'window': None})>
<Event(4-MouseMotion {'pos': (562, 82), 'rel': (54, -20), 'buttons': (0, 0, 0), 'window': None})>
<Event(4-MouseMotion {'pos': (625, 67), 'rel': (63, -15), 'buttons': (0, 0, 0), 'window': None})>
<Event(4-MouseMotion {'pos': (681, 54), 'rel': (56, -13), 'buttons': (0, 0, 0), 'window': None})>
```

如何做一个刷新的事件界面

```
# coding=utf-8
```

```
import pygame
```

```
import sys
```

#初始化pygame

pygame.init()

size=width,height=1000,800

speed=[-2,1]

bg=(0,0,0)

font_color = (0,255,0)

#设置字体

font = pygame.font.Font(None,20)

#设置初始列位置

position = 0

#获取行高

line_height = font.get_linesize()

#创建指定大小的窗口 Surface

screen = pygame.display.set_mode(size)

#设置窗口标题

pygame.display.set_caption('植树poi Demo')

#将背景刷成黑色

screen.fill(bg)

while True:

 #当在事件流中

 for event in pygame.event.get():

 if event.type == pygame.QUIT:

 sys.exit()

 #第一个为需要显示的文本，第二个是否消除锯齿(设置渲染信息)

 #此时font_render 相当于一个Surface对象

 font_render=font.render(str(event),True,font_color)

 screen.blit(font_render,(0,position))

```
position += line_height
```

```
if position > height:
```

```
    position = 0
```

```
    #重新将界面刷成黑色
```

```
    screen.fill(bg)
```

```
#更新页面
```

```
pygame.display.flip()
```

如何提高pygame游戏的颜值：

resolution 指定界面的大小,flag 指定扩展选项, depth 指定颜色位数, 一般不设置。

set_mode(resolution=(0,0),flag=0, depth) -> Surface

- flag扩展选项

选项	含义
FULLSCREEN	全屏模式
DOUBLEBUF	双缓冲模式
HWSURFACE	硬件加速支持（只有在全屏模式下才能使用）
OPENGL	使用OpenGL渲染
RESIZABLE	使得窗口可以调整大小
NOFRAME	使得窗口没有边框和控制按钮

如何设置全屏模式：

```
#设置全屏标志
```

```
fullscreen = False
```

```
#获取当前显示器全屏分辨率
```

```
screen_list = pygame.display.list_modes()
```

```
while True:
```

```
    #当在事件流中
```

```
    for event in pygame.event.get():
```

```
        if event.type == pygame.QUIT:
```

```
            sys.exit()
```

```
        if event.type == pygame.KEYDOWN:
```

```
            if event.key == pygame.K_LEFT:
```

```
                classic = l_head
```

```
                speed = [-1,0]
```

```
            if event.key == pygame.K_RIGHT:
```

```
                classic = r_head
```

```
                speed = [1,0]
```

```
            if event.key == pygame.K_UP:
```

```
                speed = [0,-1]
```

```
            if event.key == pygame.K_DOWN:
```

```
                speed = [0,1]
```

```
    #全屏【F11】
```

```
    if event.key == pygame.K_F11:
```

```
        fullscreen = not fullscreen
```

```
        if fullscreen:
```

```
            #屏幕开始全屏，并启动硬件加速
```

```
            screen = pygame.display.set_mode(screen_list[0],\
                                                pygame.FULLSCREEN | \
                                                pygame.HWSURFACE)
```

```
        else:
```

```
            screen = pygame.display.set_mode(size)
```

如何设置可调节的窗口:

```
    else:
```

```
        screen = pygame.display.set_mode(size)
```

```
    #用户调整窗口尺寸
```

```
    if event.type == VIDEORESIZE:
```

```
        size = event.size
```

```
        width,height = size
```

```
        print(size)
```

```
        screen = pygame.display.set_mode(size, RESIZABLE)
```

→ 并设置事件函数，重新调整尺寸。

将RESIZABLE模式打开

移动图像

如何设置 (surface) 的放大和缩小

```
#设置放大缩小的比例
```

```
ratio =1.0
```

```
#加载图片
```

```
oclassic=pygame.image.load('F:\\\\学习文档总结\\\\python\\\\乔巴.png')
```

```
classic = oclassic
```

```
#获得图像的位置矩形
```

```
oclassic_rect = oclassic.get_rect()
```

```
position = classic_rect = oclassic_rect
```



```

#图片放大缩小的比例，空格恢复原始尺寸
if event.key == K_EQUALS or event.key == K_MINUS \
    or event.key == K_SPACE:
    #最大只能放大一倍，缩小50%
    if event.key == K_EQUALS and ratio < 2:
        ratio += 0.1
    if event.key == K_MINUS and ratio > 0.5:
        ratio -= 0.1
    if event.key == K_SPACE:
        ratio = 1.0
    #第二个参数是一个元组
    classic= pygame.transform.smoothscale(oclassic,\
                                           (int(oclassic_rect.width*ratio),\
                                           int(oclassic_rect.height*ratio)))

```

pygame8108rotate贴边飞行.py

通过rotate实现贴边飞行

rotate 是逆时针方向旋转。

```

#移动图像
position = position.move(speed)

#如果超出右边，则图片更新，并重新获取position
#position 有4个属性，left,top左上角的位置属性
#width,height,矩形的长，高 right,所在最右边的位置
#bottom则是所在底布的位置
if position.right > width:
    classic = classic_right
    position = classic_rect = classic.get_rect()
    position.left = width - classic_rect.width
    speed = [0,5]

if position.bottom > height:
    classic = classic_bottom
    position = classic_rect = classic.get_rect()
    position.left = width - classic_rect.width
    position.top = height - classic_rect.height
    speed = [-5,0]

if position.left < 0:
    classic = classic_left
    position = classic_rect = classic.get_rect()
    position.top = height - classic_rect.height
    speed = [0,-5]

if position.top < 0:
    classic = classic_top
    position = classic_rect = classic.get_rect()
    speed = [5,0]

```

pygame8108rotate贴边飞行.py

pygame8108rotate贴边飞行.py 练习内容。

https://blog.csdn.net/qq_38970783/article/details/89160731

实现裁剪功能的实现

```

import pygame
import sys
from pygame.locals import *

```

#初始化pygame

pygame.init()

size=width,height=1000,600

bg=(255,255,255)

```
clock = pygame.time.Clock()
```

#创建指定大小的窗口 Surface

```
screen = pygame.display.set_mode(size)
```

#设置窗口标题

```
pygame.display.set_caption('初次见面，请多多关照')
```

#加载图片

```
classic = pygame.image.load('乔巴.png')
```

#0 -> 未选择, 1 -> 选择中, 2 -> 完成选择

```
select = 0
```

```
select_rect = pygame.Rect(0,0,0,0)
```

#0 -> 未拖拽, 1 -> 拖拽中, 2 -> 完成拖拽

drag = 0

#获得图像的位置矩形

```
position = classic.get_rect()
```

```
position.center = width // 2,height // 2
```

```
while True:
```

#当在事件流中

```
for event in pygame.event.get():
```

```
if event.type == QUIT:
```

```
sys.exit()
```

```
elif event.type == MOUSEBUTTONDOWN:
```

```
if event.button == 1:
```

#第一次点击，选择范围

```
if select == 0 and drag == 0:
```

```
pos start = event.pos
```

select = 1

#第二次点击，拖拽图像

```

elif select == 2 and drag == 0:
    #获得它的子对象,即裁剪的图像
    capture=screen.subsurface(select_rect).copy()
    cap_rect = capture.get_rect()
    drag = 1
    # 第三次点击, 初始化
elif select == 2 and drag ==2 :
    select = 0
    drag = 0

```

```

elif event.type == MOUSEBUTTONUP:
    if event.button == 1:
        #第一次释放, 结束选择
        if select ==1 and drag == 0:
            pos_stop = event.pos
            select = 2
        #第二次释放, 结束拖拽
        if select == 2 and drag ==1:
            drag = 2

```

#填充背景

```
screen.fill(bg)
```

#更新图像

```
screen.blit(classic,position)
```

#实时绘制选择框

```

if select:
    mouse_pos = pygame.mouse.get_pos()
    if select == 1 :
        pos_stop = mouse_pos
        select_rect.left,select_rect.top = pos_start
        select_rect.width,select_rect.height = pos_stop[0]-pos_start[0],\
                                                pos_stop[1]-pos_start[1]
        pygame.draw.rect(screen,(0,0,0),select_rect,1)

```

#拖拽裁剪的图像

```
if drag:
```

```
if drag == 1:
    cap_rect.center = mouse_pos
    screen.blit(capture,cap_rect)
```

```
pygame.display.flip()
```

```
clock.tick(30)
```

如何把图片转化为透明

融合多通道



重要代码分析：

```
def blit_alpha(target, source, location, opacity):
    #而形参 location对应的实参是 position，就是乔巴的位置。
    x = location[0]
    y = location[1]
    #将传进来带alpha通道的图片转变成不带通道的，
    #其实我们只是需要它的一个矩形区域。
    temp = pygame.Surface((source.get_width(),\
                           source.get_height())).convert()
    #这里的(-x,-y)是背景的起点位置，在这绘制背景
    temp.blit(target, (-x,-y))
    #然后我们就在A的位置贴上背景透明的小甲鱼图片
    temp.blit(source, (0,0))
    #将整个temp区域透明度设置为200
    temp.set_alpha(opacity)

    target.blit(temp, location)
```

target=背景, source = 目标图片, location = 目标图片起始坐标 (左上角), opacity = 透明度

pygame里面的绘制内容

绘制矩形

```
pygame.draw.rect()
```

```
rect(Surface, color, Rect, width = 0)
```

在 Surface对象上绘制一个矩形。color 表示边框的颜色。Rect 参数指定矩形的位置和尺寸。width 参数指定边框的宽度, 如果设置为 0 则表示填充该矩形。

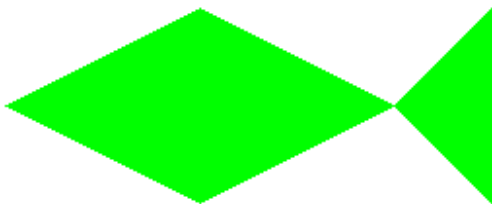


绘制多边形

```
pygame.draw.polygon() 绘制多边形。
```

```
polygon(Surface, color, pointlist, width=0)
```

在Surface对象上绘制一个多边形。pointlist 参数指定多边形的各个顶点。width 参数指定边框的宽度, 如果设置为 0 则表示填充该矩形。



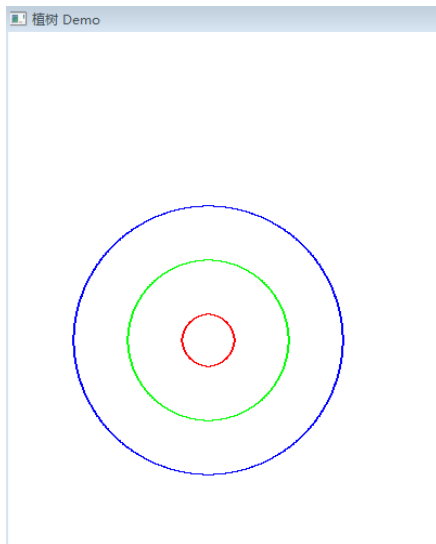
绘制圆形

```
pygame.draw.circle()
```

根据圆心和半径绘制圆形。

```
circle(Surface, color, pos, radius, width=0)
```

在 Surface对象上绘制一个圆形。pos 参数指定圆心的位置, radius 参数指定圆的半径。width 参数指定边框的宽度, 如果设置为 0 则表示填充该矩形。



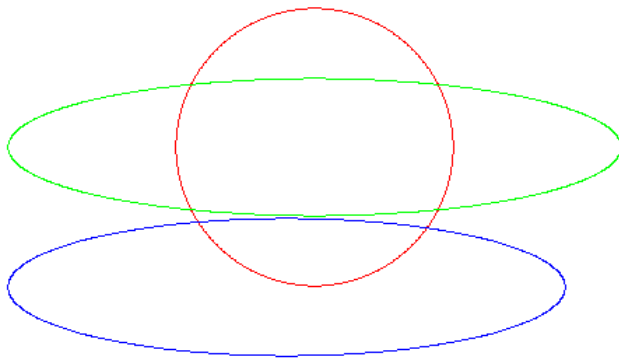
绘制椭圆形

`pygame.draw.ellipse()`

根据限定矩形绘制一个椭圆形。

`ellipse(Surface, color, Rect, width=0)`

在 Surface 对象上绘制一个椭圆形。Rect 参数指定椭圆外围的限定矩形（当此矩形为正方形时，得到的也是圆形）。width 参数指定边框的宽度，如果设置为 0 则表示填充该矩形。

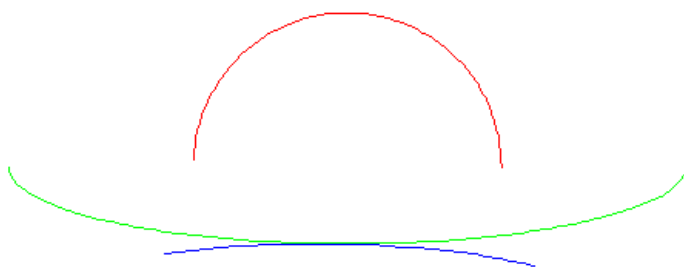


绘制弧形

`pygame.draw.arc()` 绘制弧线。

`arc(Surface, color, Rect, start_angle, stop_angle, width=1)`

在 Surface 对象上绘制一条弧线。Rect 参数指定弧线所在的椭圆外围的限定矩形。两个 angle 参数指定弧线的开始和结束位置，使用的是弧度制。width 参数指定边框的宽度。

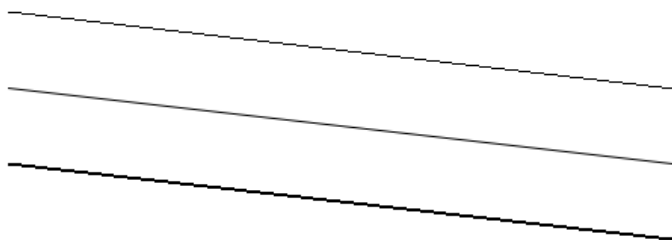


线段的绘制

`pygame.draw.aalines()` 绘制多条连续的线段（抗锯齿）。

`aalines(Surface, color, closed, pointlist, blend=1)`

在 Surface 对象上绘制一系列连续的线段（抗锯齿）。如果 `closed` 参数为 `True`，则首尾相连。`blend` 参数指定是否通过绘制混合背景的阴影来实现抗锯齿功能。该函数的结束位置允许使用浮点数。



动画精灵

```

import pygame
import sys
from pygame.locals import *
from random import *

#初始化pygame
pygame.init()

class Ball(pygame.sprite.Sprite):#继承动画精灵的基类
    def __init__(self, image, position, speed, bg_size):
        pygame.sprite.Sprite.__init__(self)

        self.image = pygame.image.load(image).convert_alpha()
        self.rect = self.image.get_rect() #获得球的尺寸，外切矩形

        self.rect.left, self.rect.top = position #将出现位置赋予给球
        self.speed = speed #速度设置
        self.width, self.height = bg_size[0], bg_size[1] #设置边界

    def move(self):
        self.rect = self.rect.move(self.speed) #根据自身的速度进行移动

        #设置球活动的边界
        #小球的右边超过左边边界时
        if self.rect.right < 0:
            self.rect.left = self.width #让他从右边界回来
        #小球的左边超过右边边界时
        if self.rect.left > self.width:
            self.rect.right = 0 #让他从左边界回来
        #小球的底部超过上边边界时
        if self.rect.bottom < 0:
            self.rect.top = self.height #让他从下边界回来
        #小球的顶部超过下边边界时
        if self.rect.top > self.height:
            self.rect.bottom = 0 #让他从上边界回来

```

如果是边界碰撞则改变速度的方向就行了。

```

def main():
    pygame.init()

    bg_image = '世界地图background.png'
    ball_image = '小球.png'

    running = True #为了以后有多种方法退出程序

    bg_size = width, height = 1024, 681 #背景大小设置
    screen = pygame.display.set_mode(bg_size) #设置背景大小
    background = pygame.image.load(bg_image).convert_alpha() #导入背景

    balls = []

    for i in range(5):#生成5个球
        #要减去100是因为球图片尺寸的大小为100，随机生成位置
        position = randint(0, width-100), randint(0, height-100)
        speed = [randint(-10, 10), randint(-10, 10)]
        #生成球的对象
        ball = Ball(ball_image, position, speed, bg_size)
        balls.append(ball) #将所有的球添加到类的对象管理

    clock = pygame.time.Clock() #生成刷新频率的控制器

    while running:
        #当在事件流中
        for event in pygame.event.get():
            if event.type == QUIT:
                sys.exit()

        screen.blit(background, (0, 0)) #将背景渲染到画布上

        for each in balls:
            each.move()
            screen.blit(each.image, each.rect)

        pygame.display.flip()

        clock.tick(30)

```


碰撞检测

内置的一个碰撞检测的函数：

`spritecollide(sprite, group, dokill, collided = None)`

第一个参数 `sprite` 是指定被检测的精灵；（就是我们写的里面的 `item`）

第二个参数 `group` 是指定一个组（就是我们写的里面的 `target` 列表），它是 `sprite` 的组，因此要使用 `sprite.Group()` 来生成；

第三个参数 `dokill` 是设置是否从组中删除检测到碰撞的精灵，设置为 `True`，则删除；

第四个参数 `collided` 是指定一个回调函数，用于定制特殊的检测方法，如果第四个参数忽略的话，默认是检测精灵之间的 `rect` 属性。

```
for i in range(5): #生成5个球
    #要减去100是因为球图片尺寸的大小为100，随机生成位置
    position = randint(0,width-100),randint(0,height-100)
    speed = [randint(-10,10),randint(-10,10)]
    #生成球的对象
    ball = Ball(ball_image,position,speed,bg_size)
    while pygame.sprite.spritecollide(ball,group,False,\
        pygame.sprite.collide_circle):
        #如果碰撞了，重新生成一个小球
        ball.rect.left,ball.rect.top = randint(0,width-100),\
            randint(0,height-100)
    balls.append(ball) #把所有的球添加到类的对象管理
    group.add(ball)

while running:
    #当在事件流中
    for event in pygame.event.get():
        if event.type == QUIT:
            sys.exit()

    screen.blit(background, (0,0)) #将背景渲染到画布上

    for each in balls:
        each.move()
        screen.blit(each.image,each.rect)
    #循环5个小球，分别判断是否与另外4个球发生碰撞
    for each in group:
        #先取出一个小球
        group.remove(each)

        if pygame.sprite.spritecollide(each,group,False,\
            pygame.sprite.collide_circle):
            #碰撞发生后，向反方向运动
            each.speed[0] = -each.speed[0]
            each.speed[1] = -each.speed[1]
            #把小球放回原位
            group.add(each)

    pygame.display.flip()

    clock.tick(30)
```

如何给游戏插入声音

播放音效：

`pygame.mixer.Sound()`

播放音效我们使用 `mixer` 模块，在使用之前，需要先生成一个 `Sound` 对象，对这个 `Sound` 对象进行控制，`Sound` 对象的 `play()` 方法就是播放音效，稍候我

们来讲这些方法。

播放背景音乐： -pygame.mixer.music

播放背景音乐我们使用 music 模块，music 模块和 mixer 模块是紧密关联的。

- Sound 的对象方法

方法	含义
play()	播放音效
stop()	停止播放
fadeout()	淡出
set_volume()	设置音量
get_volume()	获取音量
get_num_channels()	计算该音效播放了多少次
get_length()	获得该音效的长度
get_raw()	将该音效以二进制格式的字符串返回

- music的对象方法：

方法	含义
load()	载入音乐
play()	播放音乐
rewind()	重新播放
stop()	停止播放
pause()	暂停播放
unpause()	恢复播放
fadeout()	淡出
set_volume()	设置音量
get_volume()	获取音量
get_busy()	检测音乐流是否正在播放
set_pos()	设置开始播放的位置
get_pos()	获取已经播放的时间
queue()	将音乐文件放入待播放列表中
set_endevent()	在音乐播放完毕时发送事件
get_endevent()	获取音乐播放完毕时发送的事件类型

关于自定义鼠标的光标：

我们的做法就是使用 set_visible() 方法来将光标设为不可见，然后使用 get_pos() 方法获取鼠标的实时位置，获取之后，我们实时的把 hand.png 图片画到这个位置上去，这样子就相当于实现了自定义光标。

pygame.mouse.set_visible() —— 隐藏或显示鼠标光标

pygame.mouse.get_pos() —— 获取鼠标光标的位置

```
# running = False #防止循环
#如果事件类型为鼠标移动
elif event.type == MOUSEMOTION:
    #获取鼠标当前位置
    mouse_x, mouse_y = pygame.mouse.get_pos()
    if (glass_rect.left <= mouse_x <= glass_rect.right) \
        and (glass_rect.top <= mouse_y <= glass_rect.bottom):
        flag = True
    else : #不在范围内鼠标可见
        flag = False

screen.blit(glass, (glass_rect.left, glass_rect.top))
if flag: #如果鼠标进入玻璃板
    #设置鼠标不可见
    mouse_x, mouse_y = pygame.mouse.get_pos()
    pygame.mouse.set_visible(False) #设置鼠标不可见
    screen.blit(hand, (mouse_x, mouse_y)) #画上我们自定义的鼠标
else: #如果鼠标没进入玻璃板
    pygame.mouse.set_visible(True)
    screen.blit(hand2, (mouse_x, mouse_y))
```

设置小球响应光标的移动频率

步骤如下：

- 1、为每个小球设定一个不同的目标;
- 2、创建一个motion 变量来记录每一秒钟产生事件数量;
- 3、为小球添加一个 check() 方法, 用于判断鼠标在1秒钟内产生的事件数量是否匹配此目标;
- 4、添加一个自定义事件, 每一秒钟触发一次。调用每个小球的 check() 检测 motion 的值是否匹配某一个小球的目标, 并将motion重新初始化, 以便记录下1秒鼠标事件数量;
- 5、小球应该添加一个 control 属性, 用于记录当前的状态 (绿色 -> 玩家控制 or 灰色 -> 随机移动) 。
- 6、通过检查 control 属性决定绘制什么颜色的小球。

```
import pygame
import sys
from pygame.locals import *
from random import *

#初始化pygame
pygame.init()

class Ball(pygame.sprite.Sprite):#继承动画精灵的基类
    def __init__(self,image,position,speed,bg_size,target):
        pygame.sprite.Sprite.__init__(self)

        self.image = pygame.image.load(image).convert_alpha()
        self.rect = self.image.get_rect() #获得球的尺寸, 外切矩形

        self.rect.left,self.rect.top = position #将出现位置赋予给球
        self.speed = speed #速度设置
        self.target = target #设置一个使小球变为可控的目标
        self.control = False #判断小球是否可控的标志
        self.width,self.height = bg_size[0],bg_size[1]#设置边界

        self.radius = self.rect.width /2

    def drawball(self,greenball_image):
        #如果小球受控
        if self.control:
            #重新绘制小球为绿色的小球
            self.image = pygame.image.load(greenball_image)

    def move(self):
        self.rect = self.rect.move(self.speed) #根据自身的速度进行移动

        #设置球活动的边界
        #小球的右边超过左边边界时
        if self.rect.right < 0:
            self.rect.left = self.width #让他从右边界回来
        #小球的左边超过右边边界时
        if self.rect.left > self.width:
            self.rect.right = 0 #让他从左边界回来
        #小球的底部超过上边界时
```

```

        if self.rect.bottom < 0:
            self.rect.top = self.height #让他从下边界回来
        #小球的顶部超过下边边界时
        if self.rect.top > self.height:
            self.rect.bottom = 0 #让他从上边界回来

    def check(self, motion): #检查鼠标移动的频率是否达到控制小球的目标值
        if self.target - 2 <= motion <= self.target + 2:
            #小球的可控标志为1
            self.control = True
            return True
        else:
            return False

def main():
    pygame.init()

    #导入所需要的图片
    bg_image = '世界地图background.png'
    grayball_image = '小球.png'
    greenball_image = '小球2.png'
    glass_image = '滑板.png'
    hand_image = 'hand.png'
    hand2_image = 'hand2.png'

    running = True #为了以后有多种方法退出程序

    bg_size = width, height = 1024, 681 #背景大小设置
    screen = pygame.display.set_mode(bg_size) #设置背景大小
    background = pygame.image.load(bg_image).convert_alpha() #导入背景

    #导入背景音乐
    #pygame.mixer.music.load('bg_music.ogg')
    #pygame.mixer.music.set_volume(0.2) #设置音量
    #pygame.mixer.music.play() #播放背景音乐

    #设置一个背景音乐完毕之后的结束时间
    GAMEOVER = USEREVENT
    #当背景音乐播完后，发送一个GAMEOVER事件
    #pygame.mixer.music.set_endevent(GAMEOVER)

    #加载音效
    #hole_music = pygame.mixer.Sound('hole.wav')
    #laugh_music = pygame.mixer.Sound('laugh.wav')
    #winner_music = pygame.mixer.Sound('winner.wav')
    #loser_music = pygame.mixer.Sound('loser.wav')

    #绘制用于摩擦的玻璃板，要放在小球前面，不然后面
    #小球会从玻璃板的下方划过
    glass = pygame.image.load(glass_image).convert_alpha()
    glass_rect = glass.get_rect() #获得玻璃板的尺寸
    #玻璃板放置的位置
    glass_rect.left, glass_rect.top = ((width - glass_rect[2]) / 2, \
                                       (height - glass_rect[3]))

    #加载鼠标图片
    #玻璃板内的鼠标
    hand = pygame.image.load(hand_image).convert_alpha()
    #平常鼠标
    hand2 = pygame.image.load(hand2_image).convert_alpha()
    #设置一个布尔变量，判断鼠标是否进入了玻璃板的范围。
    flag = False
    #用来计数一秒钟的移动的次數
    motion = 0
    #设置一个自定义事件，用来检测鼠标移动的值是否符合控制
    #小球的目标值
    MYTIMER = USEREVENT + 1
    #计时器事件设置为1秒
    pygame.time.set_timer(MYTIMER, 1000)

    balls = []
    group = pygame.sprite.Group() #设置一个检测目标组

    BALL_NUM = 5

```

```

for i in range(BALL_NUM): #生成5个球
    #要减去100是因为球图片尺寸的大小为100，随机生成位置
    position = randint(0,width-100),randint(0,height-100)
    speed = [randint(-10,10),randint(-10,10)]
    #生成球的对象
    ball = Ball(grayball_image,position,speed,bg_size,5*(i+1))
    while pygame.sprite.spritecollide(ball,group,False,\
        pygame.sprite.collide_circle):
        #如果碰撞了，重新生成一个小球，防止因为连续碰撞后，小球被卡在
        #某个小球的区域内，因为无法立即调出，就会产生抖动。
        ball.rect.left,ball.rect.top = randint(0,width-100),\
            randint(0,height-100)
    balls.append(ball) #将所有的球添加到类的对象管理
    group.add(ball)

clock = pygame.time.Clock() #生成刷新频率的控制器

```

```

while running:
    #当事件流中
    for event in pygame.event.get():
        if event.type == QUIT:
            sys.exit()
        #elif event.type == GAMEOVER: #如果游戏结束
        #    loser_music.play() #播放失败音乐
        #    time.sleep(3) #延时2秒
        #    laugh_music.play() #播放大笑音乐
        #    running = False #终止循环
        #如果事件类型为鼠标移动
        elif event.type == MOUSEMOTION:
            #获取鼠标当前位置
            mouse_x,mouse_y = pygame.mouse.get_pos()
            if (glass_rect.left <= mouse_x <=glass_rect.right)\
                and (glass_rect.top <=mouse_y <= glass_rect.bottom):
                motion += 1 #鼠标在玻璃板内滑动，那么motion+1
                flag = True
            else : #不在范围内鼠标可见
                flag = False
            #如果事件类型为自己定义的定时器事件
        elif event.type == MYTIMER:
            for each in balls:
                #调用他们各自的check()函数
                if each.check(motion):
                    each.drawball(greenball_image)
                    each.speed = [0,0] #将其速度设置为0，进入可控状态
                #等到所有的球都进行判断之后将motion重设置为0，
                #进入下一秒循环
                motion = 0

    screen.blit(background, (0,0)) #将背景渲染到画布上
    #将玻璃板绘制在screen上
    screen.blit(glass,(glass_rect.left,glass_rect.top))
    #如果鼠标进入玻璃板

```

```

if flag: #如果鼠标进入玻璃板
    #设置鼠标不可见
    mouse_x, mouse_y = pygame.mouse.get_pos()
    pygame.mouse.set_visible(False) #设置鼠标不可见
    screen.blit(hand, (mouse_x, mouse_y)) #画上我们自定义的鼠标
else: #如果鼠标没进入玻璃板
    pygame.mouse.set_visible(True)
    screen.blit(hand2, (mouse_x, mouse_y))

for each in balls:
    each.move()
    screen.blit(each.image, each.rect)
#循环5个小球，分别判断是否与另外4个球发生碰撞
for each in group:
    #先取出一个小球
    group.remove(each)

    if pygame.sprite.spritecollide(each, group, False, \
                                    pygame.sprite.collide_circle):
        #碰撞发生后，向反方向运动
        each.speed[0] = -each.speed[0]
        each.speed[1] = -each.speed[1]
        #把小球放回原位
        group.add(each)

pygame.display.flip()

clock.tick(30)

if __name__ == '__main__':
    main()

```

[小球游戏.md](#)

飞机大战

游戏基本介绍:

- 1、敌方共有大中小3款飞机，分别为高中低三中速度，自己还可以设置一个游戏等级窗口，来改变自己刚开始进入游戏时难度等级；
- 2、子弹的射程并非全屏，而大概是屏幕长度的80%；
- 3、消灭飞机需要1发子弹，中飞机需要8发子弹，大飞机需要20发子弹；
- 4、每消灭一架飞机得1000分，中飞机4000分，打飞机10000分；
- 5、每隔30秒有一个随机的道具补给，分为两种道具，全屏炸弹和双倍子弹；
- 6、全屏炸弹最多只能存放3枚，双倍子弹可以维持时间为18秒；
- 7、游戏将根据分数来逐渐提高难度。难度的提高表现为飞机数量的增多以及速度的加快。

- 循环播放只需要设置 `pygame.mixer.music.play(-1)` 将其中的值设为 -1即可
- traceback模块介绍：

- 1、`traceback.print_exc()`
- 2、`traceback.format_exc()`
- 3、`traceback.print_exception()`

1、`print_exc()`：是对异常栈输出

2、`format_exc()`：是把异常栈以字符串的形式返回，

`print(traceback.format_exc())` 就相当于 `traceback.print_exc()`

3、`print_exception(): traceback.print_exc()` 实现方式就是
`traceback.print_exception(sys.exc_info())`，可以点
`sys.exc_info()` 进去看看实现

- `sprite` 模块里有一个叫做 `collide_mask` 的函数可以利用，这个函数要求检测对象拥有一个 `mask` 属性，这个属性就是用于指定检测的范围，关于 `mask`，`Pygame` 还专门写了一个 `mask` 模块，其中有一个叫做 `from_surface()` 的函数，可以将 `Surface` 对象中非透明的部分标记为 `mask` 并返回。

首先，给每个类加上 `mask` 属性：

```
self.mask = pygame.mask.from_sueface(self.image) #将非透明部分标记为mask
```

[飞机大战.md](#)